

Geethanjali College of Engineering and Technology

(UGC AUTONOMOUS INSTITUTION)

(Accredited by NBA and NAAC with 'A' grade, Approved by AICTE New Delhi and Affiliated to JNTUH)

Cheeryal (V), Keesara (M), Medchal (Dist), Telangana – 501 301.

COMPUTER NETWORKS LAB

(20CS31L02)

Laboratory Manual

III Year B.Tech. CS – I Semester



Geethanjali

DEPARTMENT OF CYBER SECURITY

2024-2025

Lab - Incharge

HOD-CSE

Dr.G.Kalyani
Associate Professor

Geethanjali College of Engineering and Technology
(Autonomous)

Cheeryal (V), Keesara (M), Medchal District, Telangana State– 501 301

DEPARTMENT OF CYBER SECURITY

Name of the Lab Course : Computer Networks Lab

(COURSE CODE: 20CS31L2)

Programme: UG

Branch: CSE

Version No: 02

Year: III

Document Number : GCET/CS/CN/01

Semester: I

No. of Pages: 109

Section: III – CS-A

Classification status (Unrestricted/Restricted) :

Distribution List:

Prepared by :

1) Name : Mrs.M.Alphonsa

2) Designation: Asst.Prof

3) Sign :

4) Date :

Updated by :

1)Name: Dr.M.Sreenu

2) Designation: Assoc.Prof

3) Sign :

4) Date :

Verified by :

1) Name :

2) Sign :

3) Design :

4) Date :

***For Q.C only**

1)Name :

2) Sign :

3) Design :

4) Date :

Approved by (HOD) :

1) Name : Dr.G.Kalyani

2) Sign :

3) Date :

Geethanjali College of Engineering and Technology

(UGC AUTONOMOUS INSTITUTION)

(Accredited by NBA and NAAC with 'A' grade, Approved by AICTE New Delhi and Affiliated to JNTUH)

Cheeryal (V), Keesara (M), Medchal (Dist), Telangana – 501 301.



Geethanjali

CERTIFICATE

This is to certify that Mr. / Miss _____

*has satisfactorily completed _____ number of experiments in the **Computer Networks Laboratory**.*

Roll No: _____

Branch: _____ Section: _____

Year: _____

Academic Year: _____

Head

Dept. of CSE

Faculty

In-charge

Internal Examiner

External Examiner

PERFORMANCE INDICATOR

[illegible]

Vision of the Institution

Geethanjali visualizes dissemination of knowledge and skills to students, who would eventually contribute to well being of the people of the nation and global community.

Mission of the Institution

1. To impart adequate fundamental knowledge in all basic sciences and engineering, technical and Inter-personal skills to students.
2. To bring out creativity in students that would promote innovation, research and entrepreneurship.
3. To Preserve and promote cultural heritage, humanistic and spiritual values promoting peace and harmony in society.

Vision of the Department

To produce globally competent and socially responsible computer science engineers contributing to the advancement of engineering and technology which involves creativity and innovation by providing excellent learning environment with world class facilities.

Mission of the Department

1. To be a centre of excellence in instruction, innovation in research and scholarship, and service to the stake holders, the profession, and the public.
2. To prepare graduates to enter a rapidly changing field as a competent computer science engineer.
3. To prepare graduate capable in all phases of software development, possess a firm understanding of hardware technologies, have the strong mathematical background necessary for scientific computing, and be sufficiently well versed in general theory to allow growth within the discipline as it advances.
4. To prepare graduates to assume leadership roles by possessing good communication skills, the ability to work effectively as team members, and an appreciation for their social and ethical responsibility in a global setting

PROGRAM EDUCATIONAL OBJECTIVES

1. To provide graduates with a good foundation in mathematics, sciences and engineering fundamentals required to solve engineering problems that will facilitate them to find employment in industry and / or to pursue postgraduate studies with an appreciation for lifelong learning.
2. To provide graduates with analytical and problem-solving skills to design algorithms, other hardware / software systems, and inculcate professional ethics, inter-personal skills to work in a multi-cultural team.
3. To facilitate graduates, get familiarized with state-of-the-art software / hardware tools, imbibing creativity and Innovation that would enable them to develop cutting-edge technologies of multi-disciplinary nature for societal development.

PROGRAM OUTCOMES (POs)

The Program outcomes of B.Tech (CS) department are being modified with reference to Graduate Attributes given by NBA in June 2015 and existing POS of B.Tech (CS) as below

PO1. Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

PO2. Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO3. Design/development of solutions : Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO4. Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO5. Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

PO6. The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO7. Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

PO8. Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO9. Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

PO10. Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write

effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO11 .Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO12. Life-long learning : Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

PROGRAM SPECIFIC OUTCOMES (PSOs)

PSO1: Demonstrate competency in Programming and problem solving skills and apply those skills in solving computing problems

PSO2: Select appropriate programming languages, Data structures and algorithms in combination with modern technologies and apply them in developing innovative solutions

PSO3: Apply cryptographic algorithms for ensuring cyber security as per cyber laws, demonstrate awareness of all the security related issues and employ state of the art technologies to protect the digital assets of an organization

Course Overview

GEETHANJALI COLLEGE OF ENGINEERING AND TECHNOLOGY

(Autonomous)

Cheeryal (V), Keesara (M), Medchal Dist., Telangana-501301

20CS31L2 – COMPUTER NETWORKS LAB

III Year. B.Tech. (CS) – I Sem

L	T	P/D	C
-	-	2/-	1

Prerequisite(s):

- 20CS11L1 - Programming for Problem Solving Lab
- 20CS21L3 - Object Oriented Programming through JAVA Lab

Course Objectives:

Develop ability to

1. Develop an understanding of modern network architectures from a design and performance perspective.
2. Understand the protocols of data link layer and MAC sub layer and apply different techniques of error detection and error correction.
3. Distinguish and explain different network layer protocols and routing algorithms.
4. Describe the functions of TCP and UDP protocols.
5. Illustrate the application layer protocols such as HTTP, FTP, SMTP, DNS and TELNET.

Course Outcomes (COs):

After completion of the course, student would be able to

- CO1. Identify the different types of network topologies, protocols and explain the layers of the OSI and TCP/IP model.
- CO2. Design a wide area networks (WANs), local area networks (LANs) and wireless LANs (WLANs) for a given requirement (small scale) based on the market available components and describe the protocols of data link layer and MAC Sub layer.
- CO3. Classify and compare the major routing protocols and congestion control algorithms.
- CO4. Develop a program for a given problem related to TCP/IP and UDP protocols using network programming.
- CO5. Analyze the application layer protocols using open source available software and tools.

List of Exercises:

- Week 1. Study of different types of Network cables and practically implement the cross-wired cable and straight through cable using Crimping tool.
- Week 2. Connect the computers in LAN, Study of basic network commands and network configuration commands.

- Week 3. Study of Network simulator tool and implement IP Address configuration in Network simulator tool.
- Week 4. Configure different network topologies using packet tracer/Network Simulator tool.
- Week 5.
- Write a program to implement the Data link layer framing methods such as character stuffing and bit stuffing.
 - Write a program to simulate Stop and wait protocol and Sliding Window Protocols.
- Week 6. Write a program to implement on a data set of characters using the three Cyclic Redundancy Check
- Week 7. Write a program to simulate Carrier Sense Multiple Access/Collision Detection (CSMA/CD) and Carrier Sense Multiple Access/Collision Avoidance (CSMA/CA).
- Week 8. Configure a network using Distance Vector Routing protocol using packet tracer tool.
- Week 9. Write a program to implement Client - Server communication for chat using Transmission Control Protocol (TCP).
- Week 10. Configure FTP Server on a Linux/Windows machine using a FTP client/SFTP client. Use a TFTP client and repeat the experiment
- Week 11. Install Telnet on one of the systems connected by a switch and telnet to it from the other system. Using Wireshark tool, capture the packets and analyze the TCP 3-way Handshake for connection establishment and tear down.
- Week 12. Using RSA Algorithm Encrypt a Text data and Decrypt the same.

Software's used:

- C/ Java/ Equivalent compiler.
- Network Simulator like NS2/NS3/CISCO Packet tracer tool/Wireshark tool.

Course Objectives

Develop ability to

1. Develop an understanding of modern network architectures from a design and performance perspective.
2. Understand the protocols of data link layer and MAC sub layer and apply different techniques of error detection and error correction.
3. Distinguish and explain different network layer protocols and routing algorithms.
4. Describe the functions of TCP and UDP protocols.
5. Illustrate the application layer protocols such as HTTP, FTP, SMTP, DNS and TELNET.

Course Outcomes (COs)

After completion of the course, student would be able to

CO1: Identify the different types of network topologies, protocols and understand the layers of the OSI and TCP/IP model. Understand the different protocols in Data link layer.

CO2: Have a basic knowledge of the use of Data link layer protocols.

CO3: Analyze, specify and design the topological and routing strategies for an IP based networking infrastructure.

CO4: Analyze the Datagrams in the Internet using Network Tools.

CO5: Have a working knowledge of Application layer protocols.

Mapping of Lab Course with Programme Educational Objectives

Course	Course Code	PEOs	POs & PSO
COMPUTER NETWORKS LAB	20CS31L2	PEO1, PEO2, PEO3	PO1,PO2,PO3,PO4,PO5, PO9 ,PO12,PSO1, PSO2, PSO3

Mapping of Lab Course outcomes with Programme outcomes:

Course Outcomes - Computer Networks (20CS31L2)	Program Outcomes and Program Specific Outcomes														
	1	2	3	4	5	6	7	8	9	10	11	12	PSO1	PSO2	PSO3
CO1. Identify the different types of network topologies, protocols and understand the layers of the OSI and TCP/IP model. Understand the different protocols in Data link layer.	2	2	2	2	2	-	-	-	1	-	-	1	1	2	1
CO2. Have a basic knowledge of the use of Data link layer protocols.	2	2	2	2	2	-	-	-	1	-	-	1	1	2	1
CO3. Analyze, specify and design the topological and routing strategies for an IP based networking infrastructure.	2	2	3	2	3	-	-	-	1	-	-	2	2	2	1
CO4. Analyze the Datagrams in the Internet using Network Tools.	2	2	3	2	3	-	-	-	1	-	-	2	1	2	1
CO5. Have a working knowledge of Application layer protocols.	2	2	2	2	2	-	-	-	1	-	-	1	1	2	1

Prerequisites

1. Programming for Problem Solving Lab
2. Object Oriented Programming through Java Lab

Evaluation details of Internal/External Examinations

Evaluation of Internal Marks:

a) 15 Marks are awarded for day to day work

- 1) Record and Observation book ----- 5Marks
- 2) Attendance and behavior of student ----- 5 Marks
- 3) Viva and performance -----5 Marks

b) 15 Marks are awarded for conducting laboratory test as follows:

- 1) Write up and program-----5 Marks
- 2) Execution of Program -----5 Marks
- 3) Viva and performance -----5 Marks

Evaluation of External Marks:

70 Marks are awarded for conducting laboratory test as follows:

- 1) Algorithm ----- 25 Marks.
- 2) Write up and program----- 15 Marks
- 3) Execution of Program ----- 15 Marks
- 4) Viva ----- 15 Marks

INSTRUCTIONS TO THE STUDENTS:

1. Students are required to attend all labs.

2. Students should be dressed in formals when attending the laboratory sessions.
3. Students will work individually in computer laboratories.
4. While coming to the lab bring the observation book and Work book etc.
5. Before coming to the lab, prepare the pre-lab questions. Read through the lab experiment to familiarize you.
6. Utilize 3 hours' time properly to perform the experiment and noting down the outputs.
7. If the experiment is not completed in the prescribed time, the pending work has to be done in the leisure hour or extended hours.
8. You will be expected to submit the completed work book according to the deadlines set up by your instructor.

INSTRUCTIONS TO LABORATORY TEACHERS:

1. Observation book and lab records submitted for the lab work are to be checked and signed before the next lab session.
2. Students should be instructed to switch ON the power supply after the connections are checked by the lab assistant / teacher.
3. The promptness of submission should be strictly insisted by awarding the marks accordingly.
4. Ask viva questions at the end of the experiment.
5. Do not allow students who come late to the lab class.
6. Encourage the students to do the experiments innovatively.
7. Fill continuous Evaluation sheet, on regular basis.
8. Ensure that the students are dressed in formals

COMPUTER NETWORKS LAB

Week 1. Study of different types of network cables and practically implement the cross-wired cable and straight through cable using clamping tool.

Objective: To Analyze the different types of network cables and use them in different communication networks.

Outcome: Student will able to create a Communication network using different types of network cables and Network devices.

Apparatus (Components): RJ-45 connector, Crimping Tool, Twisted pair Cable.

Procedure: To do these practical following steps should be done:

1. Start by stripping off about 2 inches of the plastic jacket off the end of the cable. Be very careful at this point, as to not nick or cut into the wires, which are inside. Doing so could alter the characteristics of your cable, or even worse render is useless. Check the wires, one more time for nicks or cuts. If there are any, just whack the whole end off, and start over.
2. Spread the wires apart, but be sure to hold onto the base of the jacket with your other hand. You do not want the wires to become untwisted down inside the jacket. Category 5 cable must only have 1/2 of an inch of 'untwisted' wire at the end; otherwise it will be 'out of spec'. At this point, you obviously have ALOT more than 1/2 of an inch of un-twisted wire.
3. You have 2 end jacks, which must be installed on your cable. If you are using a pre-made cable, with one of the ends whacked off, you only have one end to install - the crossed over end. Below are two diagrams, which show how you need to arrange the cables for each type of cable end. Decide at this point which end you are making and examine the associated picture below.

RJ45 Pin # (END 1)	Wire Color	Diagram End #1	RJ45 Pin # (END 2)	Wire Color	Diagram End #2
1	White/Orange		1	White/Green	
2	Orange		2	Green	
3	White/Green		3	White/Orange	
4	Blue		4	White/Brown	
5	White/Blue		5	Brown	
6	Green		6	Orange	
7	White/Brown		7	Blue	
8	Brown		8	White/Blue	

fig.1 Diagram shows you how to prepare Cross wired connection

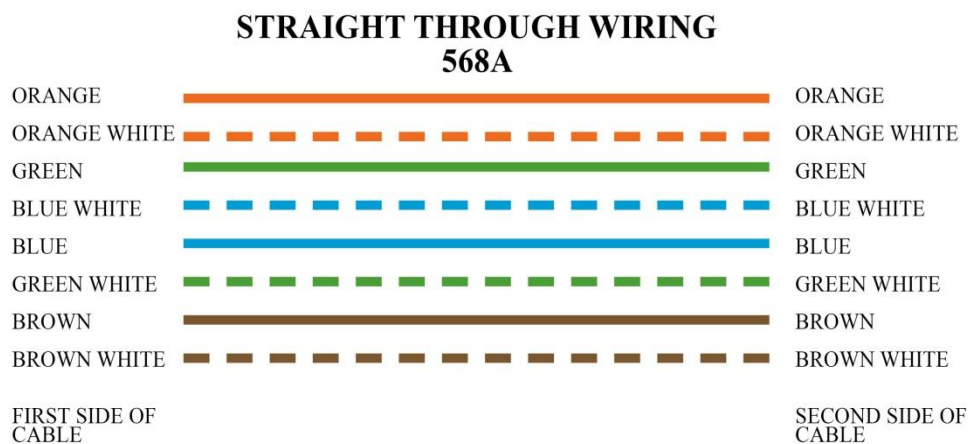
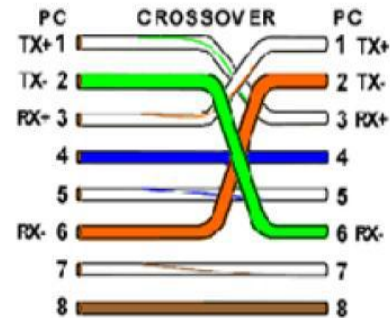
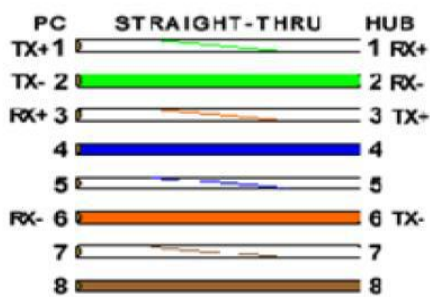


fig 2. Diagram shows you how to prepare straight through wired connection

Cable crimping Steps:

1. Remove the outmost vinyl shield for 12mm at one end of the cable (we call this side A-side).
2. Arrange the metal wires in parallel.
3. Insert the metal wires into RJ45 connector on keeping the metal wire arrangement.
4. Set the RJ45 connector (with the cable) on the pliers, and squeeze it tightly.
5. Make the other side of the cable (we call this side B-side) in the same way.
6. After you made it, you don't need to take care of the direction of the cable.



Types of Network Cables:

Twisted Pair Cable:

Twisted pair cable is a kind of wiring in which two conductors of a single circuit are twisted together, with one conductor serving as the forward circuit and the other as the return circuit. The twisted cabling design helps to reduce noise from outside sources and crosstalk on multi-pair cables, making it suitable for use in the field of telecommunication for a long time.

Twisted pair cable comes in two versions (image shown below): unshielded twisted-pair (UTP) and shielded twisted-pair (STP). Since UTP cable lacks shielding, it's more susceptible to interference from electronic devices. Or in another way, STP cable consists of a foil jacket which helps to prevent crosstalk and noise from outside source. STP is commonly used in Token Ring networks and UTP is in Ethernet networks.

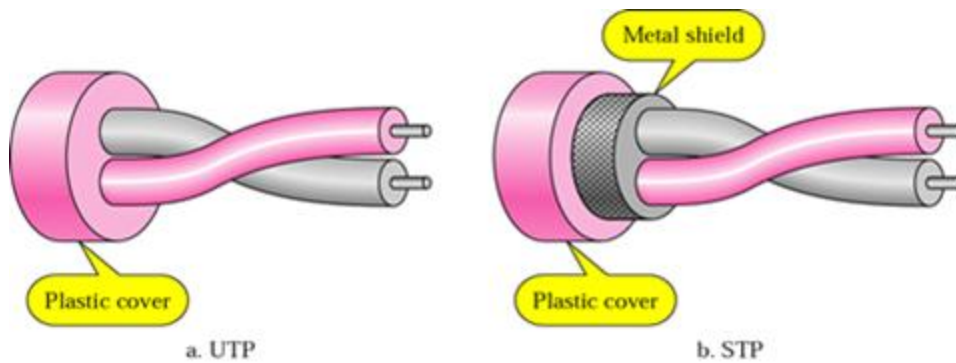


Fig. Twisted pair cable

Co-axial Cable:

Co-axial cable, or sometimes referred to as coax, is a type of cable that has four layers. That is, at the core is an inner conductor which is surrounded by a foam insulation, symmetrically surrounded by a woven braided metal shield, then covered in a plastic jacket (just as the following picture shows). Because of its insulating property, coaxial cable can carry analogy signals with a wide range of frequencies. It's seldom used in computer networks anymore, but widely used in cable television services and video connections, like those used by closed circuit surveillance systems.

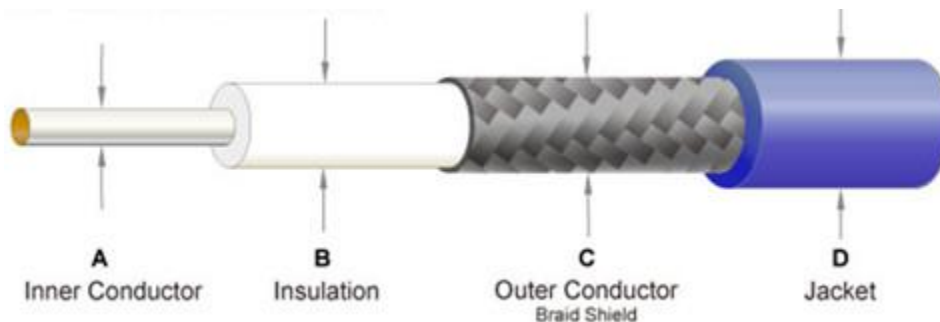


Fig. Co-axial cable

Fiber Optic Cable:

Fiber optic cable is the complete assembly of fibers, and uses light pulses to transmit information down fiber lines instead of using electronic pulses to transmit information down copper lines. Each of the optical fiber elements is individually coated by plastic layers and contained in a protective tube. A fiber optic cable can accommodate many wavelengths (or channels), able to accommodate ever-increasing data capacity requirements. When terminated with LC/SC/ST/FC/MTRJ/MU/SMA connectors on both ends, such as LC-LC, LC-ST, SC-SC, ST-ST, LC SC cable, fiber optic cables can achieve fiber link connection between equipment during fiber cabling.



Fig. Fiber optic cable

Crimping tool:

A crimping tool is a device used to conjoin two pieces of metal by deforming one or both of them in a way that causes them to hold each other. The result of the tool's work is called a **crimp**. A good example of crimping is the process of affixing a connector to the end of a cable.

RJ-11 (6-Pin) and RJ-45 (8-Pin) Crimping Tool



Fig. Crimping Tool

Ethernet Cable Tips:

1. A straight-thru cable has identical ends.
2. A crossover cable has different ends.
3. A straight-thru is used as a patch cord in Ethernet connections.
4. A crossover is used to connect two Ethernet devices without a hub or for connecting two hubs.
5. A crossover has one end with the Orange set of wires switched with the Green set.

6. Odd numbered pins are always striped; even numbered pins are always solid colored.
7. Looking at the RJ-45 with the clip facing away from you, Brown is always on the right, and pin 1 is on the left.
8. No more than 1/2" of the Ethernet cable should be untwisted otherwise it will be susceptible to crosstalk.
9. Do not deform, do not bend, do not stretch, do not staple, do not run parallel with power cables, and do not run Ethernet cables near noise inducing components.

Discussion Topics:

1. Define Crimping tool.

A crimping tool is a device used to conjoin two pieces of metal by deforming one or both of them in a way that causes them to hold each other. The result of the tool's work is called a crimp. A good example of crimping is the process of affixing a connector to the end of a cable.

2. What is Network Cabling? List types of Network Cables.

Cable is the medium through which information usually moves from one network device to another. Types of Network cables:

- Unshielded Twisted Pair (UTP) Cable
- Shielded Twisted Pair (STP) Cable
- Coaxial Cable
- Fiber Optic Cable.

3. What is the Use of Registered Jack 45(RJ45) Connector?

Registered Jack 45 (RJ45) is a standard type of physical connector for network cables. RJ45 connectors are most commonly seen with Ethernet cables and networks. Modern Ethernet cables feature small plastic plugs on each end that are inserted into the RJ45 jacks of Ethernet devices.

4. What is the purpose of Crossover Ethernet Cable?

The purpose of a Crossover Ethernet cable is to directly connect one computer to another computer (or device) without going through a router, switch or hub.

5. Compare Shielded Twisted Pair (STP) and Unshielded Twisted Pair(UTP).

STP cable is also divided by overall shield and individual shield. Individual shielded twisted pair is with aluminum foil for each twisted pair or quad. This type of shielding protects cable from external

electromagnetic interference (EMI) entering or exiting the cable and also protects neighboring pairs from crosstalk. Overall shielded twisted pair is with overall foil or braided shield across all of the pairs within the 100 Ω twisted pair cable. This type of shielding helps prevent EMI from entering or exiting the cable. One STP cable can have both overall and individual shielding.

UTP cable without shielding is more prone to outside interference. For this reason, this cable type is more often found in indoor telephone applications. Outdoor telephone cables contain hundreds or thousands pairs. Pairs that have the same twisted rate within the cable can experience some degree of crosstalk, so wire pairs are usually selected carefully within a large cable to reduce the crosstalk. Most UTP cable uses RJ45 connectors, which look like telephone connectors (RJ11) but have eight wires instead of four.

6. Differentiate between Fast Ethernet and Gigabit Ethernet.

- The data rate provided by fast ethernet is up to 100 Mbps. On the other hand, gigabit- ethernet offers up to 1 Gbps speed.
- As the speed is increased in Gigabit ethernet, so the delay is decreased whereas the fast ethernet generates more delay.
- The configuration of fast ethernet is simpler than gigabit ethernet.
- The distance covered by fast ethernet is at most 10 km. On the contrary, the gigabit ethernet covers 70 km.
- Round trip delay of fast ethernet is 100-500 bit times. As against, gigabit ethernet has the delay of 4000 bit times.

Week 2. Connect the computers in LAN, Study of basic network commands and network configuration commands.

Objective: To show how the computers are connected in LAN and Analyze the Host information, Packet Information and details of network by using different network commands and network configuration commands.

Outcome: Student will able to Connect Computers in LAN and implement basic network commands and configuration commands.

Procedure:

Procedure to connect the computers in LAN:

On the host computer, follow these steps to share the Internet connection:

1. Log on to the host computer as Administrator or as Owner.
2. Click Start, and then click Control Panel.
3. Click Network and Internet Connections.
4. Click Network Connections.
5. Right-click the connection that you use to connect to the Internet. For example, if you connect to the Internet by using a modem, right-click the connection that you want under Dial-up/other network available.
6. Click Properties.
7. Click the Advanced tab.
8. Under Internet Connection Sharing, select the Allow other network users to connect through this computer's Internet connection check box.
9. If you are sharing a dial-up Internet connection, select the Establish a dial-up connection whenever a computer on my network attempts to access the Internet check box if you want to permit your computer to automatically connect to the Internet.
10. Click OK. You receive the following message: When Internet Connection Sharing is enabled; your LAN adapter will be set to use IP address 192.168.0.1. Your computer may lose connectivity with other computers on your network. If these other computers have static IP addresses, it is a good idea to set them to obtain their IP addresses automatically. Are you sure you want to enable Internet Connection Sharing.
11. Click Yes.

The connection to the Internet is shared to other computers on the local area network (LAN). The network adapter that is connected to the LAN is configured with a static IP address of 192.168.0.1 and a subnet mask of 255.255.255.0

On the client computer:

To connect to the Internet by using the shared connection, you must confirm the LAN adapter IP configuration, and then configure the client computer.

To confirm the LAN adapter IP configuration, follow these steps:

1. Log on to the client computer as Administrator or as Owner.
2. Click Start, and then click Control Panel.
3. Click Network and Internet Connections.
4. Click Network Connections.
5. Right-click Local Area Connection and then click Properties.
6. Click the General tab, click Internet Protocol (TCP/IP) in the connection uses the following items list, and then click Properties.
7. In the Internet Protocol (TCP/IP) Properties dialog box, click Obtain an IP address automatically (if it is not already selected), and then click OK.

Note: You can also assign a unique static IP address in the range of 192.168.0.2 to 254. For example, you can assign the following static IP address, subnet mask, and default gateway.

8. IP Address 192.168.31.202
9. Subnet mask 255.255.255.0
10. Default gateway 192.168.31.1
11. In the Local Area Connection Properties dialog box, click OK.
12. Quit Control Panel.

Create a LAN Connection using two computers with a LAN Cable:

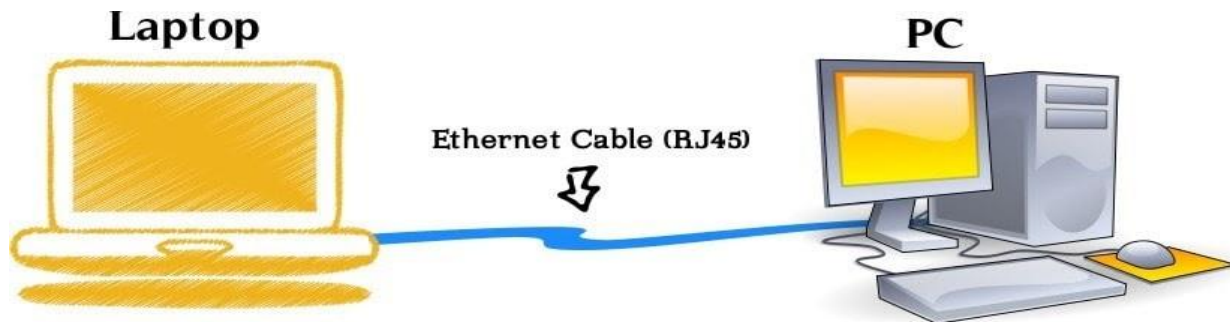
Setup Two Computers Via LAN For Sharing:

The first thing you'll have to make sure is, you do have a crossover cable in your hand to connect to two computers **because cross-over and straight cable both looks same but works differently.** Mainly **to connect two different networking devices** we need to use a **straight Ethernet cable** and **to connect two same devices** we need to use a **crossover Ethernet cable**, and the actually different in these two different kinds of cables is the internal wire connection.

So, make sure you have a **crossover cable** because it's the only way to connect two computers without a modem or switch, or otherwise you can use a straight cable if both of your computers is connected to the **network device** like a **Router, Network Switch** or a **Modem**.

Step 1 – Creating A Physical Connection:

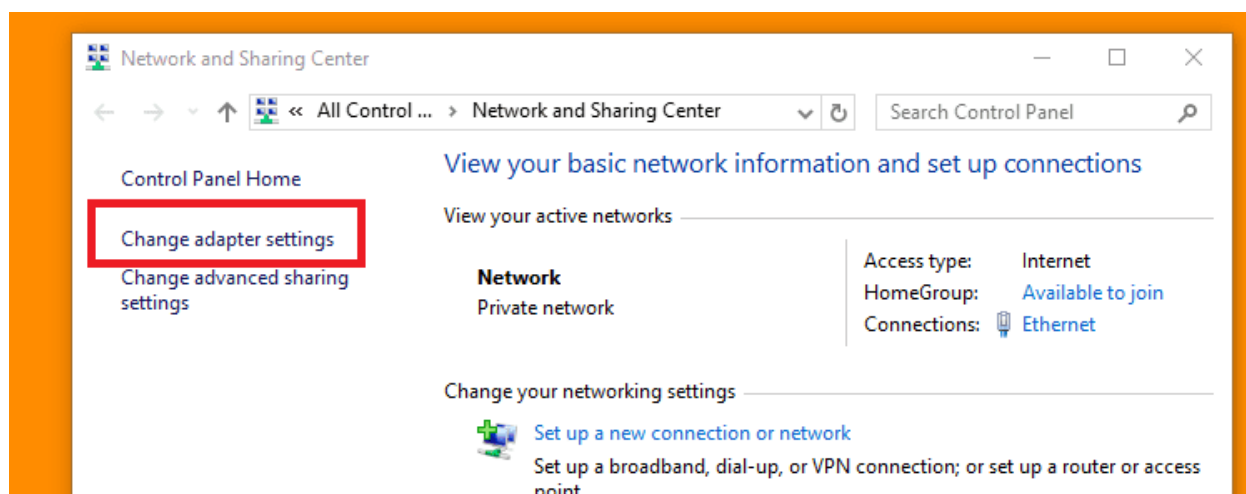
Now the first thing what you'll need to do is to connect each end of your **LAN** cable into both of your computer's **Ethernet Port** to connect both computers with each other. If you're using a **modem** or **router switch** then make sure everything is connected properly.



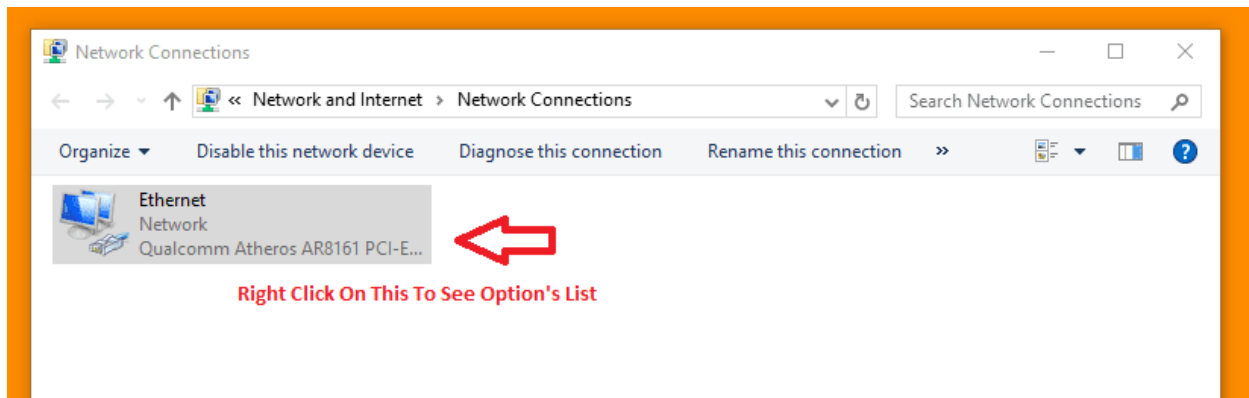
Step 2 –LAN Connection Setting

After connecting everything now it's time to setup an **IP address** in both your computers to make your computer to identify the connection. You'll just need to follow this same procedure in both of your computers or maybe laptops.

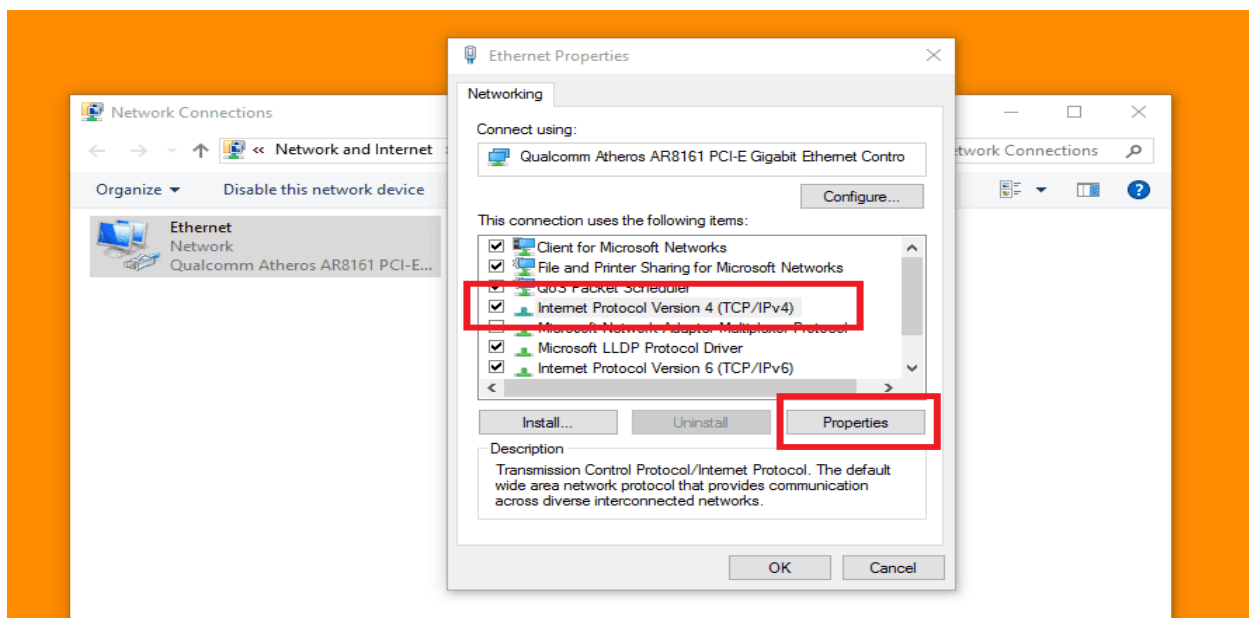
- Open your **Windows Control Panel**, or you can search for “**Network and Sharing Center**” in the **Start Menu**.
- Select the **Network and Sharing Center** option.
- From the left sidebar, click on “**Change Adapter Settings**” option.



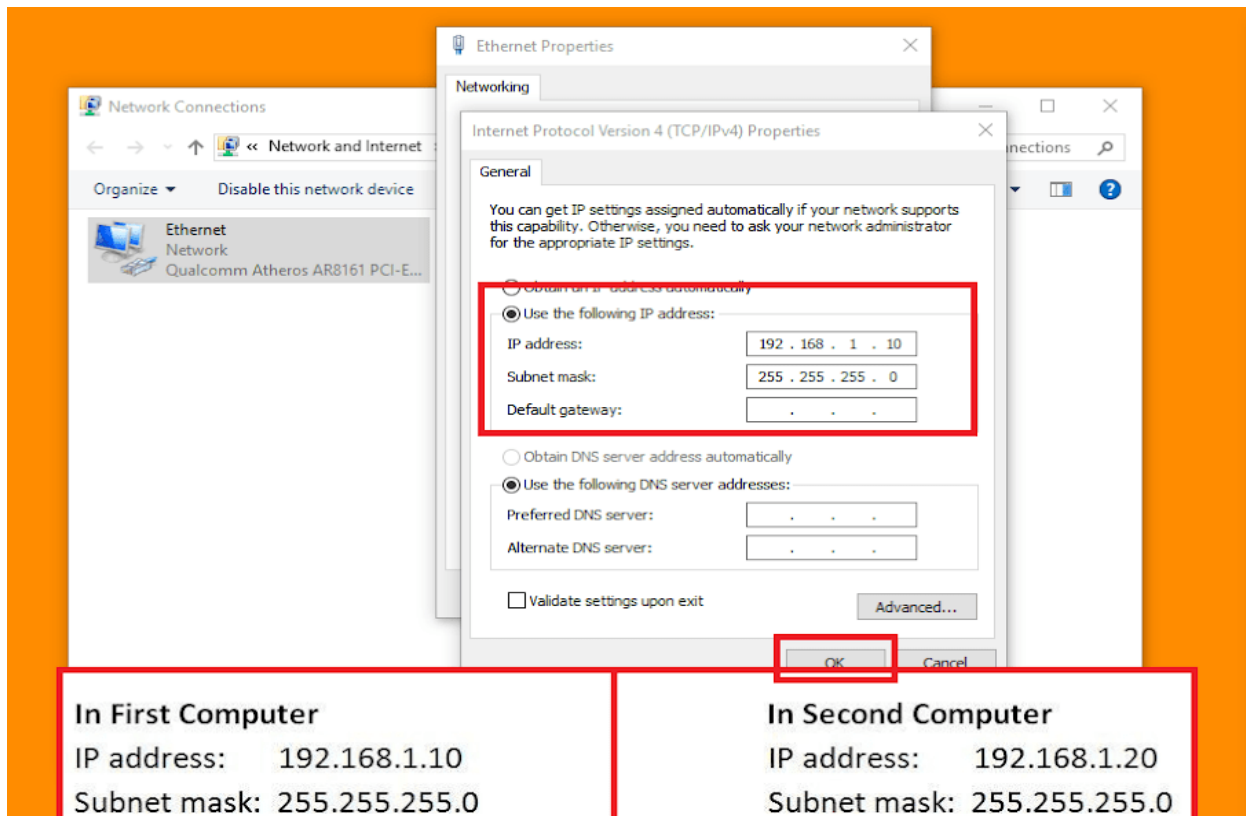
- After that, open your **“Local Area Connection”** or I say **Ethernet Adaptor’s** properties.



- Now select the **“Internet Protocol Version 4 (TCP/IPv4)”** option and then click on its **“Properties”** button.



- Now when the **“Internet Protocol Version 4 (TCP/IPv4) properties”** windows get popped up. Enter the IP address manually as shown in the picture given below and click on the **“Ok”** button after entering it.



This IP address given the picture is only going to work when you connect both of your computers via crossover cable.

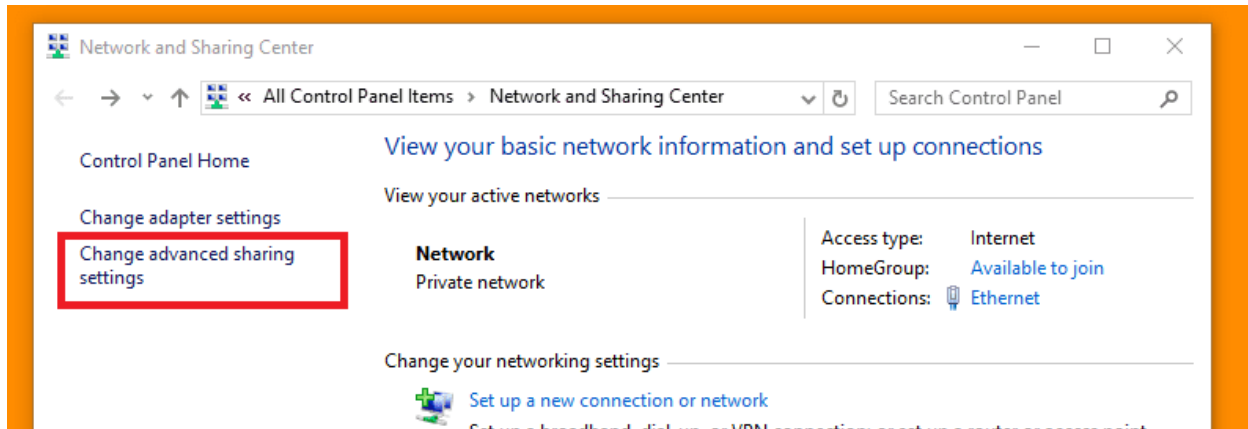
NOTE: If you have a **modem** or **WIFI router**, then you'll have to add the gateway IP as well. In that case, your **IP** should need to look like this –

In First Computer	In Second Computer	For Third, Fourth And So On
IP address: 192.168.1.10	IP address: 192.168.1.20	change the last two digit no. and it
Subnet mask: 255.255.255.0	Subnet mask: 255.255.255.0	will be differ for all different
Default gateway: 192.168.1.1	Default gateway: 192.168.1.1	computers

Step 3 – Change Default Sharing Setting:

After entering the **IP Address** in both of your computers, now you'll need to change the sharing setting of your computers to permit **access rights** for another computer.

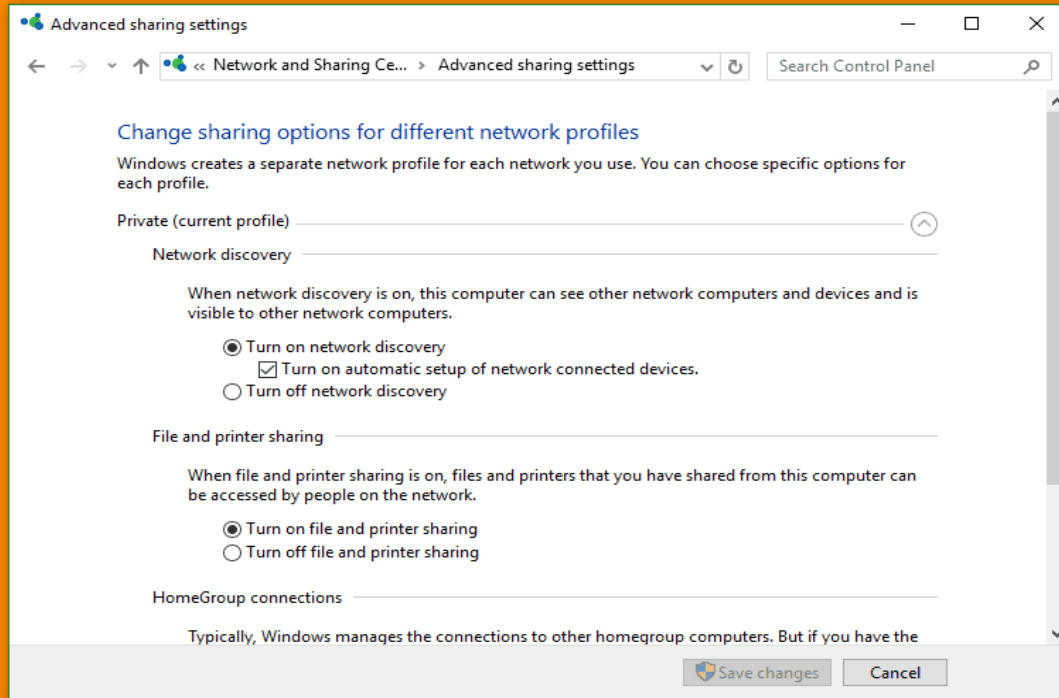
To do that, first, you'll again have to go back to the Network and Sharing Center settings page of your Windows and then select the “**Advanced sharing settings**” option to see all your computers network sharing setting.



After when the sharing setting window gets opened, now you'll need to change your network setting to those options as shown listed below:

- **Turn on** network discovery
- **Turn on** file and printer sharing
- **Allow** windows to manage homegroup connection (recommended)
- **Turn on** sharing so anyone with network access can read and write files in the public folders
- **Use 128-bit** encryption to help protect files sharing connections (recommended)
- **Turn off** password protected sharing.

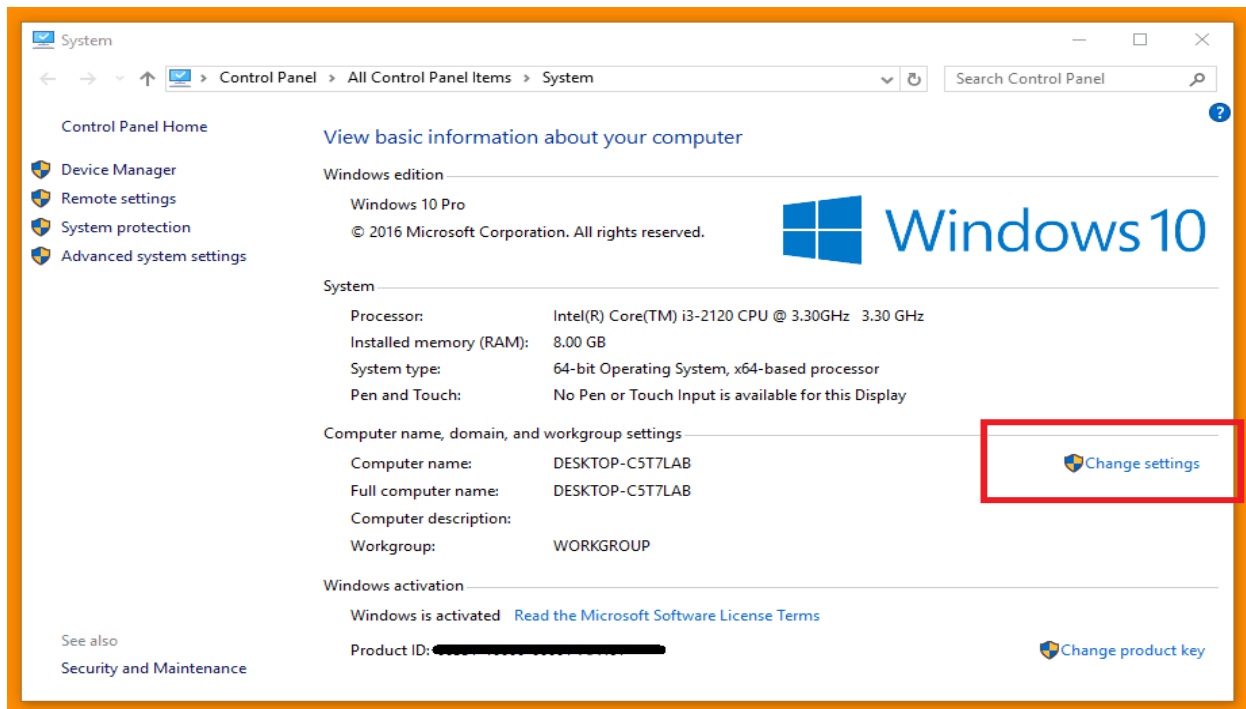
Make these changes in all network profiles;



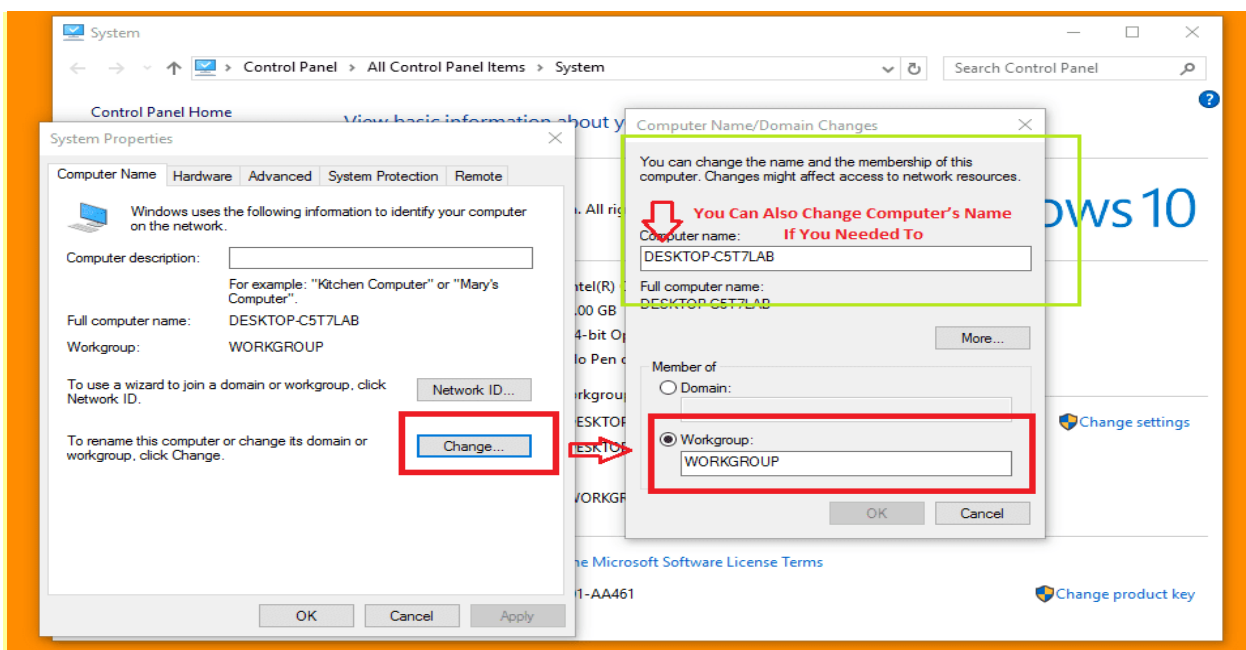
Step 4 – Change Default Workgroup Setting

By the use of this setting, you can create the same **workgroup** for both computers to start sharing, by connecting both of them in the same workgroup. **Workgroup setting** must be the same in both your computers but the computer's name should be different to identify both.

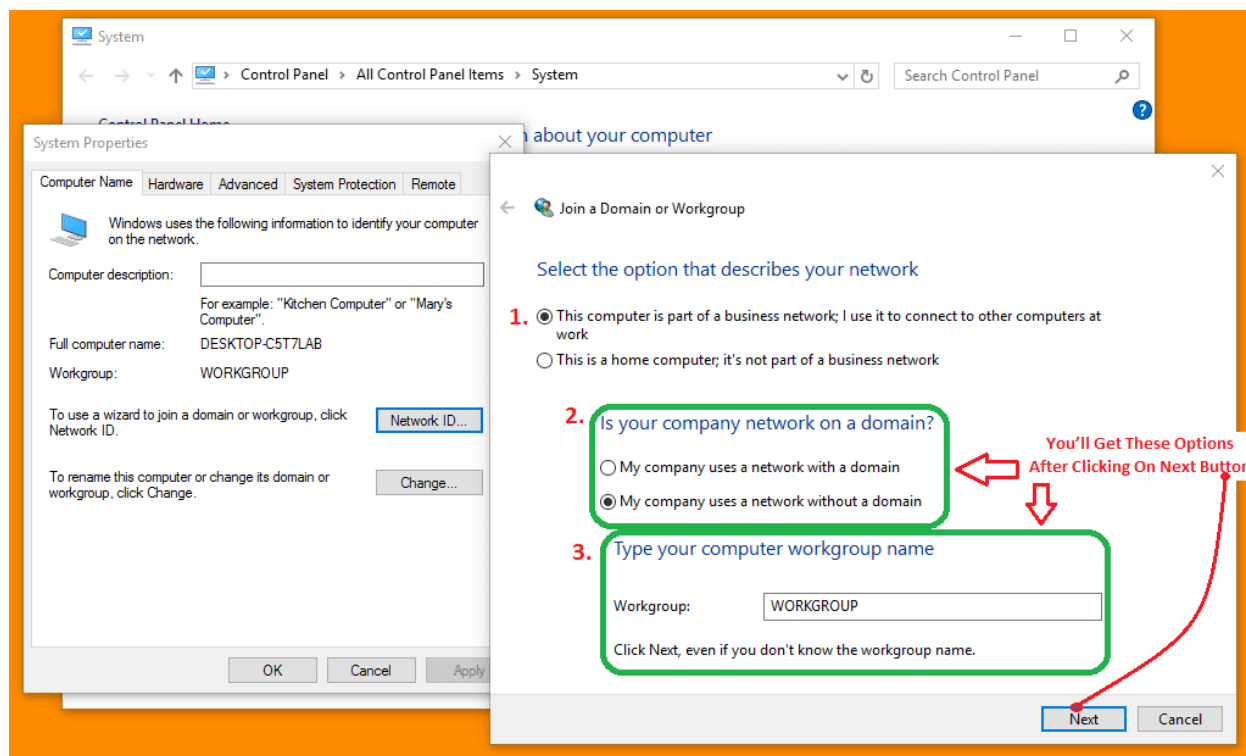
- First, you'll have to Right click on Computer and then select properties option. (or) press **Windows + Pause Break** Key on your keyboard to open your system's properties window.
- After that, click on "**Change setting**" in workgroup settings category as shown in the picture given below.



Now in the System properties, you'll need to click on the change button, and then you will be able to see the other options for changing your **Computer's Name**. So first, click on the "Change" button then type "WORKGROUP" in the 'Workgroup:' options and then click on "ok". After that, you'll have to do the same thing again on your different computer which you wanted to connect (or maybe on all computers of your LAN sharing computers).



After that click on “**Network ID**” and do this manual setting as given in the picture given below:



After that click on **Finish** and restart both of your computers.

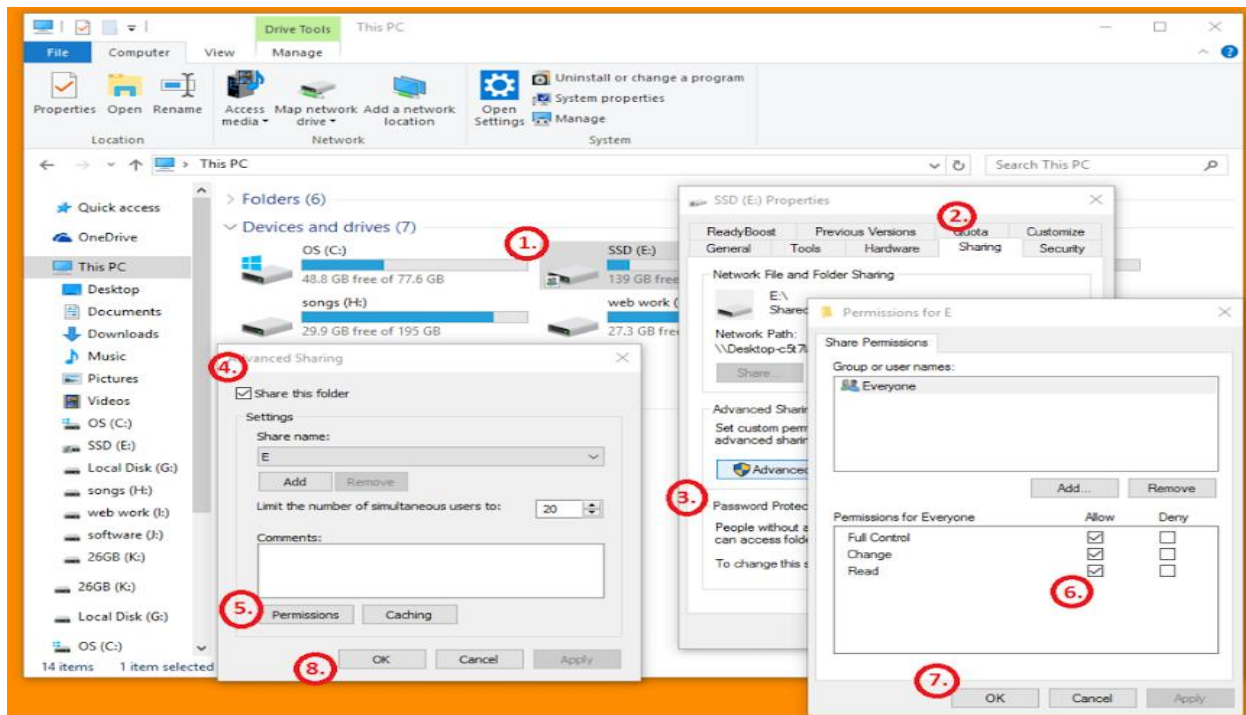
Step 5- At Last, Turn On Drive Sharing:

Now you will be able to see your both computer in your windows network place location and to share your drive via network you'll just need to Turn On the drive sharing by making some changes in that drive's properties.

Here's the list of some steps what you can follow to turn on the drive sharing in your computer or laptop:

- Right click on your drive and select **Properties**.
- Then, select “**Sharing**”.
- Then, click on “**Advanced Sharing**”.
- Then, tick the “**Share this folder**”.

- Then, click on “**Permissions**”.
- Then, tick on “**Full control**”.
- Then, click on “**OK**”.
- At last, click on “**OK**” again and that’s it.



Result: LAN Connection created successfully using Two Computers with a LAN cable.

Basic network commands and network configuration commands:

C:>ping

Ping is the most basic TCP/IP command, and it's the same as placing a phone call to your best friend. You pick up your telephone and dial a number, expecting your best friend to reply with "Hello" on the other end. Computers make phone calls to each other over a network by using a Ping command. The Ping commands main purpose is to place a phone call to another computer on the network, and request an answer. Ping has 2 options it can use to place a phone call to another computer on the network. It can use the computers name or IP address.

C:>ipconfig

The ipconfig command displays information about the host (the computer your sitting at)computer TCP/IP configuration.

C:>ipconfig /all

This command displays detailed configuration information about your TCP/IP connection including Router, Gateway, DNS, DHCP, and type of Ethernet adapter in your system.

C:>ipconfig /release

This command allows you to drop the IP lease from the DHCP server.

C:>ipconfig /renew

Using this command will renew all your IP addresses that you are currently (leasing) borrowing from the DHCP server. This command is a quick problem solver if you are having connection issues, but does not work if you have been configured with a static IP address.

C:\>nbtstat -a

This command helps solve problems with NetBIOS name resolution. (Nbt stands for NetBIOS over TCP/IP) Definitions

C:\>arp -a

it is short for address resolution protocol, It will show the IP address of your computer along with the IP address and MAC address of your router.

C:\>hostname:

This is the simplest of all TCP/IP commands. It simply displays the name of your computer.

C:\>Ipconfig /renew:

Using this command will renew all your IP addresses that you are currently (leasing) borrowing from the DHCP server. This command is a quick problem solver if you are having connection issues, but does not work if you have been configured with a static IP address.

C:\>Ipconfig /release:

This command allows you to drop the IP lease from the DHCP server.

C:\>ipconfig /flushdns:

This command is only needed if you're having trouble with your network's DNS configuration. The best time to use this command is after network configuration frustration sets in, and you really need the computer to reply with flushed.

C:\>nbtstat -a:

This command helps solve problems with NetBIOS name resolution. (Nbt stands for NetBIOS over TCP/IP) Definitions.

C:\>netdiag:

Netdiag is a network testing utility that performs a variety of network diagnostic tests, allowing you to pinpoint problems in your network. Netdiag isn't installed by default, but can be installed from the Windows XP CD after saying no to the install. Navigate to the CD ROM drive letter and open the support\tools folder on the XP CD and click the setup.exe icon in the support\tools folder.

C:\>netstat:

Netstat displays a variety of statistics about a computer's active TCP/IP connections. This tool is most useful when you're having trouble with TCP/IP applications such as HTTP, and FTP.

C:\>nslookup:

nslookup is used for diagnosing DNS problems. If you can access a resource by specifying an IP address but not its DNS you have a DNS problem.

C:\>pathping:

Pathping is unique to Windows, and is basically a combination of the Ping and Tracert commands. Pathping traces the route to the destination address then launches a 25 second test of each router along the way, gathering statistics on the rate of data loss along each hop.

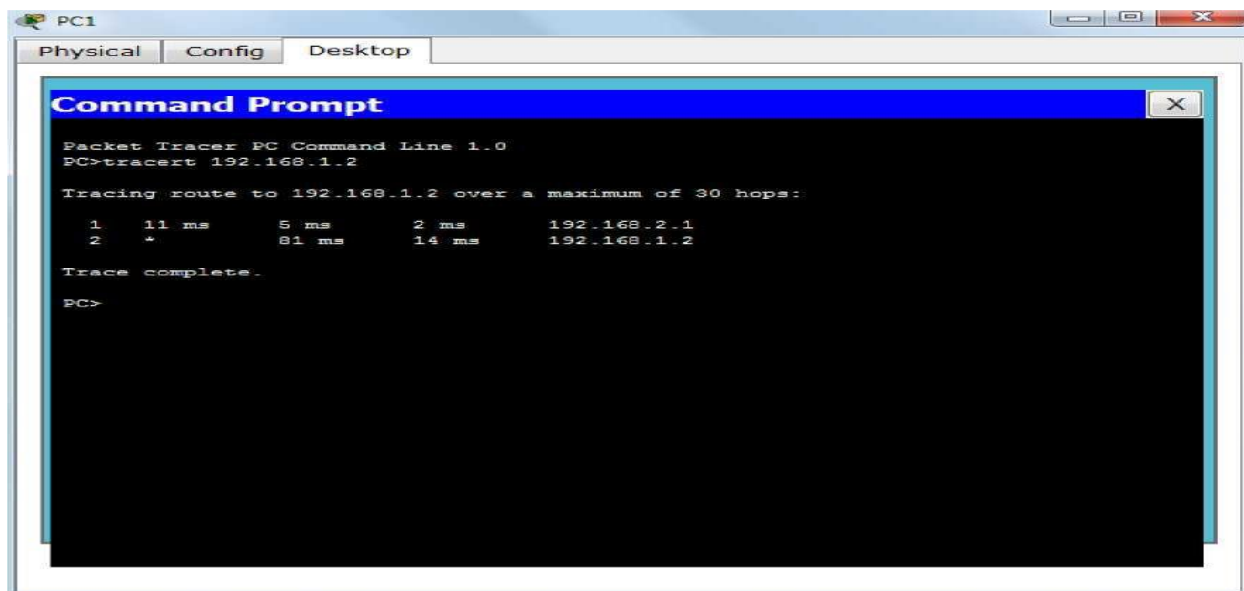
C:\>route:

The route command displays the computers routing table. A typical computer, with a single network interface, connected to a LAN, with a router is fairly simple and generally doesn't pose any network problems. But if you're having trouble accessing other computers on your network, you can use the route command to make sure the entries in the routing table are correct.

C:\>tracert:

The tracert command displays a list of all the routers that a packet has to go through to get from the computer where tracert is run to any other computer on the internet.

Input/ Output:



The screenshot shows a Packet Tracer PC Command Line window for PC1. The window has tabs for Physical, Config, and Desktop. The Command Prompt is open, displaying the output of the tracert command. The output shows the route from PC1 to 192.168.1.2, with two hops. The first hop is 192.168.2.1 with a delay of 11 ms. The second hop is 192.168.1.2 with a delay of 81 ms. The trace is complete.

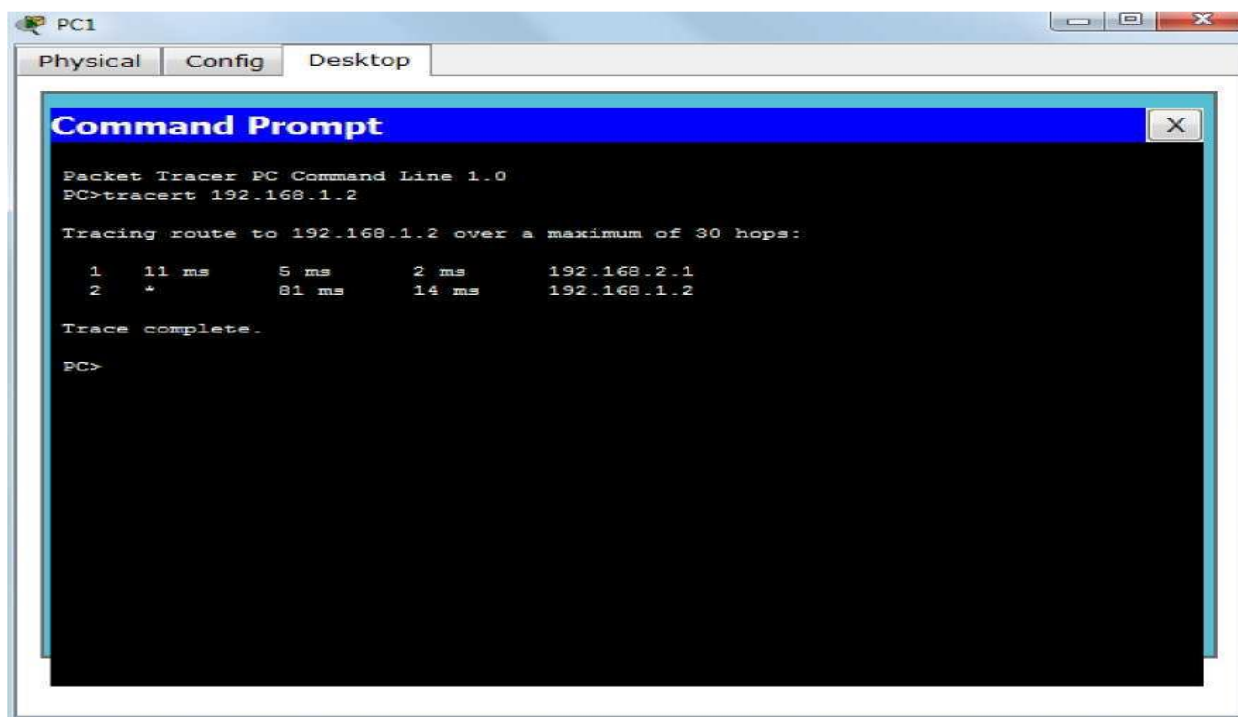
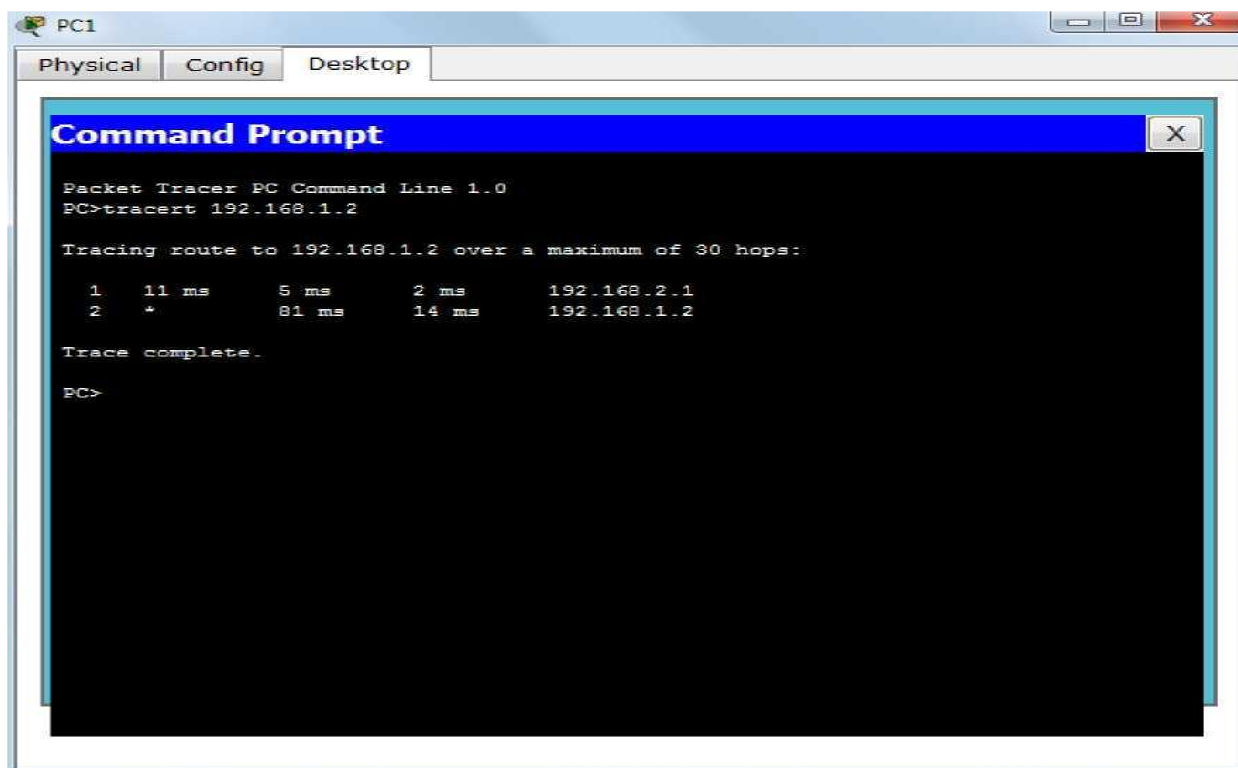
```
Packet Tracer PC Command Line 1.0
PC>tracert 192.168.1.2

Tracing route to 192.168.1.2 over a maximum of 30 hops:

  0  11 ms    5 ms     2 ms     192.168.2.1
  1  *         81 ms    14 ms    192.168.1.2

Trace complete.

PC>
```



Discussion Topics:

1. What are ipconfig and ifconfig?

Ipconfig stands for Internet Protocol Configuration and this command is used on Microsoft Windows to view and configure the network interface.

The command ipconfig is useful for displaying all TCP/IP network summary information currently available on a network. It also helps to modify the DHCP protocol and DNS setting.

Ifconfig (Interface Configuration) is a command that is used on Linux, Mac, and UNIX operating system. It is used to configure, control the TCP/IP network interface parameters from CLI i.e. Command Line Interface. It allows you to see the IP addresses of these network interfaces.

2. What is the purpose of DHCP?

DHCP stands for Dynamic Host Configuration Protocol and it automatically assigns IP addresses to the network devices. It completely removes the process of manual allocation of IP addresses and reduces the errors caused due to this. This entire process is centralized so that TCP/IP configuration can also be completed from a central location. DHCP has “pool of IP addresses” from which it allocates the IP address to the network devices. DHCP cannot recognize if any device is configured manually and assigned with the same IP address from the DHCP pool.

3. Compare LAN, WAN, MAN?

There are 4 major types of network.

- **Personal Area Network (PAN):** It is a smallest and basic network type that is often used at home. It is a connection between the computer and another device such as phone, printer, modem tablets etc
- **Local Area Network (LAN):** LAN is used in small offices and internet cafe to connect a small group of computers to each other. Usually, they are used to transfer a file or for playing the game in a network.
- **Metropolitan Area Network (MAN):** It is a powerful network type than LAN. The area covered by MAN is a small town, city etc. A huge server is used to cover such a large span of area for connection.
- **Wide Area Network (WAN):** It is more complex than LAN and covers a large span of area typically a large physical distance. The Internet is the largest WAN which is spread across the world. WAN is not owned by any single organization but it has distributed ownership.

4. What is the use of nslookup and Tracert Command?

nslookup : Displays information from Domain Name System (DNS) name servers.

Traceroute:

Tracert is a command which can show you the path a packet of information takes from your computer to one you specify. It will list all the routers it passes through until it reaches its destination, or fails to and is discarded. In addition to this, it will tell you how long each 'hop' from router to router takes.

5. Distinguish between multicasting and broadcasting.

in the **broadcast**, the packet is forwarded to all the hosts connected to the network whereas, in **multicast**, the packet is forwarded only to intended recipients.

In **broadcast**, the transmission of a packet is one-to-all whereas, in **multicast** the transmission of a packet is one-to-many. No group management is required in **broadcasting** whereas; the group management is required in **multicasting** to define the networks in which at least one host is interested in receiving the packet.

Week 3: Study of Network simulator tool and implement IP Address configuration in Network simulator tool.

Network simulator:

The network simulator is discrete event packet level simulator. The network simulator covers a very large number of applications of different kind of protocols of different network types consisting of different network elements and traffic models. Network simulator is a package of tools that simulates behavior of networks such as creating network topologies, log events that happen under any load, analyze the events and understand the network. Well the main aim of our first experiment is to learn how to use network simulator and to get acquainted with the simulated objects and understand the operations of network simulation and we also need to analyze the behavior of the simulation object using network simulation.

Platform required to run network simulator

- Unix and Unix like systems
- Linux (Use Fedora or Ubuntu versions)
- Free BSD
- SunOS/Solaris
- Windows 95/98/NT/2000/XP

Backend Environment of Network Simulator

Network Simulator is mainly based on two languages. They are C++ and OTcl. OTcl is the object oriented version of Tool Command language. The network simulator is a bank of of different network and protocol objects. C++ helps in the following way:

- It helps to increase the efficiency of simulation.
- Its is used to provide details of the protocols and their operation.
- It is used to reduce packet and event processing time.

OTcl helps in the following way:

- With the help of OTcl we can describe different network topologies
- It helps us to specify the protocols and their applications

Initialization

To start a new simulator we write

1. set ns [new Simulator]

From the above command we get that a variable ns is being initialized by using the set command.

Here

the code [new Simulator] is a instantiation of the class Simulator which uses the reserved word 'new'.

So we can call all the methods present inside the class simulator by using the variable ns.

Creating the output files

1 #To create the trace files we write

2 set tracefile1 [open out.tr w]

3 \$ns trace-all \$tracefile1

#To create the nam files we write

1. set namfile1 [open out.nam w]

2. \$ns namtrace-all \$namfile

01# Defining the 'finish' procedure'

```

02#
03 proc finish {} {
04 global ns tracefile1 namfile1
05 $ns flush-trace
06 close $tracefile
07 close $namfile
08 exec nam out.nam &
09 exit 0
10 }

```

In ns we end the program by calling the 'finish' procedure

```

1 #end the program
2 $ns at 125.0 "finish"

```

Thus the entire operation ends at 125 seconds. To begin the simulation we will use the command

```

1 #start the the simulation process
2 $ns run

```

Defining nodes, links, queues and topology

Way to create a node:

view source

print?

```
1 set n0 [ns node]
```

In the above we created a node that is pointed by a variable n0. While referring the node in the script we use \$n0. Similarly we create another node n2. Now we will set a link between the two nodes.

```
1 $ns duplex-link $n0 $n2 10Mb 10ms DropTail
```

So now we will define the buffer capacity of the queue related to the above link

```

1 #Set queue size of the link
2 $ns queue-limit $n0 $n2 20

```

so, if we summarize the above three things we get

```

01 # create nodes
set n0 [$ns node]
04 set n1 [$ns node]
05 set n2 [$ns node]
06 set n3 [$ns node]
07 set n4 [$ns node]
08 set n5 [$ns node]
09
10 #create links between the nodes
11
12 $ns duplex-link $n0 $n2 10Mb 10ms DropTail
13 $ns duplex-link $n1 $n2 10Mb 10ms DropTail
14 $ns simplex-link $n2 $n3 0.3Mb 100ms DropTail
15 $ns simplex-link $n3 $n2 0.3Mb 100ms DropTail

```

```

$ns duplex-link $n0 $n2 0.5Mb 40ms DropTail
17 $ns duplex-link $n0 $n2 0.5Mb 40ms DropTail
18
19 #set queue-size of the link (n2-n3) to 20
20 $ns queue-limit $n2 $n3 20

```

TCP:

TCP is a dynamic reliable congestion protocol which is used to provide reliable transport of packets from one host to another host by sending acknowledgements on proper transfer or loss of packets. Thus TCP requires bi-directional links in order for acknowledgements to return to the source.

Now we will show how to set up tcp connection between two nodes

```

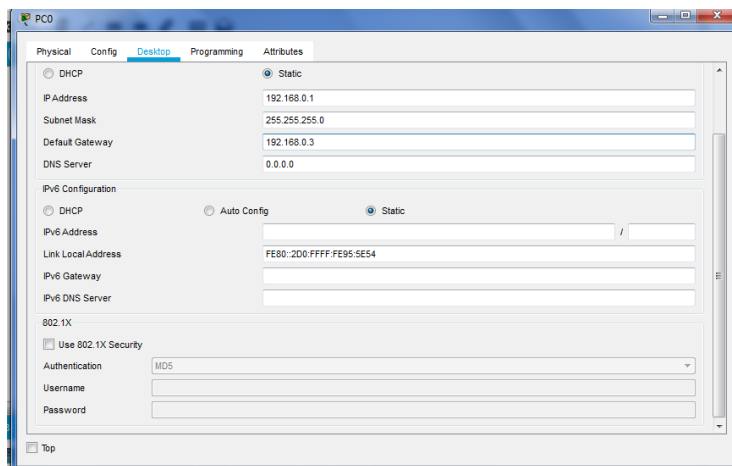
1 #setting a tcp connection
2 set tcp [new Agent/TCP]
3 $ns attach-agent $n0 $tcp
4 set sink [new Agent/TCPSink]
5 $ns attach-agent $n4 $sink
6 $ns connect $tcp $sink
7 $tcp set fid_1
8 $tcp set packetSize_552

```

Output: TCP Connection Established

IP Address configuration:

PC0 Configuration:



PC1 Configuration

PC1 Configuration window showing IP Configuration for Interface FastEthernet0. The IP Configuration section has the Static radio button selected. The IP Address is 192.168.0.2, Subnet Mask is 255.255.255.0, Default Gateway is 192.168.0.1, and DNS Server is 0.0.0.0. The IPv6 Configuration section has the Static radio button selected. The IPv6 Address is empty, Link Local Address is FE80::2D0:D3FF:FE8B:E899, IPv6 Gateway is empty, and IPv6 DNS Server is empty. The 802.1X section has the Use 802.1X Security checkbox unchecked.

IP Configuration	
Interface	FastEthernet0
☐ DHCP ☒ Static	
IP Address	192.168.0.2
Subnet Mask	255.255.255.0
Default Gateway	192.168.0.1
DNS Server	0.0.0.0

IPv6 Configuration	
☐ DHCP ☐ Auto Config ☒ Static	
IPv6 Address	
Link Local Address	FE80::2D0:D3FF:FE8B:E899
IPv6 Gateway	
IPv6 DNS Server	

802.1X

☐ Use 802.1X Security

PC2 Configuration

PC2 Configuration window showing IP Configuration for Interface FastEthernet0. The IP Configuration section has the Static radio button selected. The IP Address is 192.168.1.1, Subnet Mask is 255.255.255.0, Default Gateway is 192.168.1.3, and DNS Server is 0.0.0.0. The IPv6 Configuration section has the Static radio button selected. The IPv6 Address is empty, Link Local Address is FE80::260:70FF:FEDB:4479, IPv6 Gateway is empty, and IPv6 DNS Server is empty. The 802.1X section has the Use 802.1X Security checkbox unchecked. The Authentication dropdown is set to MD5, and the Username field is empty.

IP Configuration	
☐ DHCP ☒ Static	
IP Address	192.168.1.1
Subnet Mask	255.255.255.0
Default Gateway	192.168.1.3
DNS Server	0.0.0.0

IPv6 Configuration	
☐ DHCP ☐ Auto Config ☒ Static	
IPv6 Address	
Link Local Address	FE80::260:70FF:FEDB:4479
IPv6 Gateway	
IPv6 DNS Server	

802.1X

☐ Use 802.1X Security

Authentication: MD5

Username:

PC3 Configuration

PC3 Configuration window showing IP Configuration for Interface FastEthernet0. The IP Configuration section has the Static radio button selected. The IP Address is 192.168.1.2, Subnet Mask is 255.255.255.0, Default Gateway is 192.168.1.3, and DNS Server is 0.0.0.0. The IPv6 Configuration section has the Static radio button selected. The IPv6 Address is empty, Link Local Address is FE80::2D0:FFFF:FE48:2C71, IPv6 Gateway is empty, and IPv6 DNS Server is empty. The 802.1X section has the Use 802.1X Security checkbox unchecked. The Authentication dropdown is set to MD5, and the Username and Password fields are empty.

IP Configuration	
☐ DHCP ☒ Static	
IP Address	192.168.1.2
Subnet Mask	255.255.255.0
Default Gateway	192.168.1.3
DNS Server	0.0.0.0

IPv6 Configuration	
☐ DHCP ☐ Auto Config ☒ Static	
IPv6 Address	
Link Local Address	FE80::2D0:FFFF:FE48:2C71
IPv6 Gateway	
IPv6 DNS Server	

802.1X

☐ Use 802.1X Security

Authentication: MD5

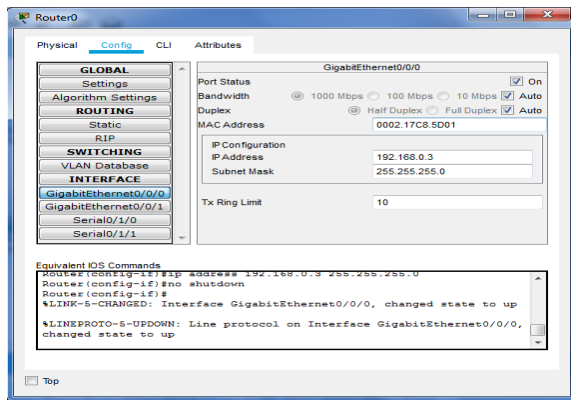
Username:

Password:

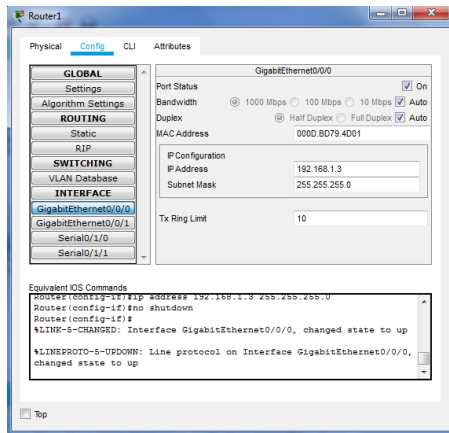
☐ Top

Router Configuration

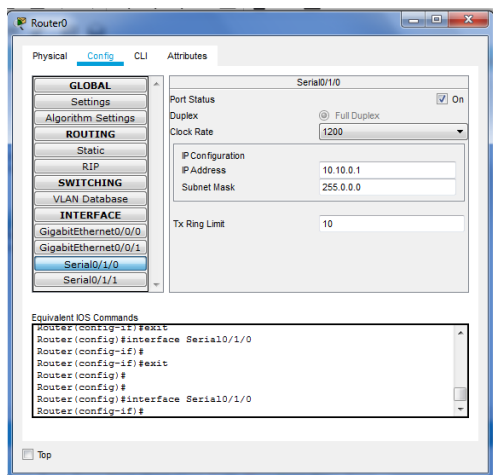
Router0:



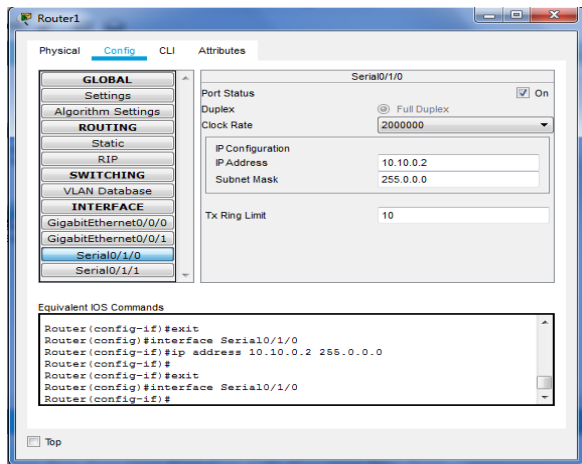
Router1:



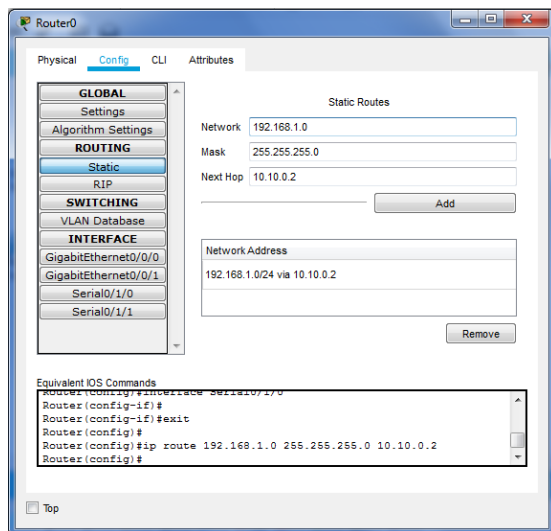
Serial cable for Router0 Configuration:



Serial Cable for router1 Configuration:



Router0 Static Configuration:



Week 4: Configure different network topologies using packet tracer/Network Simulator tool.

DESIGNING OF TOPOLOGY

Bus Topology:

In local area network, it is a single network cable runs in the building or campus and all nodes are connected along with this communication line with two endpoints called the bus or backbone. In other words, it is a multipoint data communication circuit that is easily control data flow between the computers because this configuration allows all stations to receive every transmission over the network . For bus topology we build network using three generic pc which are serially connected with three switches using copper straight through cable and switches are interconnected using copper cross over cable.

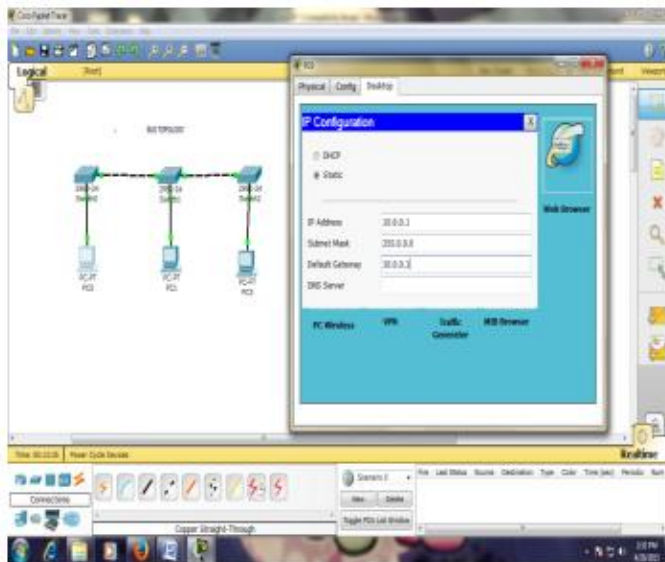


Fig -1: Design of bus topology

Star Topology

In star topology, all the cables run from the computers to a central location where they are all connected by a device called a hub. It is a concentrated network, points are directly reachable from a central location when network is expanded. [4]. Ethernet 10 base T is a popular network based on the star topology. For star topology we build network using five generic pc which are centrally connected to single switch 2950-24 using copper straight through cable.

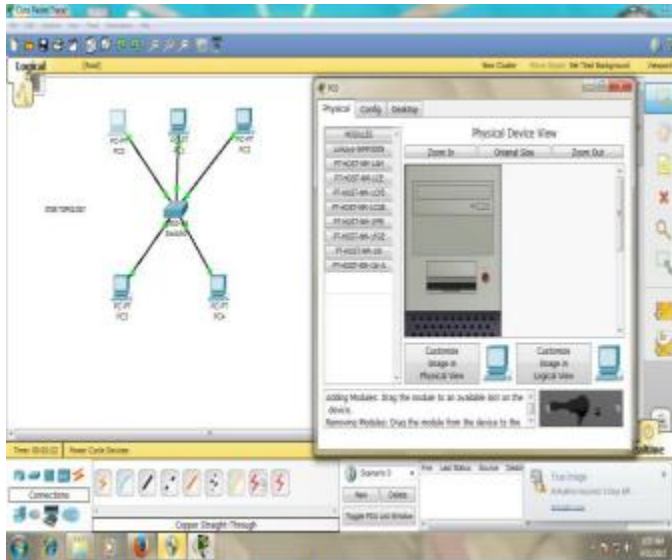


Fig -2: Design of star topology

Mesh Topology

In mesh topology every device has a dedicated point to point link to every other device. The term dedicated stand for link carries traffic only between two devices it connects. It is a well-connected topology; in this every node has a connection to every other node in the network. The cable requirements are high and it can include multiple topologies. Failure in one of the computers does not cause the network to break down, as they have alternative paths to other computers star topology, all the cables run from the computers to a central location.

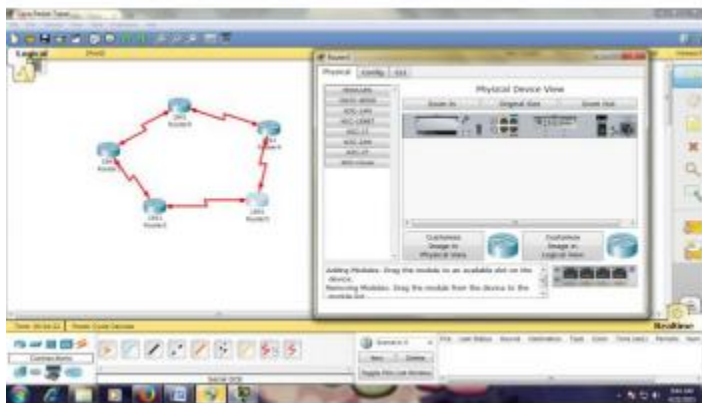


Fig -3: Design of mesh topology

For mesh topology we build network using five 1841 router. To design four serial port router click on router- >turn off->drag the WIC2T module two times.->power on. To establish connection between router to router using DCE cables.

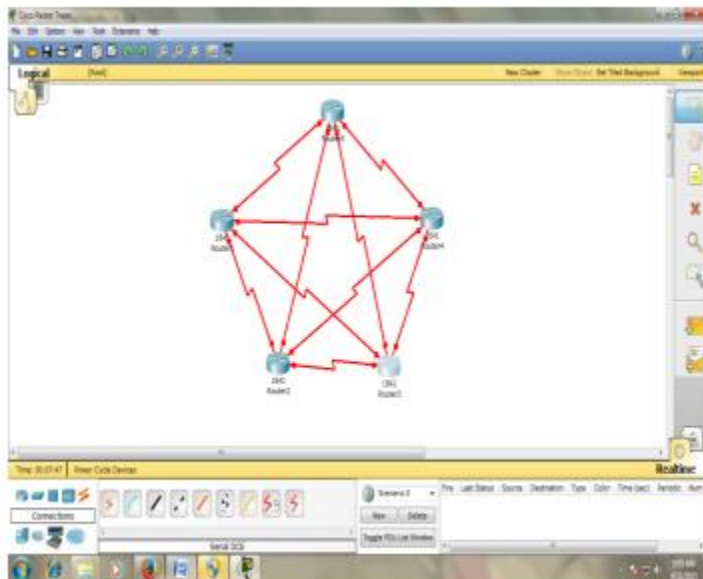


Fig -4: Connection of mesh topology

CONFIGURATION OF COMPONENT

1 Bus topology: To configure the IP address of an interface, we configure all PC one by one click on pc, open DESKTOP window, fill IP Address, Fill subnet mask and default gateway. After that, simulate the network using simulation

2 Star topology: To configure the IP address of an interface, we configure all PC one by one click on pc, open DESKTOP window, fill IP Address, Fill subnet mask and default gateway. After that, simulate the network using simulation

3 Mesh topology: To configure the IP address of an interface, we configure all routers one by one. Click on router, open config window, fill IP Address of serial port which are connected to router, Fill subnet mask, set clock rate and port status is ON. After that, simulate the network using simulation mode.

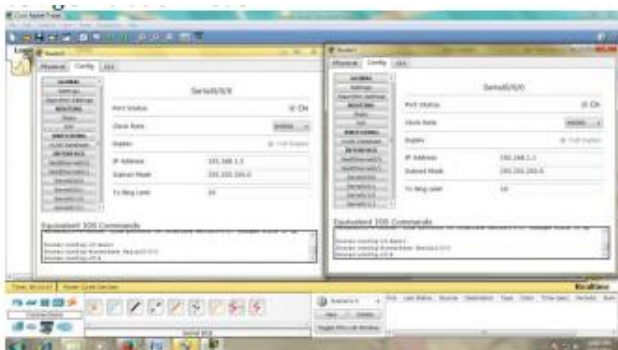


Fig -5: Router configuration of mesh topology

SIMULATION OF NETWORK TOPOLOGY

1 Simulation modal of bus topology

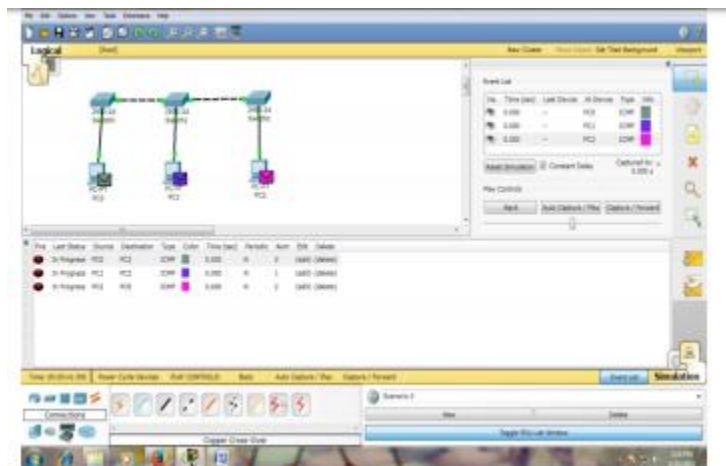


Fig -6: Simulation of bus topology

2 Simulation modal of star topology

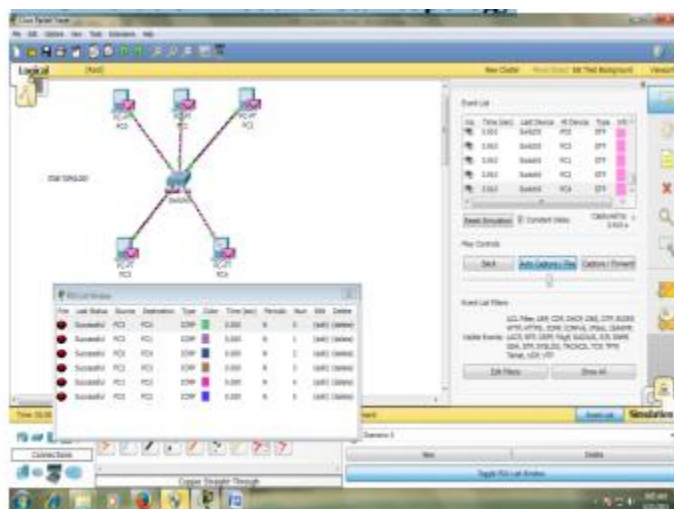


Fig -7: Simulation of star topology

3 Simulation modal of mesh topology

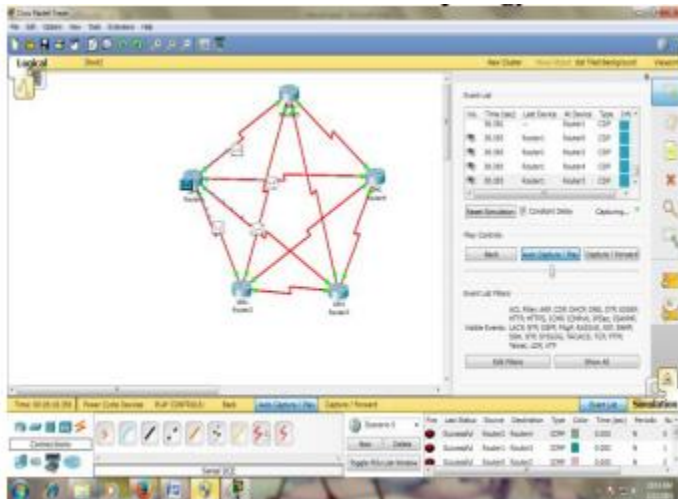
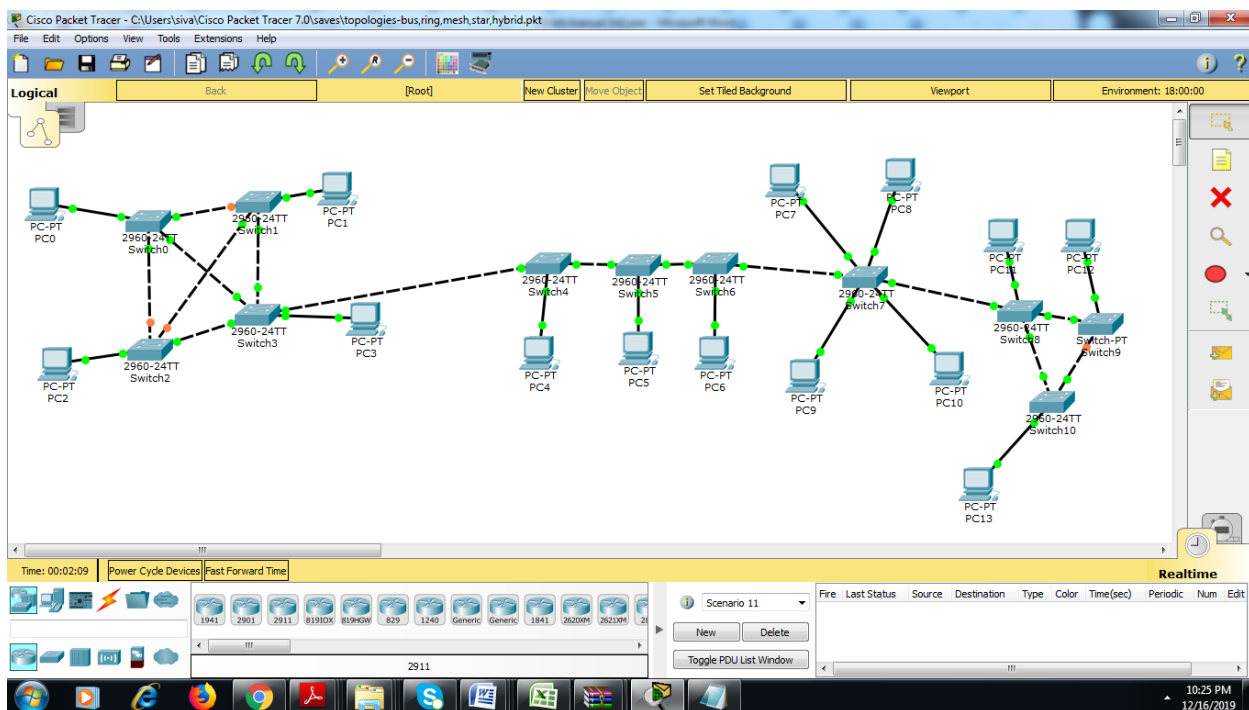
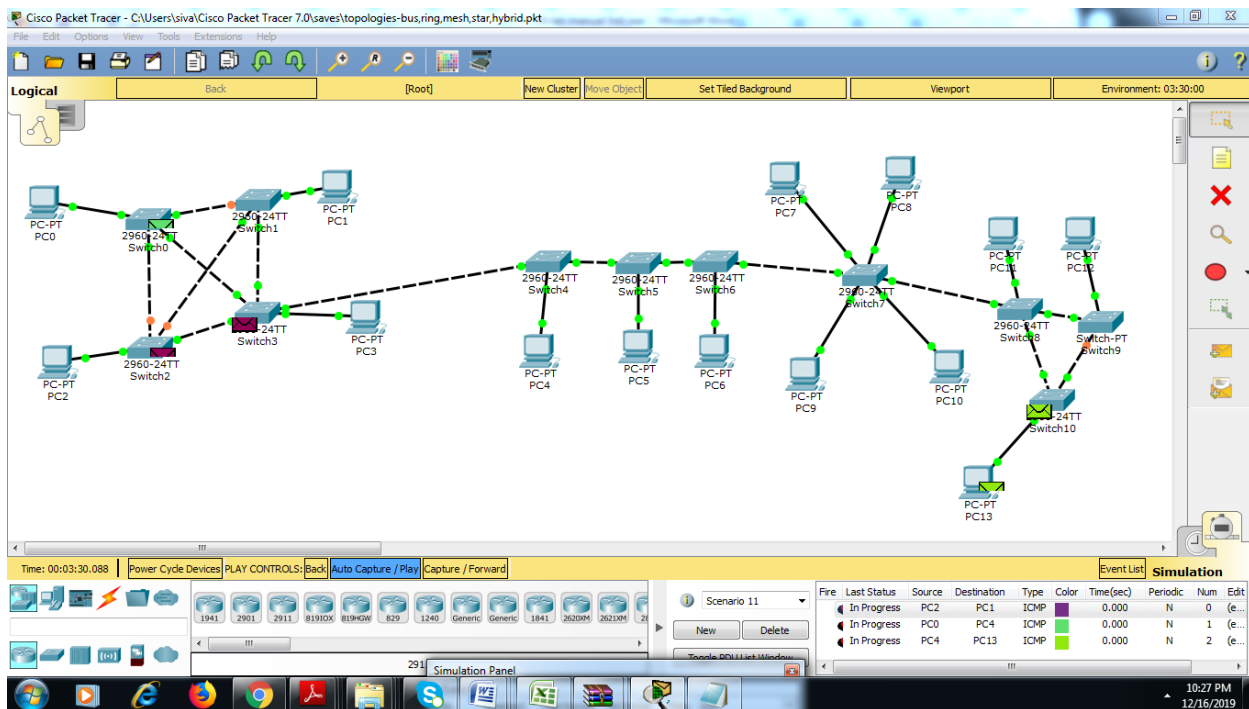
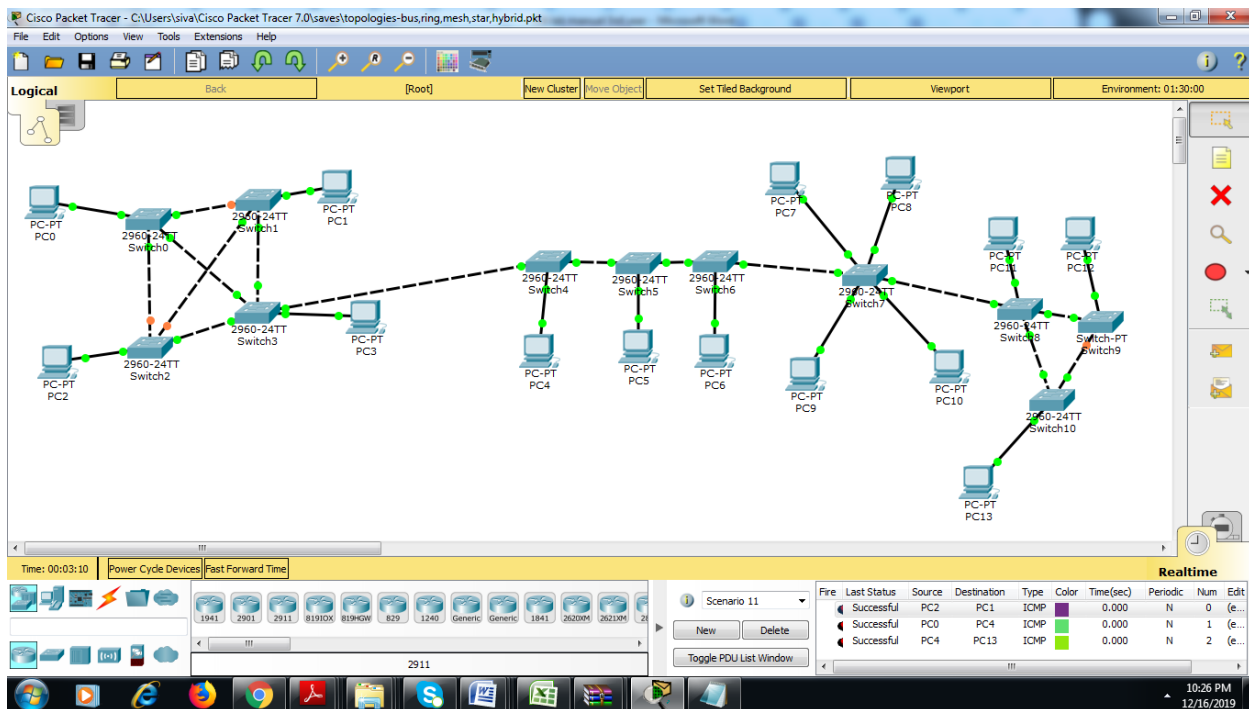


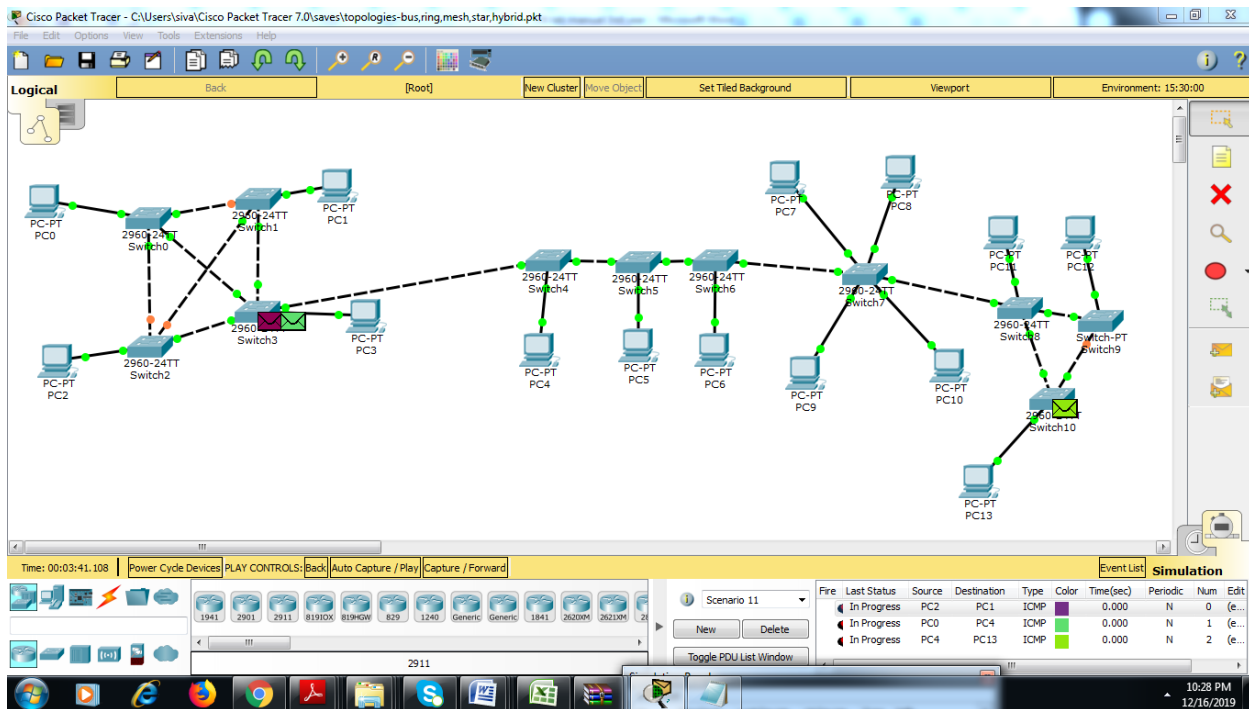
Fig -8: Simulation of mesh topology

BUS, Star, Mesh, Hybrid Topologies:



Output:





Week 5:

a. Write a program to implement the Data link layer framing methods such as character stuffing and bit stuffing.

Program for character stuffing:

At Sender (Client) Side:

// Java Code for Byte_Stuffing Sender

```
package byte_stuffing;
import java.io.*;
import java.util.*;
import java.net.*;

public class Byte_Stuffing_Client {
    public static void main(String args[]) throws IOException
    {
        InetAddress ip = InetAddress.getLocalHost();
        int port = 45678;
        Scanner sc = new Scanner(System.in);

        // Opens a socket for connection
        Socket s = new Socket(ip, port);

        // Declaring I/O Streams
        DataInputStream dis = new DataInputStream(s.getInputStream());
        DataOutputStream dos = new DataOutputStream(s.getOutputStream());

        while (true) {
            System.out.println("Enter the Message to be Sent : ");
            String data = sc.nextLine();
            String res = new String();

            // Data in each frame is stuffed by 'F' at beginning and end
            data = 'F' + data + 'F';
            for (int i = 0; i < data.length(); i++) {

                // Stuff with 'E' if 'F' is found in the data to be sent
                if (data.charAt(i) == 'F' && i != 0 && i != (data.length() - 1))
                    res = res + 'E' + data.charAt(i);

                // Stuff with 'E' if 'E' is found in the data to be sent
                else if (data.charAt(i) == 'E')
                    res = res + 'E' + data.charAt(i);
                else
                    res = res + data.charAt(i);
            }
        }
    }
}
```

```

    }

    System.out.println("The data being sent (with byte stuffed) is : " + res);

    // Send the data to the receiver
    dos.writeUTF(res);
    System.out.println("Seding Message....");
    if (dis.readUTF().equals("success"))
        System.out.println("Thanks for the Feedback Server!!");

    // End Messaging
    dos.writeUTF("bye");
    break;
}

// Close all connections
s.close();
dis.close();
dos.close();
}
}

```

At Receiver (Server) Side:

```

// Java code for Byte_Stuffing Receiver
package byte_stuffing;
import java.io.*;
import java.net.*;

public class Byte_Stuffing {
    public static void main(String[] args) throws IOException
    {
        // Opens a socket for connection
        ServerSocket servsock = new ServerSocket(45678);

        // Used to block until a client connects to the server
        Socket socket = servsock.accept();

        // Declaring I/O Streams
        DataInputStream dis = new DataInputStream(socket.getInputStream());
        DataOutputStream dos = new DataOutputStream(socket.getOutputStream());

        while (true) {
            String out = new String();
            // Used to read the data sent by client
            String res = dis.readUTF();
            System.out.println("Message Recevied...Successfully!!!");

```

```

        System.out.println("The Stuffed Message is : " + res);

        for (int i = 1; i < res.length() - 1; i++) {

            // If data contains a 'D' or 'F' do not unstuff it
            if (res.charAt(i) == 'D' || res.charAt(i) == 'F')
                out = out + res.charAt(i);

            // If data contains 'E' followed by 'E', de-stuff the former 'E'
            else if (res.charAt(i) == 'E' && res.charAt(i + 1) == 'E') {
                out = out + 'E';
                i++;
            }
        }
        System.out.println("The Destuffed Message is : " + out);
        dos.writeUTF("success");
        String ch = dis.readUTF();
        if (ch.equals("bye")) {
            System.out.println("Messaging is over.....EXITING");
            break;
        }
    }
}

// Closing all connections
socket.close();
dis.close();
dos.close();
}
}

```

Output:

At Client (Sender) Side

Enter the Message to be Sent :

DFEDDFED

The data being sent (with byte stuffed) is : FDEFEEDEFEEDF

Sending Message....

Thanks for the Feedback Server!!

At Server (Receiver) Side

Message Received...Successfully!!!

The Stuffed Message is : FDEFEEDEFEEDF

The Destuffed Message is : DFEDDFED

Messaging is over.....EXITING

Bit stuffing Program:

```
import java.util.*;
public class SEFBS
{
    public static void main(String[] args)
    {
        System.out.print("Enter the Binary message: ");
        Scanner sn=new Scanner(System.in);
        String data = sn.nextLine();
        String res = new String();
        String out=new String();
        int counter = 0;
        for(int i=0;i<data.length();i++)
        {
            if (data.charAt(i)!='1' && data.charAt(i)!='0')
            {
                System.out.println("Enter only Binary values!!!");
                return;
            }
            if(data.charAt(i) == '1')
            {
                counter++;
                res = res + data.charAt(i);
            }
            else
            {
                res = res + data.charAt(i);
                counter = 0;
            }
            if(counter == 5)
            {
                res = res + '0';
                counter = 0;
            }
        }
        String inc="01111110"+res+"01111110";
        System.out.println("The Message to be transfered: " +inc);
        System.out.println("Seding Message....");
        counter=0;
        for(int i=0;i<res.length();i++)
        {
            if(res.charAt(i) == '1')
            {
                counter++;
                out = out + res.charAt(i);
            }
        }
    }
}
```

```

else
{
out = out + res.charAt(i);
counter = 0;
}
if(counter == 5)
{
if((i+2)!=res.length())
out = out + res.charAt(i+2);
else
out=out + '1';
i=i+2;
counter = 1;
}
}
System.out.println("Message Recevied...Successfully!!!");
System.out.println("The Destuffed Message is: "+out);
}
}

```

OUTPUT:

Case 1:

Enter the Binary message: 011111110

The Message to be transferred: 0111111001111011100111110

Sending Message....

Message Received...Successfully!!!

The Destuffed Message is: 011111110

Case 2:

Enter the Binary message: 01111110

The Message to be transferred: 01111110011110100111110

Sending Message....

Message Received...Successfully!!!

The Destuffed Message is: 01111110

b. Write a program to simulate Stop and wait protocol and Sliding Window Protocols.

Stop and wait protocol Program:

/SENDER//

```
import java.io.*;
import java.net.*;
import java.util.Scanner;
class stopwaitsender
{
public static void main(String args[]) throws Exception
{
stopwaitsender sws = new stopwaitsender();
sws.run();
}
public void run() throws Exception
{
Scanner sc=new Scanner(System.in);
System.out.println("Enter no of frames to be sent:");
int n=sc.nextInt();
Socket myskt=new Socket("localhost",9999);
PrintStream myps=new PrintStream(myskt.getOutputStream());
for(int i=0;i<=n;)
{
if(i==n)
{
mysps.println("exit");
break;
}
System.out.println("Frame no "+i+" is sent");
mysps.println(i);
BufferedReader bf=new BufferedReader(new InputStreamReader(myskt.getInputStream()));
String ack=bf.readLine();
if(ack!=null)
{
System.out.println("Acknowledgement was Received from receiver");
i++;
Thread.sleep(4000);
}
else
{
mysps.println(i);
```

```

}
}
}
}
//RECEIVER//
import java.io.*;
import java.net.*;
class stopwaitreceiver
{
public static void main(String args[])throws Exception
{
stopwaitreceiver swr = new stopwaitreceiver();
swr.run();
}
public void run() throws Exception
{
String temp="any message",str="exit";
ServerSocket myss=new ServerSocket(9999);
Socket ss_accept=myss.accept();
BufferedReader ss_bf=new BufferedReader(new InputStreamReader(ss_accept.getInputStream()));
PrintStream myps=new PrintStream(ss_accept.getOutputStream());
while(temp.compareTo(str)!=0)
{
Thread.sleep(1000);
temp=ss_bf.readLine();
if(temp.compareTo(str)==0)
{ break;}
System.out.println("Frame "+temp+" was received");
Thread.sleep(500);
myps.println("Received");
}
System.out.println("ALL FRAMES WERE RECEIVED SUCCESSFULLY");
}
}

```

OUTPUT FOR SENDER:

C:\javaprogram>javac stopwaitsender.java

C:\javaprogram>java stopwaitsender

Enter no of frames to be sent:

4

Frame no 0 is sent

Acknowledgement was Received from receiver

Frame no 1 is sent

Acknowledgement was Received from receiver

Frame no 2 is sent

Acknowledgement was Received from receiver

Frame no 3 is sent

Acknowledgement was Received from receiver

-

OUTPUT FOR RECEIVER:

C:\javaprogram>javac stopwaitreceiver.java

C:\javaprogram>java stopwaitreceiver

Frame 0 was received

Frame 1 was received

Frame 2 was received

Frame 3 was received

ALL FRAMES WERE RECEIVED SUCCESSFULLY

GO BACK - N Sliding Window Protocol:

/*Server Program*/

import java.net.*;

import java.io.*;

import java.util.*;

public class Server

{

public static void main(String args[]) throws Exception

{

ServerSocket server=new ServerSocket(6262);

System.out.println("Server established.");

Socket client=server.accept();

ObjectOutputStream oos=new ObjectOutputStream(client.getOutputStream());

ObjectInputStream ois=new ObjectInputStream(client.getInputStream());

System.out.println("Client is now connected.");

int x=(Integer)ois.readObject();

int k=(Integer)ois.readObject();

int j=0;

int i=(Integer)ois.readObject();

boolean flag=true;

Random r=new Random(6);

int mod=r.nextInt(6);

while(mod==1||mod==0)

```

mod=r.nextInt(6);
while(true)
{
int c=k;
for(int h=0;h<=x;h++)
{
System.out.print("|"+c+"|");
c=(c+1)%x;
}
System.out.println();
System.out.println();
if(k==j)
{
System.out.println("Frame "+k+" recieved"+"\\n"+"Data:"+j);
j++;
System.out.println();
}
else
System.out.println("Frames recieved not in correct order"+"\\n"+" Expected farne:" + j +"\\n" + "
Recieved frame no :"+ k);
System.out.println();
if(j%mod==0 && flag)
{
System.out.println("Error found. Acknowledgement not sent. ");
flag=!flag;
j--;
}
else if(k==j-1)
{
oos.writeObject(k);
System.out.println("Acknowledgement sent");
}
System.out.println();
if(j%mod==0)
flag=!flag;
k=(Integer)ois.readObject();
if(k==-1)
break;
i=(Integer)ois.readObject();
}

```

```

System.out.println("Client finished sending data. Exiting");
oos.writeObject(-1);
}
}

```

/*Client Program*/

```

import java.util.*;
import java.net.*;
import java.io.*;
public class Client
{
public static void main(String args[]) throws Exception
{
BufferedReader br=new BufferedReader(new InputStreamReader(System.in));
System.out.print("Enter the value of m : ");
int m=Integer.parseInt(br.readLine());
int x=(int)((Math.pow(2,m))-1);
System.out.print("Enter no. of frames to be sent:");
int count=Integer.parseInt(br.readLine());
int data[]=new int[count];
int h=0;
for(int i=0;i<count;i++)
{
System.out.print("Enter data for frame no " +h+ " => ");
data[i]=Integer.parseInt(br.readLine());
h=(h+1)%x;
}
Socket client=new Socket("localhost",6262);
ObjectInputStream ois=new ObjectInputStream(client.getInputStream());
ObjectOutputStream oos=new ObjectOutputStream(client.getOutputStream());
System.out.println("Connected with server.");
boolean flag=false;
GoBackNListener listener=new GoBackNListener(ois,x);
listener=new GoBackNListener(ois,x);
listener.t.start();
int strt=0;
h=0;
oos.writeObject(x);
do
{

```

```

int c=h;
for(int i=h;i<count;i++)
{
System.out.print("|"+c+"|");
c=(c+1)%x;
}
System.out.println();
System.out.println();
h=strt;
for(int i=strt;i<x;i++)
{
System.out.println("Sending frame:"+h);
h=(h+1)%x;
System.out.println();
oos.writeObject(i);
oos.writeObject(data[i]);
Thread.sleep(100);
}
listener.t.join(3500);
if(listener.reply!=x-1)
{
System.out.println("No reply from server in 3.5 seconds. Resending data from frame no " +
(listener.reply+1));
System.out.println();
strt=listener.reply+1;
flag=false;
}
else
{
System.out.println("All elements sent successfully. Exiting");
flag=true;
}
}while(!flag);
oos.writeObject(-1);
}
}
class GoBackNListener implements Runnable
{
Thread t;
ObjectInputStream ois;

```

```

int reply,x;
GoBackNListener(ObjectInputStream o,int i)
{
t=new Thread(this);
ois=o;
reply=-2;
x=i;
}
@Override
public void run() {
try
{
int temp=0;
while(reply!=-1)
{
reply=(Integer)ois.readObject();
if(reply!=-1 && reply!=temp+1)
reply=temp;
if(reply!=-1)
{
temp=reply;
System.out.println("Acknowledgement of frame no " + (reply%x) + " recieved.");
System.out.println();
}
}
reply=temp;
}
catch(Exception e)
{
System.out.println("Exception => " + e);
}
}
}

```

Client Output:

```

Enter the value of m : 7
Enter no. of frames to be sent:5
Enter data for frame no 0 => 1
Enter data for frame no 1 => 2
Enter data for frame no 2 => 3

```

Enter data for frame no 3 => 4

Enter data for frame no 4 => 5

Connected with server.

|0||1||2||3||4|

Sending frame:0

Acknowledgement of frame no 0 recieved.

Sending frame:1

Sending frame:2

Sending frame:3

Sending frame:4

Sending frame:5

Server Output:

Server established.

Client is now connected.

|0||1||2||3||4||5||6||7||8||9||10||11||12||13||14||15||16||17||18||19||20||21||22||23||24||25||26||27||28||29||30||31||32||33||34||35||36||37||38||39||40||41||42||43||44||45||46||47||48||49||50||51||52||53||54||55||56||57||58||59||60||61||62||63||64||65||66||67||68||69||70||71||72||73||74||75||76||77||78||79||80||81||82||83||84||85||86||87||88||89||90||91||92||93||94||95||96||97||98||99||100||101||102||103||104||105||106||107||108||109||110||111||112||113||114||115||116||117||118||119||120||121||122||123||124||125||126||0|

Frame 0 recieved

Data:0

Acknowledgement sent

|1||2||3||4||5||6||7||8||9||10||11||12||13||14||15||16||17||18||19||20||21||22||23||24||25||26||27||28||29||30||31||32||33||34||35||36||37||38||39||40||41||42||43||44||45||46||47||48||49||50||51||52||53||54||55||56||57||58||59||60||61||62||63||64||65||66||67||68||69||70||71||72||73||74||75||76||77||78||79||80||81||82||83||84||85||86||87||88||89||90||91||92||93||94||95||96||97||98||99||100||101||102||103||104||105||106||107||108||109||110||111||112||113||114||115||116||117||118||119||120||121||122||123||124||125||126||0||1|

Frame 1 recieved

Data:1

Error found. Acknowledgement not sent.

|2||3||4||5||6||7||8||9||10||11||12||13||14||15||16||17||18||19||20||21||22||23||24||25||26||27||28||29||30||31||32||33||34||35||36||37||38||39||40||41||42||43||44||45||46||47||48||49||50||51||52||53||54||55||56||57||58||59||60||61||62||63||64||65||66||67||68||69||70||71||72||73||74||75||76||77||78||79||80||81||82||83||84||85||86||87||88||89||90||91||

92||93||94||95||96||97||98||99||100||101||102||103||104||105||106||107||108||109||110||111||112||113||114||115||116||117||118||119||120||121||122||123||124||125||126||0||1||2|

Frames recieved not in correct order

Expected farne:1

Recieved frame no :2

|3||4||5||6||7||8||9||10||11||12||13||14||15||16||17||18||19||20||21||22||23||24||25||26||27||28||29||30||31||32||33||34||35||36||37||38||39||40||41||42||43||44||45||46||47||48||49||50||51||52||53||54||55||56||57||58||59||60||61||62||63||64||65||66||67||68||69||70||71||72||73||74||75||76||77||78||79||80||81||82||83||84||85||86||87||88||89||90||91||92||93||94||95||96||97||98||99||100||101||102||103||104||105||106||107||108||109||110||111||112||113||114||115||116||117||118||119||120||121||122||123||124||125||126||0||1||2||3|

Frames recieved not in correct order

Expected farne:1

Recieved frame no :3

|4||5||6||7||8||9||10||11||12||13||14||15||16||17||18||19||20||21||22||23||24||25||26||27||28||29||30||31||32||33||34||35||36||37||38||39||40||41||42||43||44||45||46||47||48||49||50||51||52||53||54||55||56||57||58||59||60||61||62||63||64||65||66||67||68||69||70||71||72||73||74||75||76||77||78||79||80||81||82||83||84||85||86||87||88||89||90||91||92||93||94||95||96||97||98||99||100||101||102||103||104||105||106||107||108||109||110||111||112||113||114||115||116||117||118||119||120||121||122||123||124||125||126||0||1||2||3||4|

Frames recieved not in correct order

Expected farne:1

Recieved frame no :4

Week 6:

Write a program to implement on a data set of characters using the three Cyclic Redundancy Check.

Program:

```
import java.io.*;
class crc_gen
{
    public static void main(String args[]) throws IOException
    {
        BufferedReader br=new BufferedReader(new InputStreamReader(System.in));
        int[] data;
        int[] div;
        int[] divisor;
        int[] rem;
        int[] crc;
        int data_bits, divisor_bits, tot_length;

        System.out.println("Enter number of data bits : ");
        data_bits=Integer.parseInt(br.readLine());
        data=new int[data_bits];

        System.out.println("Enter data bits : ");
        for(int i=0; i<data_bits; i++)
            data[i]=Integer.parseInt(br.readLine());

        System.out.println("Enter number of bits in divisor : ");
        divisor_bits=Integer.parseInt(br.readLine());
        divisor=new int[divisor_bits];

        System.out.println("Enter Divisor bits : ");
        for(int i=0; i<divisor_bits; i++)
            divisor[i]=Integer.parseInt(br.readLine());

        System.out.print("Data bits are : ");
        for(int i=0; i< data_bits; i++)
            System.out.print(data[i]);
        System.out.println();

        System.out.print("divisor bits are : ");
        for(int i=0; i< divisor_bits; i++)
            System.out.print(divisor[i]);
```

```

    System.out.println();

    tot_length=data_bits+divisor_bits-1;

    div=new int[tot_length];
    rem=new int[tot_length];
    crc=new int[tot_length];
    /*----- CRC GENERATION-----*/
    for(int i=0;i<data.length;i++)
        div[i]=data[i];

    System.out.print("Dividend (after appending 0's) are : ");
    for(int i=0; i< div.length; i++)
        System.out.print(div[i]);
    System.out.println();

    for(int j=0; j<div.length; j++){
        rem[j] = div[j];
    }

    rem=divide(div, divisor, rem);
    for(int i=0;i<div.length;i++)        //append dividend and remainder
    {
        crc[i]=(div[i]^rem[i]);
    }
    System.out.println();
    System.out.println("CRC code : ");
    for(int i=0;i<crc.length;i++)
        System.out.print(crc[i]);

    /*-----ERROR DETECTION-----*/
    System.out.println();
    System.out.println("Enter the Received CRC code of "+tot_length+" bits : ");
    for(int i=0; i<crc.length; i++)
        crc[i]=Integer.parseInt(br.readLine());

    System.out.print("crc bits are : ");
    for(int i=0; i< crc.length; i++)
        System.out.print(crc[i]);
    System.out.println();

```

```

for(int j=0; j<crc.length; j++){
    rem[j] = crc[j];
}

rem=divide(crc, divisor, rem);

for(int i=0; i< rem.length; i++)
{
    if(rem[i]!=0)
    {
        System.out.println("Received codeword contains Errors");
        break;
    }
    if(i==rem.length-1)
        System.out.println("No Error");
}

System.out.println("THANK YOU.... :)");
}

static int[] divide(int div[],int divisor[], int rem[])
{
    int cur=0;
    while(true)
    {
        for(int i=0;i<divisor.length;i++)
            rem[cur+i]=(rem[cur+i]^divisor[i]);

        while(rem[cur]==0 && cur!=rem.length-1)
            cur++;

        if((rem.length-cur)<divisor.length)
            break;
    }
    return rem;
}
}

```

OUTPUT :

Enter number of data bits :

7

Enter data bits :

1

0

1

1

0

0

1

Enter number of bits in divisor :

3

Enter Divisor bits :

1

0

1

Data bits are : 1011001

divisor bits are : 101

Dividend (after appending 0's) are : 101100100

CRC code :

101100111

Enter the Received CRC code of 9 bits :

1

0

1

1

0

0

1

0

1

crc bits are : 101100101

Received codeword contains Errors

THANK YOU.... :)

Week 7:

Write a program to simulate Carrier Sense Multiple Access/Collision Detection (CSMA/CD) and Carrier Sense Multiple Access/Collision Avoidance (CSMA/CA).

Carrier Sense Multiple Access/Collision Avoidance (CSMA/CA):

ALGORITHM:

1. Create a simulator object
2. Define different colors for different data flows
3. Open a nam trace file and define finish procedure then close the trace file, and execute nam on trace file.
4. Create six nodes that forms a network numbered from 0 to 5
5. Create duplex links between the nodes and add Orientation to the nodes for setting a LAN topology
6. Setup TCP Connection between n(0) and n(4)
7. Apply FTP Traffic over TCP
8. Setup UDP Connection between n(1) and n(5)
9. Apply CBR Traffic over UDP.
10. Apply CSMA/CA and CSMA/CD mechanisms and study their performance
11. Schedule events and run the program

PROGRAM:

CSMA/CA

```
set ns [new Simulator]
#Define different colors for data flows (for NAM)
$ns color 1 Blue
$ns color 2 Red
#Open the Trace files
set file1 [open out.tr w]
set winfile [open WinFile w]
$ns trace-all $file1
#Open the NAM trace file
set file2 [open out.nam w]
$ns namtrace-all $file2
#Define a 'finish' procedure
proc finish {} {
    global ns file1 file2
    $ns flush-trace
    close $file1
    close $file2
    exec nam out.nam &
    exit 0
}
#Create six nodes
set n0 [$ns node]
set n1 [$ns node]
set n2 [$ns node]
set n3 [$ns node]
```

```

set n4 [$ns node]
set n5 [$ns node]
$n1 color red
$n1 shape box
#Create links between the nodes
$ns duplex-link $n0 $n2 2Mb 10ms DropTail
$ns duplex-link $n1 $n2 2Mb 10ms DropTail
$ns simplex-link $n2 $n3 0.3Mb 100ms DropTail
$ns simplex-link $n3 $n2 0.3Mb 100ms DropTail
set lan [$ns newLan "$n3 $n4 $n5" 0.5Mb 40ms LL Queue/DropTail MAC/Csma/Ca Channel]
Setup a TCP connection
set tcp [new Agent/TCP/Newreno]
$ns attach-agent $n0 $tcp
set sink [new Agent/TCPSink/DelAck]
$ns attach-agent $n4 $sink
$ns connect $tcp $sink
$tcp set fid_ 1
$tcp set window_ 8000

$tcp set packetSize_ 552
#Setup a FTP over TCP connection
set ftp [new Application/FTP]
$ftp attach-agent $tcp
$ftp set type_ FTP
#Setup a UDP connection
set udp [new Agent/UDP]
$ns attach-agent $n1 $udp
set null [new Agent/Null]
$ns attach-agent $n5 $null
$ns connect $udp $null
$udp set fid_ 2
#Setup a CBR over UDP connection
set cbr [new Application/Traffic/CBR]
$cbr attach-agent $udp
$cbr set type_ CBR
$cbr set packet_size_ 1000
$cbr set rate_ 0.01mb
$cbr set random_ false
$ns at 0.1 "$cbr start"
$ns at 1.0 "$ftp start"
$ns at 124.0 "$ftp stop"
$ns at 124.5 "$cbr stop"
# next procedure gets two arguments: the name of the
# tcp source node, will be called here "tcp",
# and the name of output file.

```

```

proc plotWindow {tcpSource file} {

```

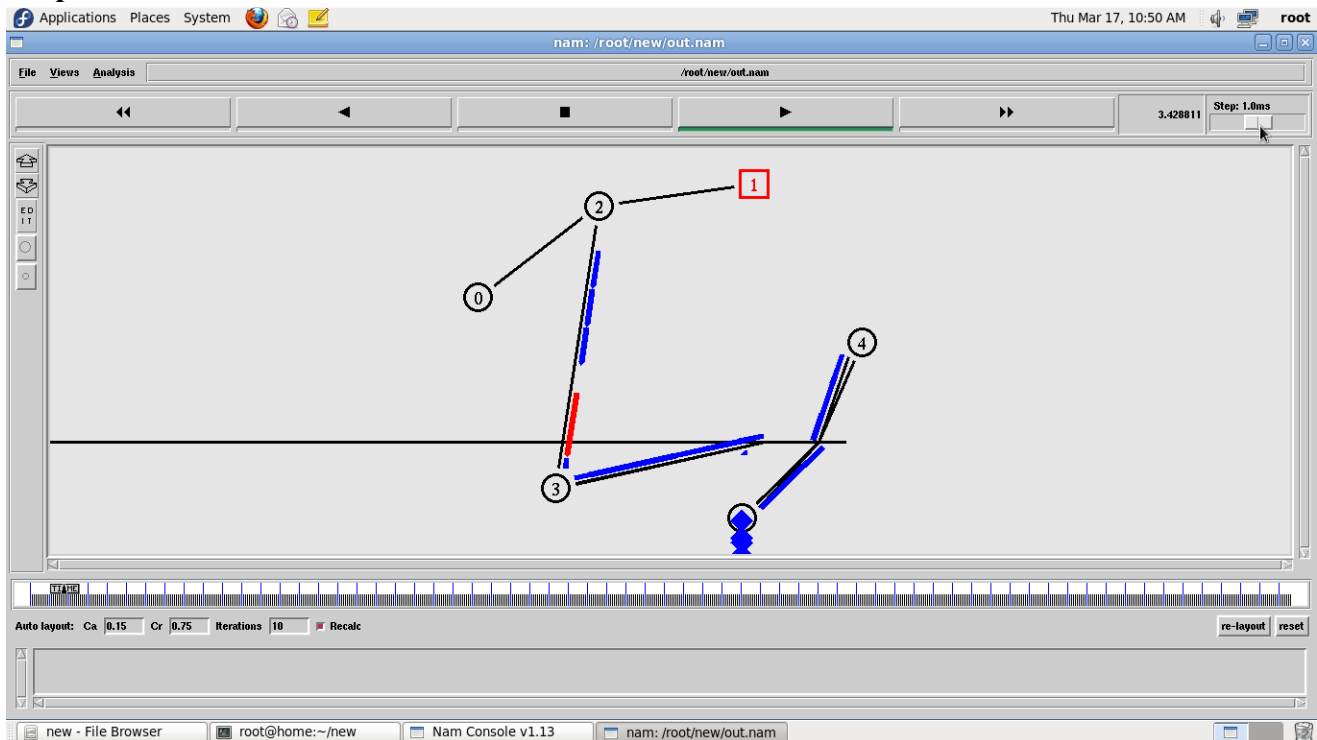
```

global ns
set time 0.1
set now [$ns now]
set cwnd [$tcpSource set cwnd_]
set wnd [$tcpSource set window_]
puts $file "$now $cwnd"
$ns at [expr $now+$time] "plotWindow $tcpSource $file" }
$ns at 0.1 "plotWindow $tcp $winfile"

$ns at 5 "$ns trace-annotate \"packet drop\""
# PPP
$ns at 125.0 "finish"
$ns run

```

Output:



Carrier Sense Multiple Access/Collision Detection (CSMA/CD):

```

set ns [new Simulator]

```

```

#Define different colors for data flows (for NAM)
$ns color 1 Blue
$ns color 2 Red
#Open the Trace files
set file1 [open out.tr w]
set winfile [open WinFile w]
$ns trace-all $file1
#Open the NAM trace file
set file2 [open out.nam w]
$ns namtrace-all $file2
#Define a 'finish' procedure
proc finish {} {
    global ns file1 file2
    $ns flush-trace
    close $file1
    close $file2
    exec nam out.nam &
    exit 0
}
#Create six nodes
set n0 [$ns node]
set n1 [$ns node]
set n2 [$ns node]
set n3 [$ns node]
set n4 [$ns node]
set n5 [$ns node]
$n1 color red
$n1 shape box

#Create links between the nodes
$ns duplex-link $n0 $n2 2Mb 10ms DropTail
$ns duplex-link $n1 $n2 2Mb 10ms DropTail
$ns simplex-link $n2 $n3 0.3Mb 100ms DropTail
$ns simplex-link $n3 $n2 0.3Mb 100ms DropTail
set lan [$ns newLan "$n3 $n4 $n5" 0.5Mb 40ms LL Queue/DropTail MAC/Csma/Cd Channel]
Setup a TCP connection
set tcp [new Agent/TCP/Newreno]
$ns attach-agent $n0 $tcp
set sink [new Agent/TCPSink/DelAck]
$ns attach-agent $n4 $sink
$ns connect $tcp $sink
$tcp set fid_ 1
$tcp set window_ 8000
$tcp set packetSize_ 552
#Setup a FTP over TCP connection
set ftp [new Application/FTP]

```

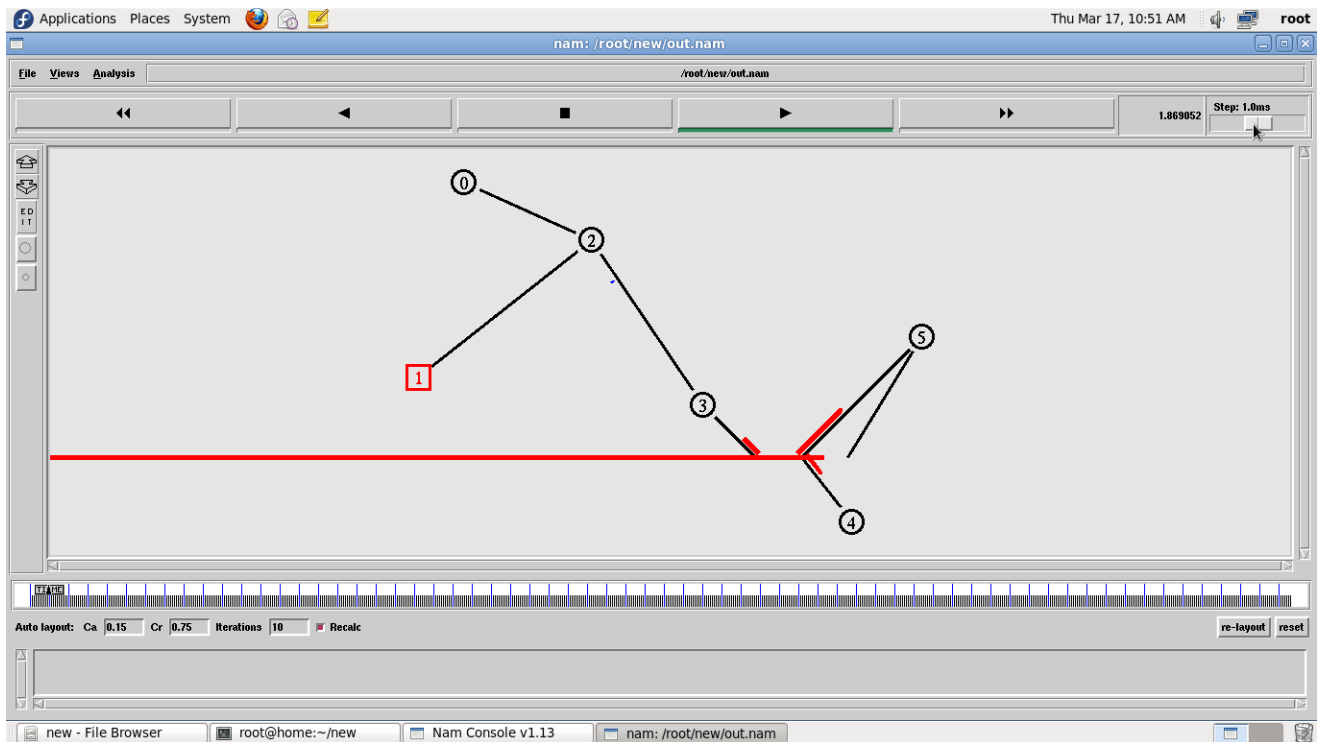
```

$ftp attach-agent $tcp
$ftp set type_ FTP
#Setup a UDP connection
set udp [new Agent/UDP]
$ns attach-agent $n1 $udp
set null [new Agent/Null]
$ns attach-agent $n5 $null
$ns connect $udp $null
$udp set fid_ 2
#Setup a CBR over UDP connection

set cbr [new Application/Traffic/CBR]
$cbr attach-agent $udp
$cbr set type_ CBR
$cbr set packet_size_ 1000
$cbr set rate_ 0.01mb
$cbr set random_ false
$ns at 0.1 "$cbr start"
$ns at 1.0 "$ftp start"
$ns at 124.0 "$ftp stop"
$ns at 124.5 "$cbr stop"
# next procedure gets two arguments: the name of the
# tcp source node, will be called here "tcp",
# and the name of output file.
proc plotWindow {tcpSource file} {
global ns
set time 0.1
set now [$ns now]
set cwnd [$tcpSource set cwnd_]
set wnd [$tcpSource set window_]
puts $file "$now $cwnd"
$ns at [expr $now+$time] "plotWindow $tcpSource $file" }
$ns at 0.1 "plotWindow $tcp $winfile"
$ns at 5 "$ns trace-annotate \"packet drop\""
# PPP
$ns at 125.0 "finish"
$ns run

```

Output:



Week 8: Configure a network using Distance Vector Routing protocol using packet tracer tool.

Objective: To understand the need of network routing algorithms.

Outcome: Student will be able to implement routing algorithms for given scenario.

Procedure:

Distance Vector Routing Algorithm:

Step 1. A router transmits its distance vector to each of its neighbors in a routing packet.

Step 2. Each router receives and saves the most recently received distance vector from each of its neighbors.

Step 3. A router recalculates its distance vector when:

a) It receives a distance vector from a neighbor containing different information than before.

b) It discovers that a link to a neighbor has gone down.

Step 4. The DV calculation is based on minimizing the cost to each destination

$D_x(y)$ = Estimate of least cost from x to y

$C(x,v)$ = Node x knows cost to each neighbor v

$D_x = [D_x(y): y \in N]$ = Node x maintains distance vector

Node x also maintains its neighbors' distance vectors

For each neighbor v, x maintains $D_v = [D_v(y): y \in N]$

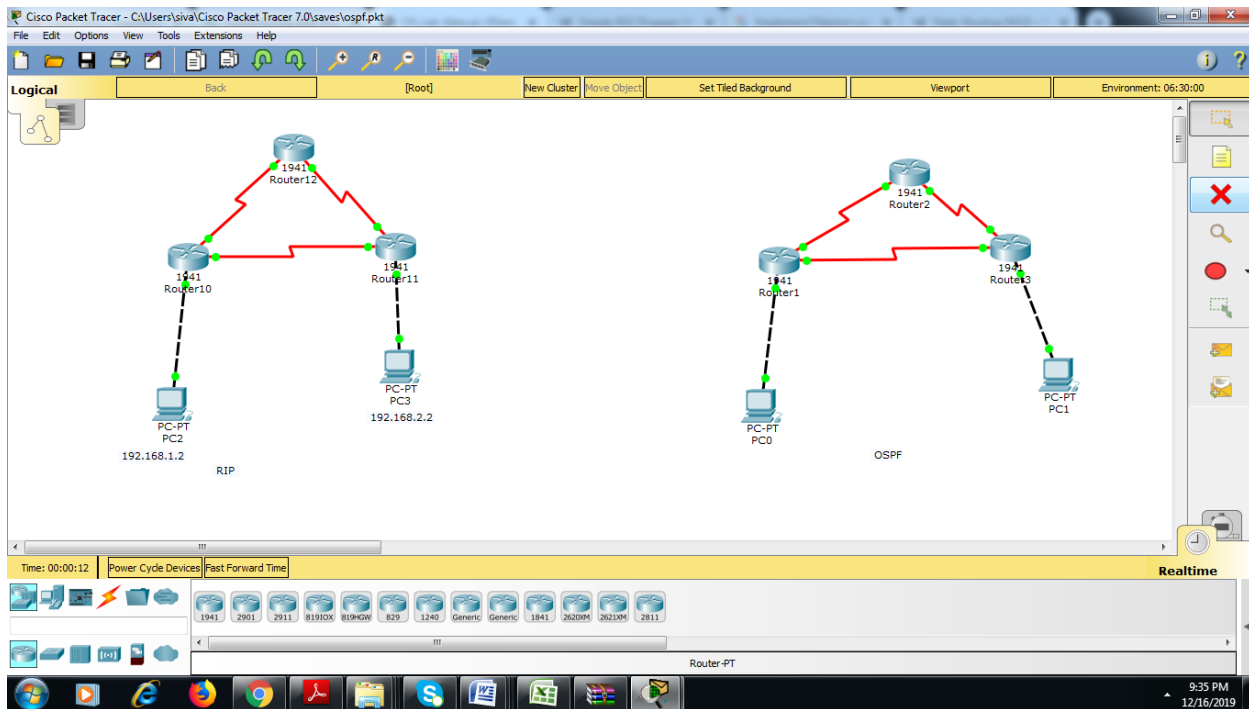
Step 5: From time-to-time, each node sends its own distance vector estimate to neighbors.

Step 6: When a node x receives new DV estimate from any neighbor v, it saves v's distance vector and it updates its own DV using B-F equation:

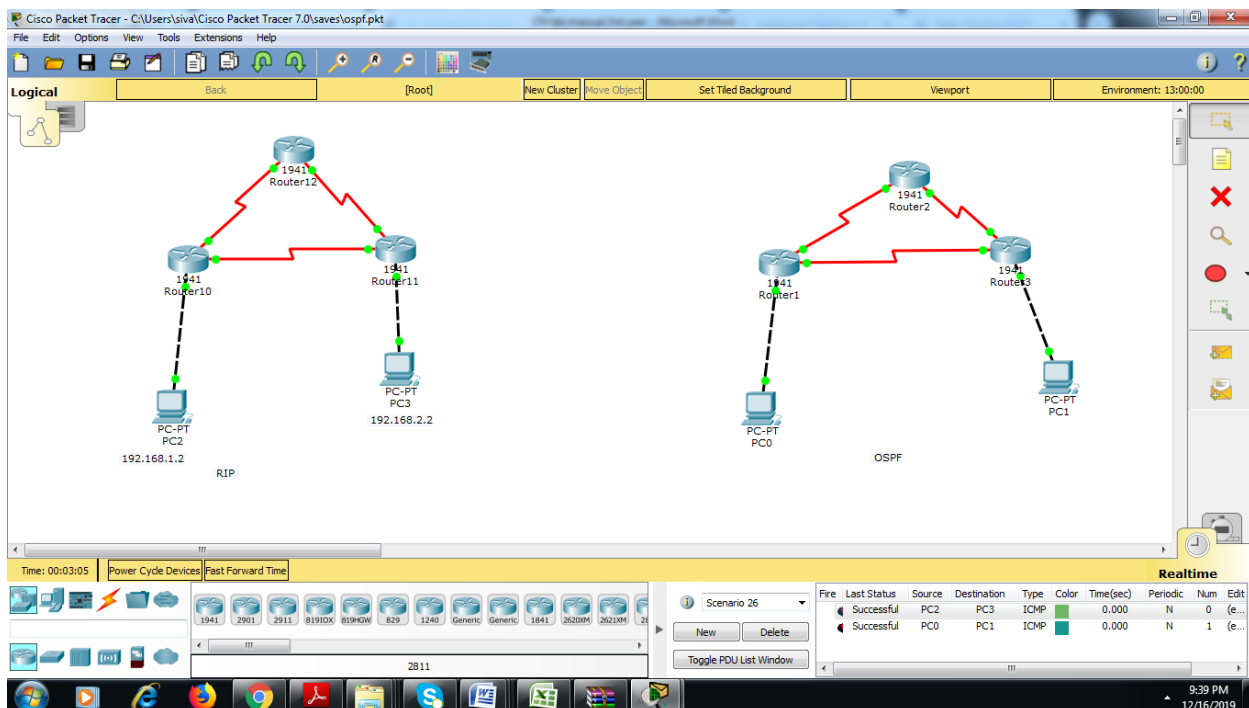
$D_x(y) = \min \{ C(x,v) + D_v(y) \}$ for each node $y \in N$

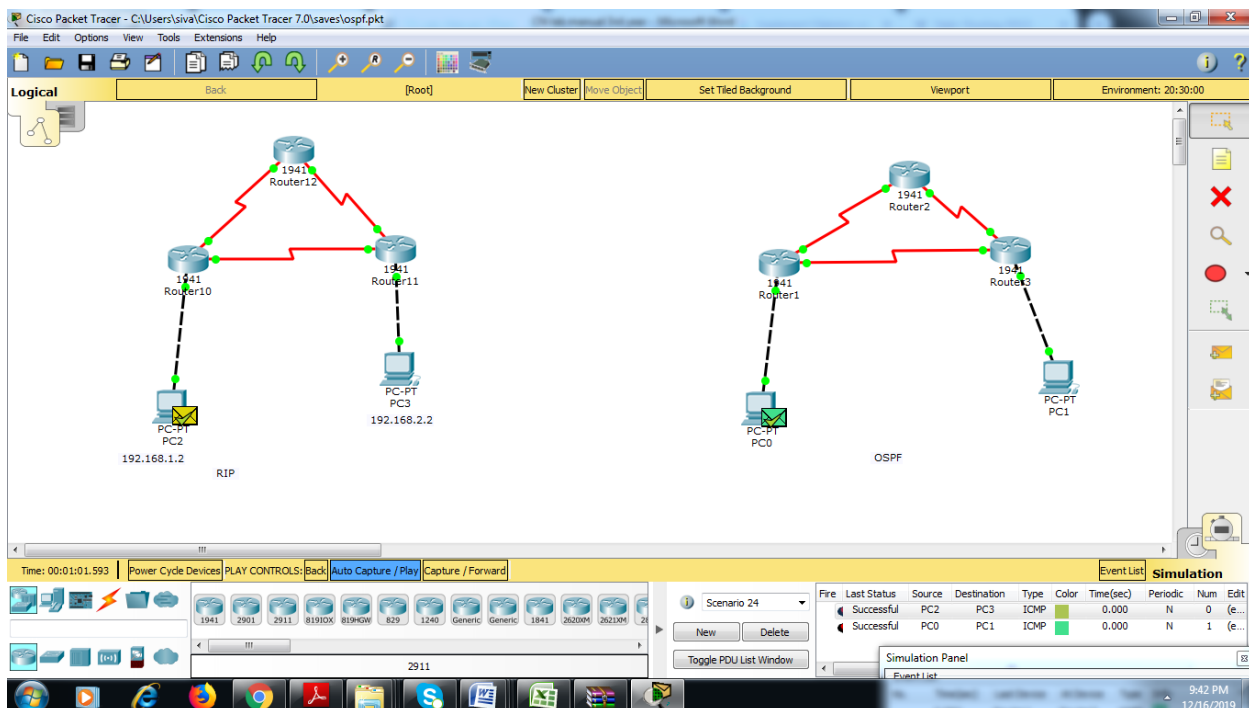
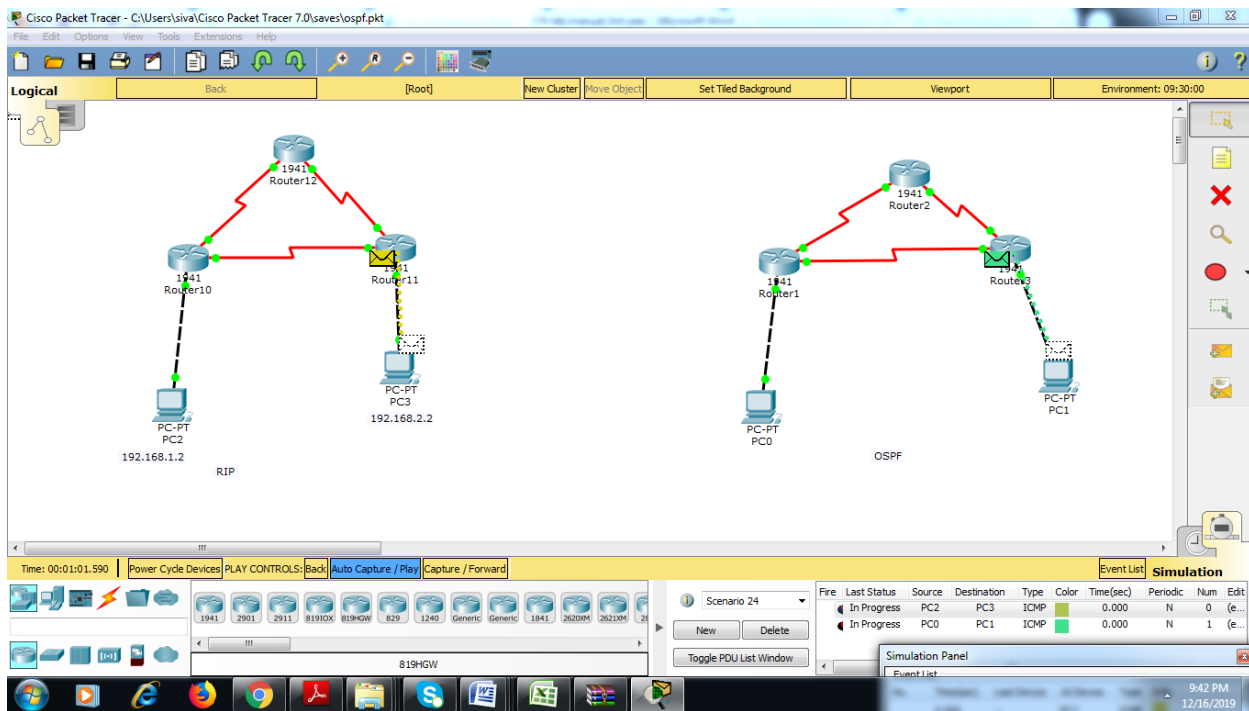
Fig a) Distance Vector Routing - Routing Information Protocol (RIP):

Fig b) Link State Routing - Open Shortest Path First(OSPF):



Output:





Week 9:

a. Write a program to implement Client - Server communication for chat using Transmission Control Protocol (TCP).

ALGORITHM :

CLIENT :

1. Start the program
2. Include necessary package in java
3. To create a socket in client to server.
4. The client establishes a connection to the server.
5. The client accept the connection and to send the data from client to server.
6. The client communicates the server to send the end of the message
7. Stop the program.

SERVER :

1. Start the program
2. Include necessary package in java
3. To create a socket in server to client
4. The server establishes a connection to the client.
5. The server accept the connection and to send the data from server to client and vice versa
6. The server communicate the client to send the end of the message.
7. Stop the program.

Program:

TCPserver1.java

```
import java.net.*;
import java.io.*;
public class TCPserver1
{
    public static void main(String arg[])
    {
        ServerSocket s=null;
        String line;
        DataInputStream is=null,is1=null;
        PrintStream os=null;
        Socket c=null;
        try
        {
            s=new ServerSocket(9999);
        }

        catch(IOException e) {

            System.out.println(e);
        }
    }
}
```

```

    try
    {
c=s.accept();
is=new DataInputStream(c.getInputStream());
is1=new DataInputStream(System.in);
    os=new PrintStream(c.getOutputStream());
    do
    {
line=is.readLine();
System.out.println("Client:"+line);
System.out.println("Server:");
line=is1.readLine();
os.println(line);
    }
    while(line.equalsIgnoreCase("quit")==false);
is.close();
os.close();
    }
    catch(IOException e)
    {
System.out.println(e);
    }
}
}

```

TCPclient1.java:

```

import java.net.*;
import java.io.*;
public class TCPclient1
{
public static void main(String arg[])
{
Socket c=null;
String line;
DataInputStream is,is1;
PrintStream os;
try
{
c=new Socket("10.0.200.36",9999);
}

catch(IOException e)
{
System.out.println(e);
}
try

```

```

{
os=new PrintStream(c.getOutputStream());
is=new DataInputStream(System.in);
is1=new DataInputStream(c.getInputStream());
do
{
System.out.println("Client:");
line=is.readLine();
os.println(line);
System.out.println("Server:" + is1.readLine());
}
while(line.equalsIgnoreCase("quit")==false);
is1.close();

os.close();
}
catch(IOException e)
{
System.out.println("Socket Closed!Message Passing is over");
}}

```

OUTPUT:

SERVER

```

C:\Program Files\Java\jdk1.5.0\bin>javac TCPserver1.java
Note: TCPserver1.java uses or overrides a deprecated API.
Note: Recompile with -deprecation for details.
C:\Program Files\Java\jdk1.5.0\bin>java TCPserver1
Client: Hai Server
Server: Hai Client
Client: How are you
Server: Fine
Client: quit
Server: quit

```

CLIENT

```

C:\Program Files\Java\jdk1.5.0\bin>javac TCPclient1.java
Note: TCPclient1.java uses or overrides a deprecated API.
Note: Recompile with -deprecation for details.
C:\Program Files\Java\jdk1.5.0\bin>java TCPclient1
Client: Hai Server
Server: Hai Client
Client: How are you
Server: Fine
Client: quit
Server: quit

```

Week 10: Configure FTP Server on a Linux/Windows machine using a FTP client.
Use a TFTP client and repeat the experiment

CONFIGURING FTP SERVER:

FTP stand for **F**ile **T**ransfer **P**rotocol. As the name suggest this network protocol allows you to transfer files or directories from one host to another over the network whether it is your LAN or Internet.

Configure FTP Server:

vsftpd package is required for FTP Server. Check whether package is installed or not. If package is missing install it first.

```
[root@server ~]# rpm -qa vsftpd*  
vsftpd-2.2.2-6.el6_0.1.x86_64  
[root@server ~]# _
```

Configure **vsftpd** service to start at boot

```
[root@server ~]# chkconfig vsftpd on  
[root@server ~]# _
```

Current status of **vsftpd** service must be running. Start if it is stopped. Restart **vsftpd** service whenever you made any change in configuration file.

```
[root@server ~]# service vsftpd status
vsftpd is stopped
[root@server ~]# service vsftpd start
Starting vsftpd for vsftpd: [ OK ]
[root@server ~]# service vsftpd restart
Shutting down vsftpd: [ OK ]
Starting vsftpd for vsftpd: [ OK ]
[root@server ~]# service vsftpd status
vsftpd (pid 3331) is running...
[root@server ~]# _
```

FTP Server is by default configured to listen on port 21. Port 21 must be opened if you have configured firewall. The configuration of a firewall for an FTP server is a relatively simple process.

```
#iptables -A INPUT -m state --state NEW -m tcp -p tcp --dport 21 -j ACCEPT
```

```
[root@server ~]# iptables -A INPUT -m state --state NEW -m tcp -p tcp --dport 21
-j ACCEPT
[root@server ~]# _
```

Create 2 normal user accounts for testing. Create a normal user.

```
[root@server ~]# useradd sanjay
[root@server ~]# passwd sanjay
Changing password for user sanjay.
New password:
BAD PASSWORD: it is too simplistic/systematic
BAD PASSWORD: is too simple
Retype new password:
passwd: all authentication tokens updated successfully.
[root@server ~]# _
```

Create another normal user:

```
[root@server ~]# useradd vikarm
[root@server ~]# passwd vikarm
Changing password for user vikarm.
New password:
BAD PASSWORD: it is too simplistic/systematic
BAD PASSWORD: is too simple
Retype new password:
passwd: all authentication tokens updated successfully.
[root@server ~]# _
```

That's all configure we need on server right now.

Configure FTP client

You will not be able to run ftp command. By default you will get following error

-bash: ftp: command not found error

```
[root@linuxclient ~]# ftp 192.168.1.1
-bash: ftp: command not found
[root@linuxclient ~]# _
```

To run **ftp** command **ftp** package is required. Install it if it is not installed.

```
[root@linuxclient ~]# rpm -qa ftp*
[root@linuxclient ~]# cd /media/RHEL_6.1\ x86_64\ Disc\ 1/Packages/
[root@linuxclient Packages]# rpm -ivh ftp*
warning: ftp-0.17-51.1.el6.x86_64.rpm: Header V3 RSA/SHA256 Signature
431d51: NOKEY
Preparing... #####
1:ftp #####
[root@linuxclient Packages]# cd
[root@linuxclient ~]# rpm -qa ftp*
ftp-0.17-51.1.el6.x86_64
[root@linuxclient ~]# _
```

Check connectivity with FTP Server.

```
[root@linuxclient ~]# ping 192.168.1.1
PING 192.168.1.1 (192.168.1.1) 56(84) bytes of data.
64 bytes from 192.168.1.1: icmp_seq=1 ttl=64 time=1.77 ms
^C
--- 192.168.1.1 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 932ms
rtt min/avg/max/mdev = 1.773/1.773/1.773/0.000 ms
[root@linuxclient ~]# _
```

Now try again to run **ftp** command

```
[root@linuxclient ~]# ftp 192.168.1.1
Connected to 192.168.1.1 (192.168.1.1).
220 (vsFTPd 2.2.2)
Name (192.168.1.1:root): _
```

We have successfully connected with FTP server.

Go on Server system and open main ftp configuration file **/etc/vsftpd/vsftpd.conf**

```
[root@server ~]# vi /etc/vsftpd/vsftpd.conf
```

vsftpd.conf is the main configuration file of FTP server and it contains lot of directives. Configuration of an anonymous-only download is relatively simple. Default configuration of **vsftpd.conf** already supports anonymous-only download. But it also supports access from local users. All you need to do is disable the directive which allows locally configured users to login with their accounts.

Comment following directives and save the file

```
#
# Allow anonymous FTP? (Beware - allowed by default)
anonymous_enable=YES
#
# Uncomment this to allow local users to log in.
#local_enable=YES _____ Comment this
#
# Uncomment this to enable any form of FTP write
write_enable=YES
```

Restart the **vsftpd** service

```
[root@server ~]# service vsftpd restart
Shutting down vsftpd: [ OK ]
Starting vsftpd for vsftpd: [ OK ]
[root@server ~]# _
```

When a user connects on the FTP server with anonymous **username**, actually that user connects on the server as a user named **ftp**. RHEL6 automatically create this account with following setting.

```
ftp:x:14:50:FTP User:/var/ftp:/sbin/nologin
```

```
[root@server ~]# cat /etc/passwd | grep ftp
ftp:x:14:50:FTP User:/var/ftp:/sbin/nologin
[root@server ~]# _
```

With these setting users are not allowed to login as the user named **ftp**. So they need to use **anonymous** as user name. So whenever an **anonymous** user logged in, he is taken to ftp user's home directory **/var/ftp**. So if you want to change the default directory associated with anonymous logins, change the home directory associated with the local user named **ftp**. Create a file on the root of the ftp directory **/var/ftp/pub**. This file will be downloaded by anonymous user.

```
# dd if=/dev/null of=/var/ftp/pub/file bs=1024 count=1000
```

```
[root@server ~]# dd if=/dev/null of=/var/ftp/pub/test_file bs=1024 count=1000
0+0 records in
0+0 records out
0 bytes (0 B) copied, 0.000114734 s, 0.0 kB/s
[root@server ~]# _
```

If you are running Linux without SELinux that's all setting which we need for this exercise. SELinux is listed in RHCE6 exam objective. So if you have configured SELinux, also configure following boolean option.

```
# chcon -R -t public_content_t /var/ftp/pub/
```

```
[root@server ~]# chcon -R -t public_content_t /var/ftp/pub/
[root@server ~]# _
```

Go on **linuxclient** system and login

```
[root@linuxclient ~]# ftp 192.168.1.1
Connected to 192.168.1.1 (192.168.1.1).
220 (vsFTPd 2.2.2)
Name (192.168.1.1:root): anonymous
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> cd pub
250 Directory successfully changed.
ftp> dir
227 Entering Passive Mode (192,168,1,1,158,116).
150 Here comes the directory listing.
-rw-r--r--    1 0      0          0 Sep 17 18:56 test_file
226 Directory send OK.
ftp> get test_file
local: test_file remote: test_file
227 Entering Passive Mode (192,168,1,1,198,22).
150 Opening BINARY mode data connection for test_file (0 bytes).
226 Transfer complete.
ftp> bye
221 Goodbye.
[root@linuxclient ~]# _
```

to the

FTP server as anonymous user and download **test_file**

Most commonly commands used on ftp prompt are

- put** To upload files on server
- get** To download files from server
- mput** To upload all files
- mget** To download all files
- ?** To see all available command on ftp prompts
- cd** To change remote directory
- lcd** To change local directory.

Create a sample file

```
[root@linuxclient ~]# cat > sample_file
this is sample file
[root@linuxclient ~]# _
```

Login from **anonymous** again and try to upload

```
[root@linuxclient ~]# ftp 192.168.1.1
Connected to 192.168.1.1 (192.168.1.1).
220 (vsFTPd 2.2.2)
Name (192.168.1.1:root): anonymous
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> put sample_file
local: sample_file remote: sample_file
227 Entering Passive Mode (192,168,1,1,88,33).
550 Permission denied.
ftp> _
```

Try to login form normal user

```
[root@linuxclient ~]# ftp 192.168.1.1
Connected to 192.168.1.1 (192.168.1.1).
220 (vsFTPd 2.2.2)
Name (192.168.1.1:root): sanjay
530 This FTP server is anonymous only.
Login failed.
ftp> _
```

Restrict anonymous user to ftp directory.

To test this login form anonymous user again

```
[root@linuxclient ~]# ftp 192.168.1.1
Connected to 192.168.1.1 (192.168.1.1).
220 (vsFTPd 2.2.2)
Name (192.168.1.1:root): anonymous
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> _
```

Try to change parent directory

```

230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> pwd
257 "/"
ftp> cd ..
250 Directory successfully changed.
ftp> pwd
257 "/"
ftp> dir
227 Entering Passive Mode (192,168,1,1,169,144).
150 Here comes the directory listing.
drwxr-xr-x  2 0      0          4096 Sep 17 18:56 pub
226 Directory send OK.
ftp> cd pub
250 Directory successfully changed.
ftp> pwd
257 "/pub"
ftp> cd ..
250 Directory successfully changed.
ftp> pwd
257 "/"
ftp> cd ..
250 Directory successfully changed.
ftp> _

```

home directory is /var/ftp/ you cannot move to parent directory /var/

child directory is pub you can move in it

FTP non-anonymous server

In this exercise we will configure FTP server that allow local users logins to their home directories. Download/upload must be allowed for these users. Go on server system and open

/etc/vsftpd/vsftpd.conf file

```
[root@server ~]# vi /etc/vsftpd/vsftpd.conf
```

Comment **anonymous_login=YES**, uncomment **local_enable** and save the file

```

# Allow anonymous FTP? (Beware - allowed by default)
#anonymous_enable=YES ——— comment this
#
# Uncomment this to allow local users to log in.
local_enable=YES ——— uncomment this
#

```

open/etc/vsftpd/user_list file

```
[root@server ~]# vi /etc/vsftpd/user_list
```

Users listed on /etc/vsftpd/user_list are not allowed to login on FTP server. Add user **vikarm** in it. This file also have an entry for root user that why root user is denied from FTP login. If you

want to enable root user for ftp session just remove its entry from this file [Enable root for FTP session is not recommended in any circumstances, change at your own risk].

```
# vsftpd userlist
# If userlist_deny=NO,
# If userlist_deny=YES
# do not even prompt for password
# Note that the default
# for users that are d
root
bin
daemon
adm
lp
sync
shutdown
halt
mail
news
uucp
operator
games
nobody
vikram
```

Configure SELinux to allow upload/download in user's home directory

```
[root@server ~]# setsebool ftp_home_dir 1
[root@server ~]# _
```

Restart the **vsftpd** service

```
[root@server ~]# service vsftpd restart
Shutting down vsftpd: [ OK ]
Starting vsftpd for vsftpd: [ OK ]
[root@server ~]# _
```

Login from normal user **sanjay** and create **aexample_file**

```
Red Hat Enterprise Linux Server release 6.1 (Santiago)
Kernel 2.6.32-131.0.15.el6.x86_64 on an x86_64

server login: sanjay
Password:
[sanjay@server ~]# cat > example_file
this is example file created on server with normal user
account. This file will be downloaded from ftp client
[sanjay@server ~]# _
```

Come back on **linuxclient** system and try to login from user **vikram**

```
[root@linuxclient ~]# ftp 192.168.1.1
Connected to 192.168.1.1 (192.168.1.1).
220 (vsFTPd 2.2.2)
Name (192.168.1.1:root): vikram
530 Permission denied.
Login failed.
ftp> bye
221 Goodbye.
[root@linuxclient ~]# _
```

Now try to login from user **sanjay**

```
[root@linuxclient ~]# ftp 192.168.1.1
Connected to 192.168.1.1 (192.168.1.1).
220 (vsFTPd 2.2.2)
Name (192.168.1.1:root): sanjay
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> _
```

upload/download file

```
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> dir
227 Entering Passive Mode (192,168,1,1,229,60).
150 Here comes the directory listing.
-rw-rw-r-- 1 508 508 110 Sep 17 22:54 example_file
226 Directory send OK.
ftp> get example_file
local: example_file remote: example_file
227 Entering Passive Mode (192,168,1,1,75,105).
150 Opening BINARY mode data connection for example_file (110 bytes).
226 Transfer complete.
110 bytes received in 9.7e-05 secs (1134.02 Kbytes/sec)
ftp> put sample_file
local: sample_file remote: sample_file
227 Entering Passive Mode (192,168,1,1,141,16).
150 Ok to send data.
226 Transfer complete.
21 bytes sent in 0.0243 secs (0.86 Kbytes/sec)
ftp> bye
221 Goodbye.
[root@linuxclient ~]# cat example_file
this is example file created on server with normal user
account. This file will be downloaded from ftp client
[root@linuxclient ~]# _
```

Login again from normal user and try to change parent directory

```
[root@linuxclient ~]# ftp 192.168.1.1
Connected to 192.168.1.1 (192.168.1.1).
220 (vsFTPD 2.2.2)
Name (192.168.1.1:root): sanjay
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> pwd
257 "/home/sanjay"
ftp> cd /
250 Directory successfully changed.
ftp> _
```

It allows you to navigate the / directory which serious security issue.

Configure FTP to chroot local users in their home directory:

Go on server and open `/etc/vsftpd/vsftpd.conf` file

```
[root@server ~]# vi /etc/vsftpd/vsftpd.conf
```

Uncomment following directive and save the file

```
chroot_local_user=YES
```

```
# You may specify an explicit list
# directory. If chroot_local_user
# users to NOT chroot().
chroot_local_user=YES
#chroot_list_enable=YES
# (default follows)
```

Restart the **vsftpd** restart

```
[root@server ~]# service vsftpd restart
Shutting down vsftpd: [ OK ]
Starting vsftpd for vsftpd: [ OK ]
[root@server ~]# _
```

Come back on linux client system and login form sanjay and try again to change directory to /

```
[root@linuxclient ~]# ftp 192.168.1.1
Connected to 192.168.1.1 (192.168.1.1).
220 (vsFTPd 2.2.2)
Name (192.168.1.1:root): sanjay
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> pwd
257 "/"
ftp> cd /
250 Directory successfully changed.
ftp> pwd
257 "/"
ftp> dir
227 Entering Passive Mode (192,168,1,1,228,189).
150 Here comes the directory listing.
-rw-rw-r-- 1 508 508 110 Sep 17 22:54 example_file
-rw-r--r-- 1 508 508 21 Sep 17 23:06 sample_file
226 Directory send OK.
ftp> bye
221 Goodbye.
[root@linuxclient ~]# _
```

Now user sanjay
is chrooted in his
home directory

Now normal user will not be able to navigate outside the home directory.

Week 11: Install Telnet on one of the systems connected by a switch and telnet to it from the other system. Using Wireshark tool, capture the packets and analyze the TCP 3-way Handshake for connection establishment and tear down.

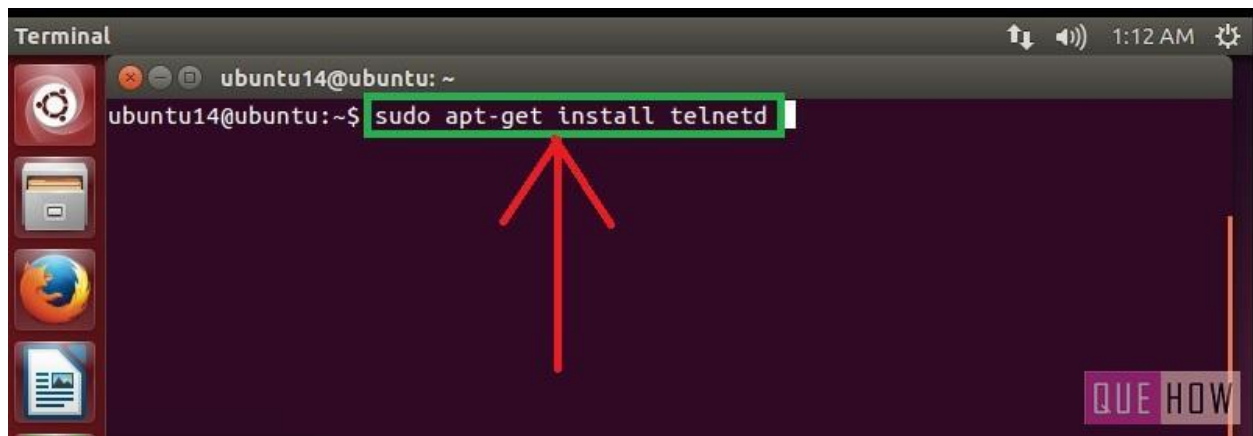
Telnet:

TELNET (Terminal Network (or) Teletype Network) is the service that allows you to access another computer by logging in to the remote system, such that, you can pass commands to another system by sitting back at your place. It allows you to send the instructions to a remote client, but in a plain-text format. Telnet listens to all the requests by the user usually on TCP port 23.

Installation of Telnet:

Step 1: sudo apt-get install telnetd

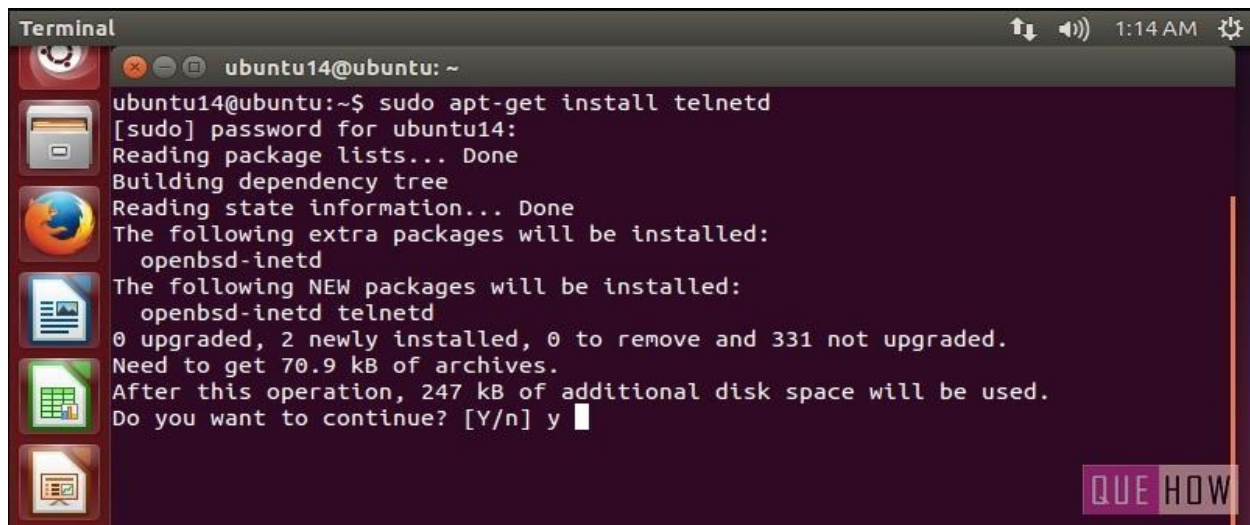
This will complete the installation of Telnet. “**telnetd**” is a daemon that gets invoked by “*inetd*” or its extension “*xinetd*”, both are the internet servers.



Step 2:

Then you are asked to enter the user password and then press enter. Processing will start as soon as you press enter. After this, I have noticed a line “**274 KB additional disk space will be used**” on the terminal screen.

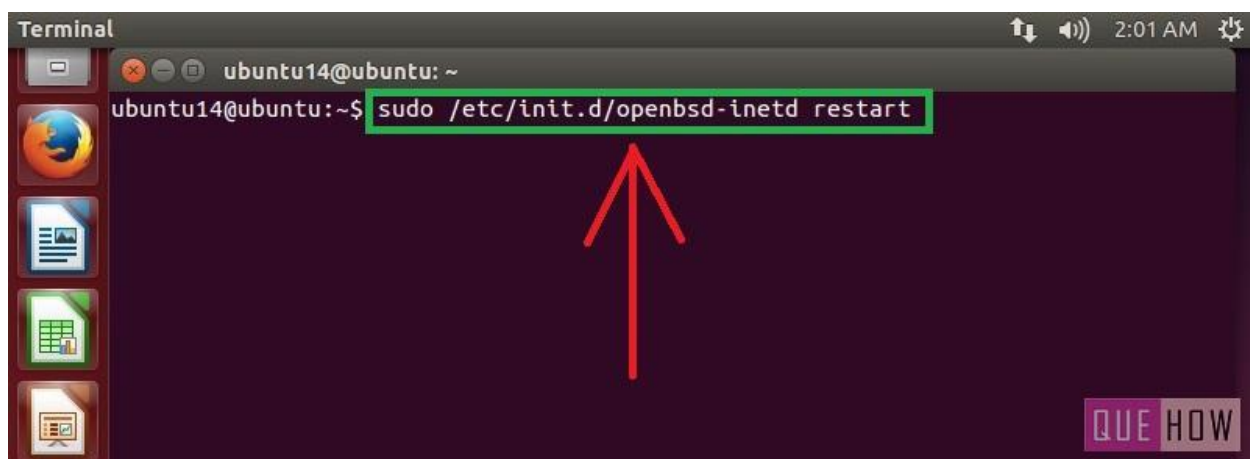
You may also observe some sort of a message like this and then you'll be asked to continue or not. Just write "y" and then press enter to continue.

A terminal window titled 'Terminal' with a dark background. The prompt is 'ubuntu14@ubuntu: ~'. The user has entered 'sudo apt-get install telnetd'. The terminal shows the password prompt, package list reading, dependency tree building, and state information reading. It lists 'openbsd-inetd' as an extra package and 'openbsd-inetd telnetd' as new packages. It shows '0 upgraded, 2 newly installed, 0 to remove and 331 not upgraded'. It states 'Need to get 70.9 kB of archives' and 'After this operation, 247 kB of additional disk space will be used'. It asks 'Do you want to continue? [Y/n]' and the user has entered 'y'.

```
Terminal
ubuntu14@ubuntu: ~
ubuntu14@ubuntu:~$ sudo apt-get install telnetd
[sudo] password for ubuntu14:
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following extra packages will be installed:
  openbsd-inetd
The following NEW packages will be installed:
  openbsd-inetd telnetd
0 upgraded, 2 newly installed, 0 to remove and 331 not upgraded.
Need to get 70.9 kB of archives.
After this operation, 247 kB of additional disk space will be used.
Do you want to continue? [Y/n] y
```

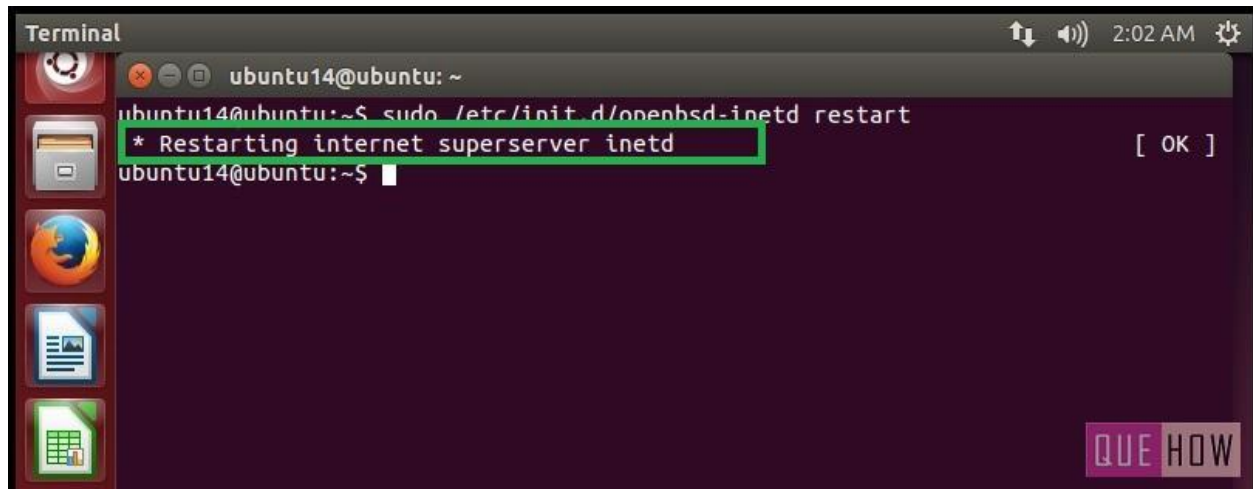
Step 3: Now when you are done with it, restart "inetd". Type "**sudo /etc/init.d/openbsd-inetd restart**".

"inetd" is daemon used for *dealing with incoming network* and it is responsible for deciding which program to run when a request comes.

A terminal window titled 'Terminal' with a dark background. The prompt is 'ubuntu14@ubuntu: ~'. The user has entered 'sudo /etc/init.d/openbsd-inetd restart'. The command is highlighted with a green box, and a red arrow points to it from below.

```
Terminal
ubuntu14@ubuntu: ~
ubuntu14@ubuntu:~$ sudo /etc/init.d/openbsd-inetd restart
```

Step 4: To ensure "inetd" is started, press enter after writing the above command.

A screenshot of a Linux terminal window titled 'Terminal'. The window shows a user prompt 'ubuntu14@ubuntu: ~'. The user has entered the command 'sudo /etc/init.d/openbsd-inetd restart'. The output of the command is '* Restarting internet superserver inetd', which is highlighted with a green rectangular box. To the right of the output is a '[OK]' status. The terminal window has a dark background and a sidebar on the left with various application icons. The top of the window shows system icons and the time '2:02 AM'.

To connect with any remote client:

Step 5: Just type:“**telnet hostipaddress**”. For an example: “telnet 192.168.0.68” and press enter.

Step 6: Then you”ll see, it is connected to “**host ip address**”. For security reasons, you are required to provide “username” and “password” as well.

Task: Using Wireshark, capture the packets and analyze the TCP 3-way Handshake for connection establishment and tear down.

TCP 3-way Handshake:

A **three-way handshake** is a method used in a **TCP/IP** network to create a connection between a local host/client and server. It is a **three**-step method that requires both the client and server to exchange SYN and ACK (acknowledgment) packets before actual data communication begins.

STEP 1 - The sender send a SYN packet to begin establishment of the session. It waits

STEP 2 - The receiver replies with an ACK packet to acknowledge the SYN that it received and also sends a SYN packet.

STEP 3 - The sender sends an ACK to acknowledge receipt of the SYN from the receiver.

The connection is now set up and data transfer can commence.

Now to terminate a connection you can do it the graceful way which is a four way process. Lets assume that the sender is finished with data transmission and now wants to end the session, similar to logging off an application.

STEP 1 - The sender sends a FIN packet and waits for a reply

STEP 2 - The receiver send and ACK packet

STEP 3 - The receiver also sends a FIN packet

STEP 4 - The sender sends an ACK packet and the session is closed.

You can also do an abrupt termination by just sending a RST packet from either sender or transmitter and the session ends abruptly.

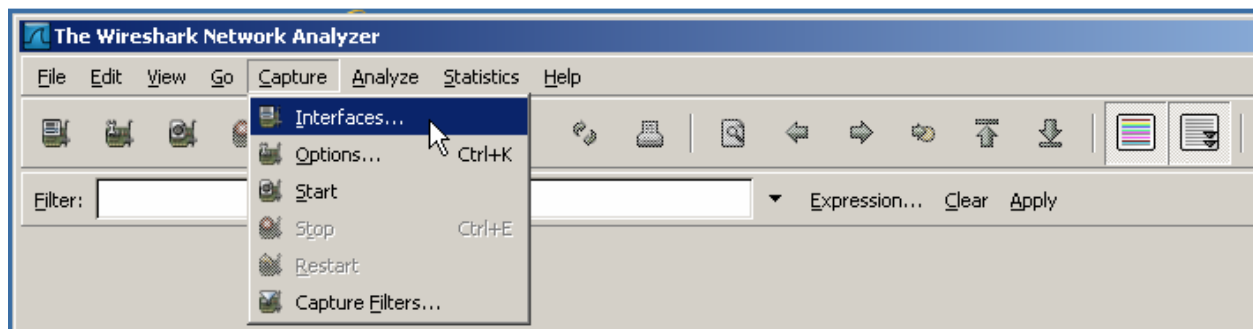
Prepare Wireshark to Capture Packets :

Step 1: Start Wireshark.

Double-click the Wireshark icon, which is located on the desktop.

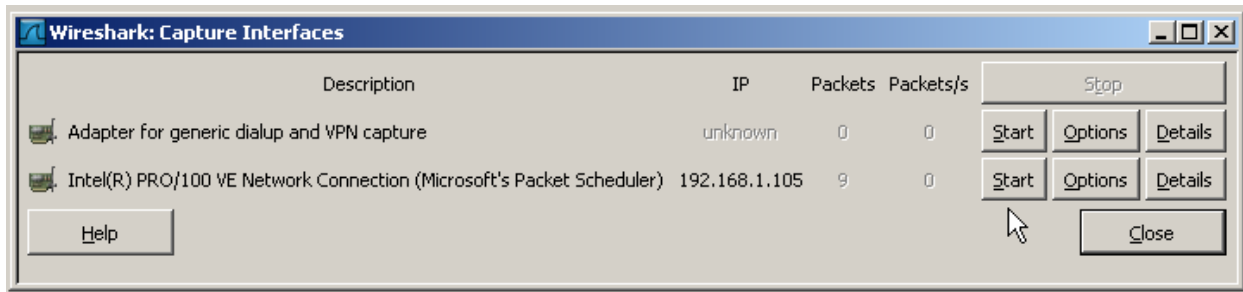
Step 2: Select an interface to use for capturing packets.

a. From the Capture menu, choose **Interfaces**.



Step 3: Start a network capture.

a. Choose the local network Ethernet interface adapter for capturing network traffic. Click the **Start** button of the chosen interface.



Generate and Analyze Captured packets:

Step 4: Open a browser and access a website.

a. go to www.google.com minimize the google window and return to wireshark.

b. The capture windows are now active. Locate Source, Destination, protocol columns on the Wireshark display screen. The HTTP data that carries web page text and graphics uses TCP for reliability.

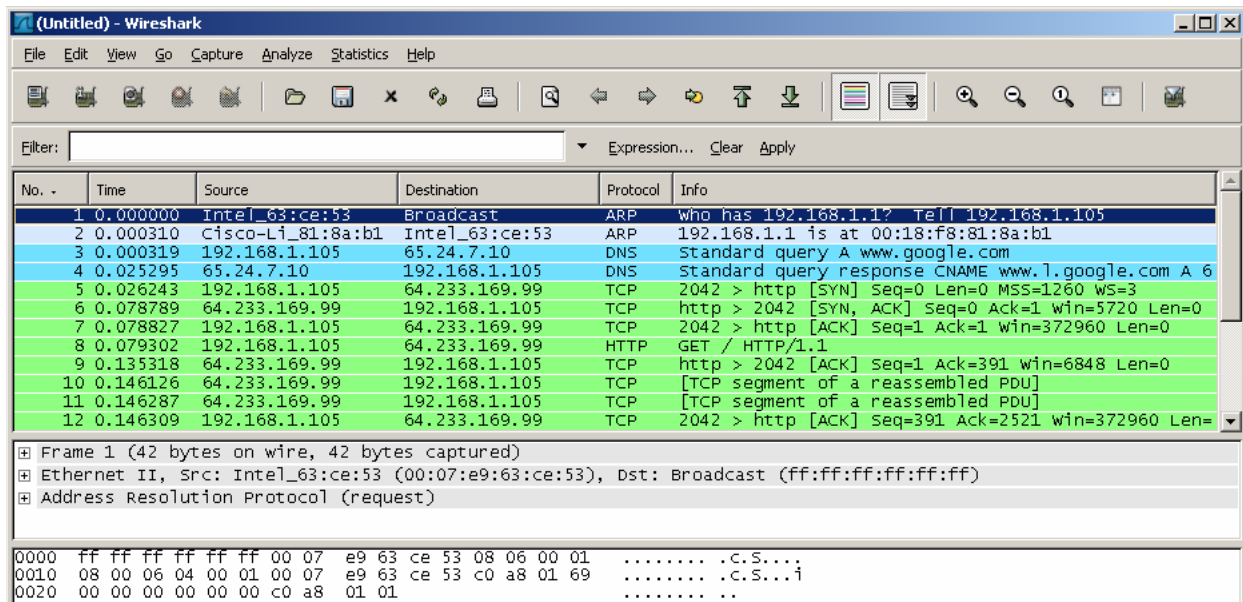
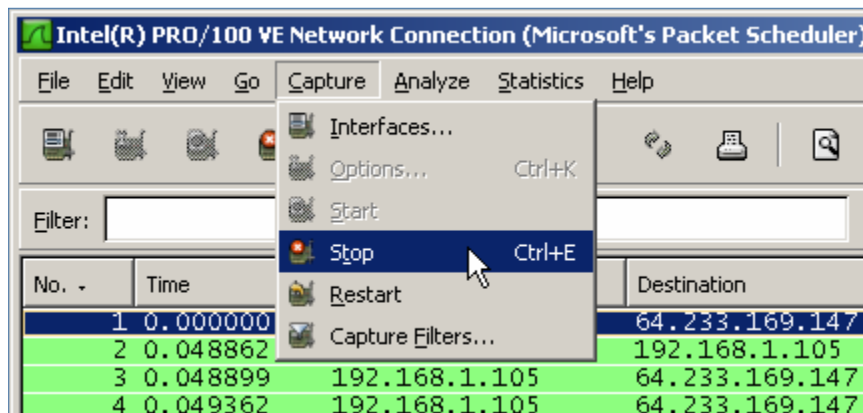


Fig. Network Captured packets

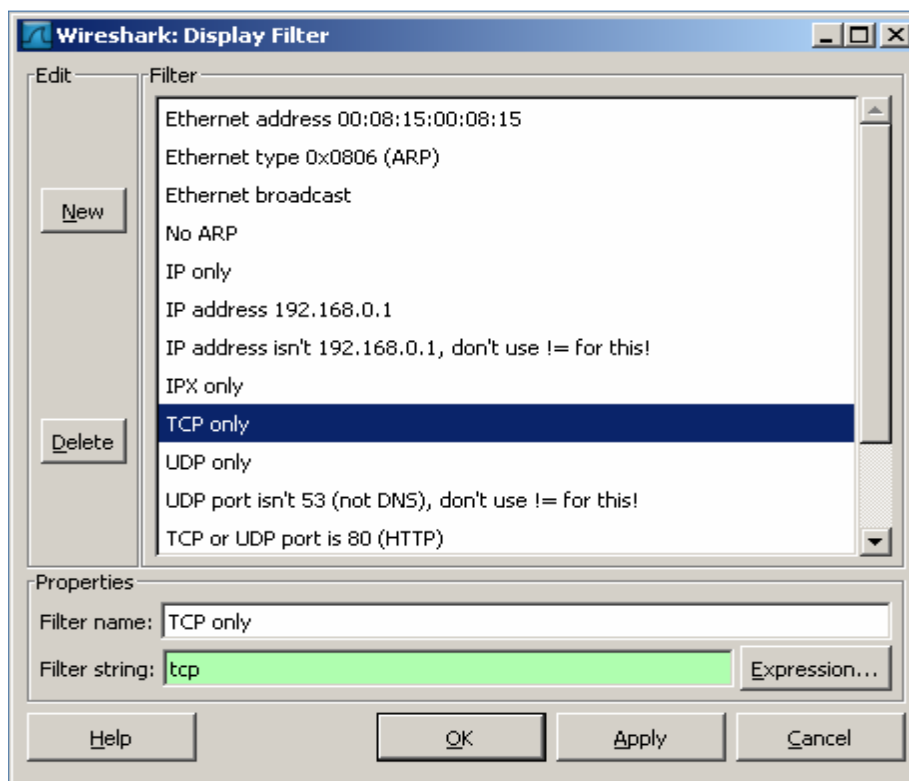
Step 5: stop the capture.



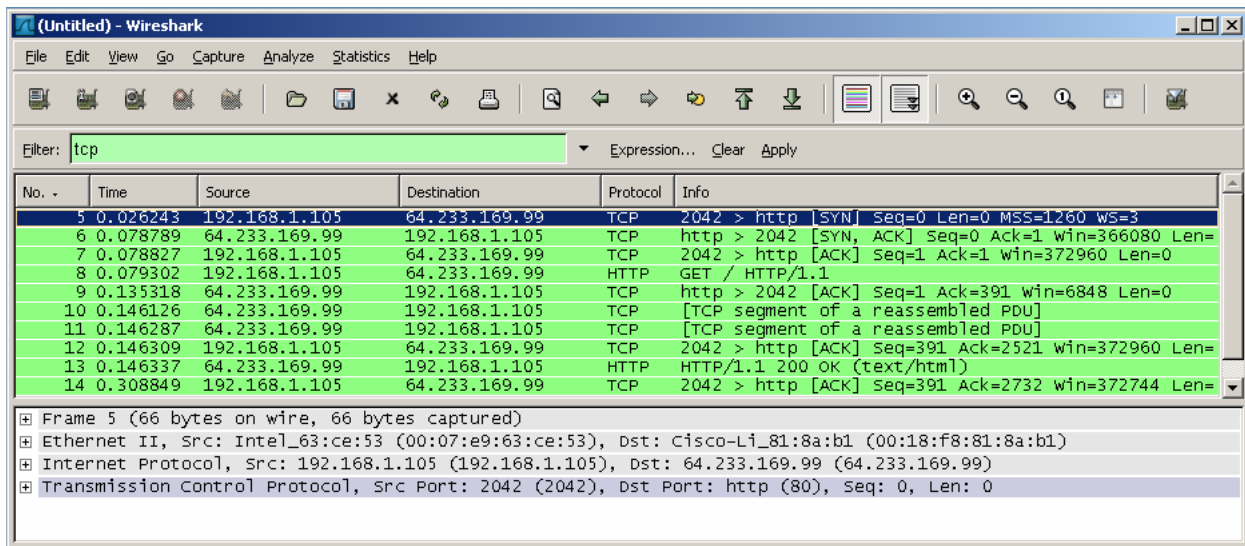
Step 6: Filter the capture to view only TCP packets.

If you have many packets that are unrelated to the TCP connection, it may be necessary to use the Wireshark filter capability.

- To use a preconfigured filter, click the **Analyze** menu option, and then click **Display Filters**.
- In the **Display Filter** window, click **TCP only**, and then click **OK**.



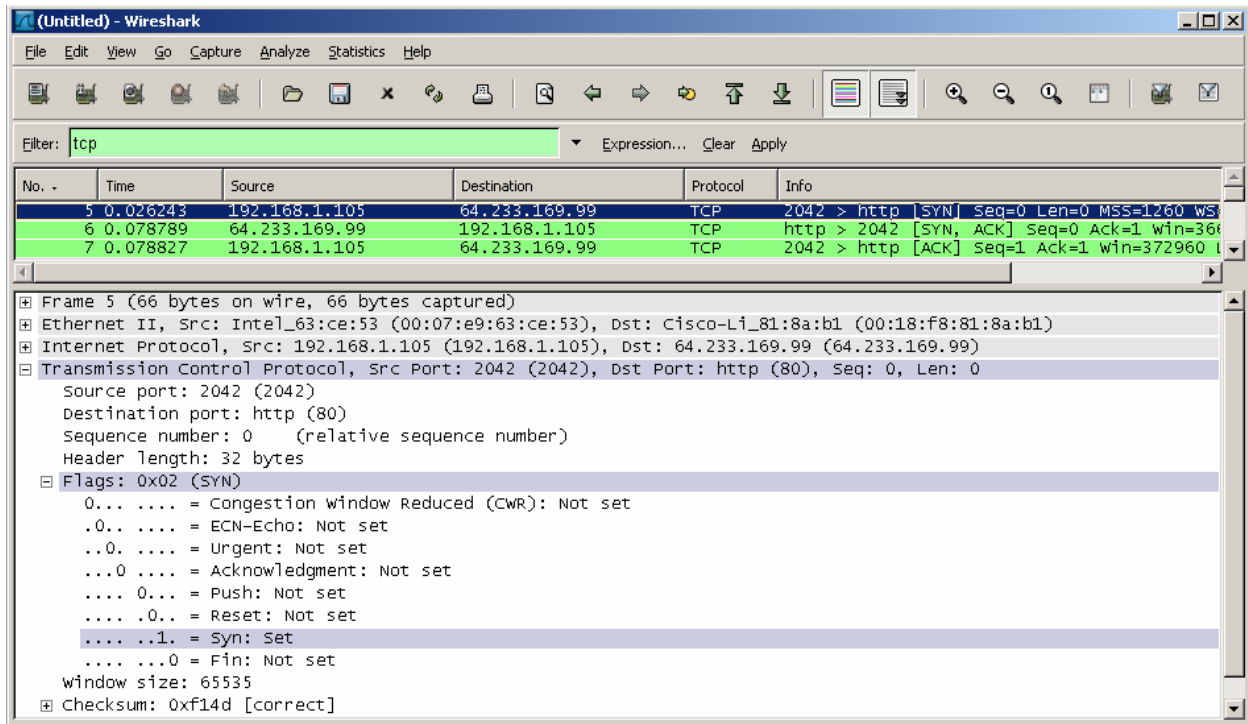
c. In the Wireshark window, scroll to the first captured TCP packet. This should be the first packet in the flow.



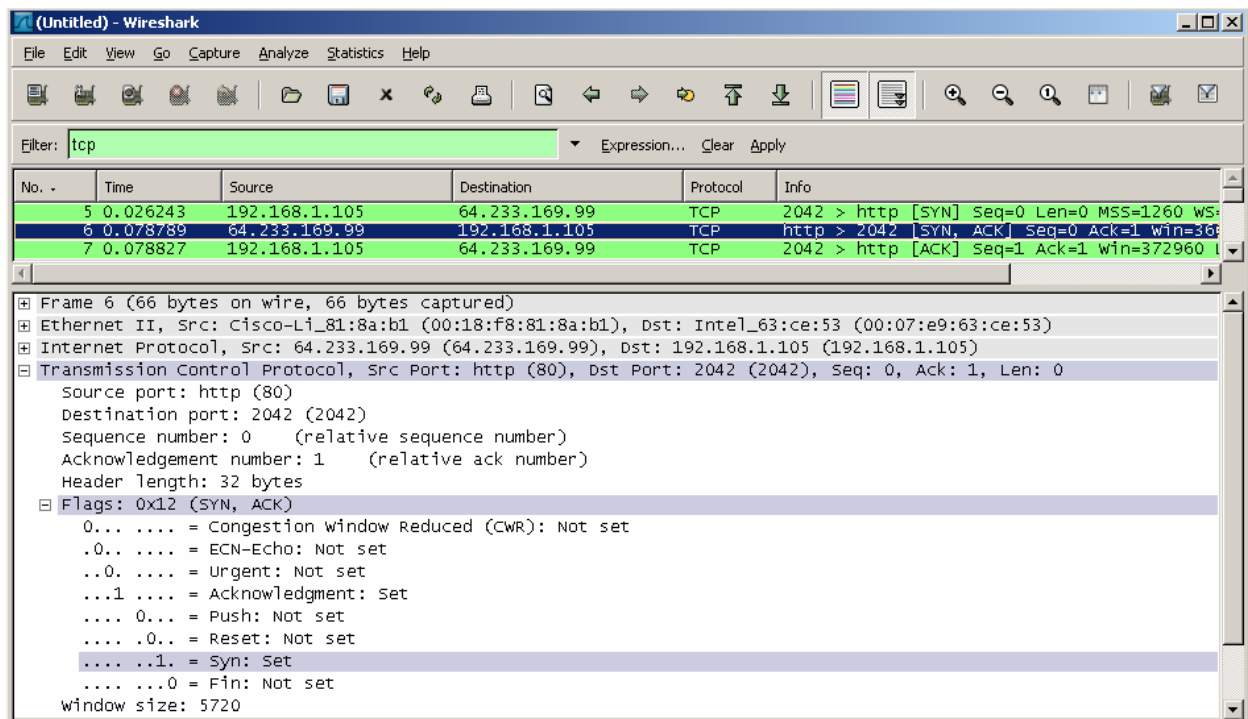
d. In the Info column, look for three packets similar to the first three shown in the window above. The first TCP packet is the [SYN] packet from the initiating computer. The second is the [SYN, ACK] response from the web server. The third packet is the [ACK] from the source computer, which completes the handshake.

Step 7: Inspect the TCP initialization Sequence

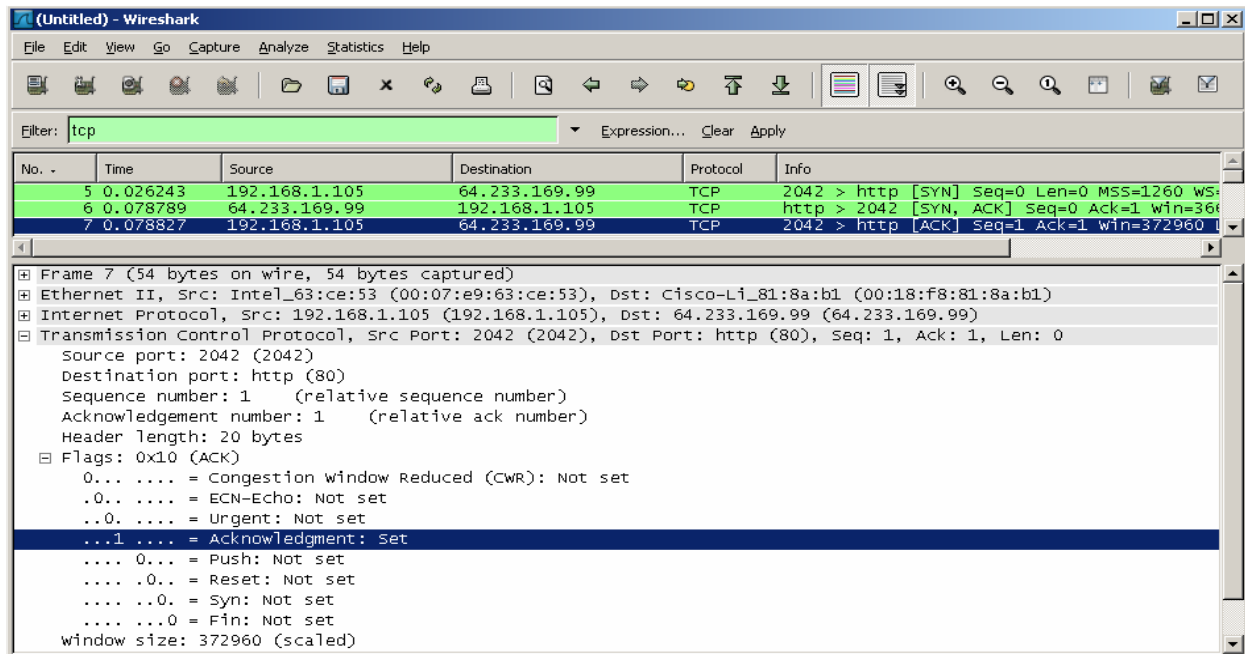
- In the top Wireshark window, click on the line containing the first packet identified in Step 4. This highlights the line and displays the decoded information from that packet in the two lower windows fill.
- Click the + icon to expand the view of the TCP information. To contract the view, click the – icon.
- Notice in the first TCP packet that the relative sequence number is set to 0, and the SYN bit is set to 1 in the Flags field.



d. Notice in the second TCP packet of the handshake that the relative sequence number is set to 0, and the SYN bit and the ACK bit are set to 1 in the Flags field.



e. In the third and final frame of the handshake, only the ACK bit is set, and the sequence number is set to the starting point of 1. The acknowledgement number is also set to 1 as a starting point. The TCP connection is now established, and communication between the source computer and the web server can begin.



OUTPUT:

TCP Connection Establishment:

1. SYN flag is set to 1:

Wireshark packet capture showing TCP connection establishment. The first packet is a SYN packet from 192.168.1.122 to 64.238.147.113 on port 80. The packet details show the SYN flag is set.

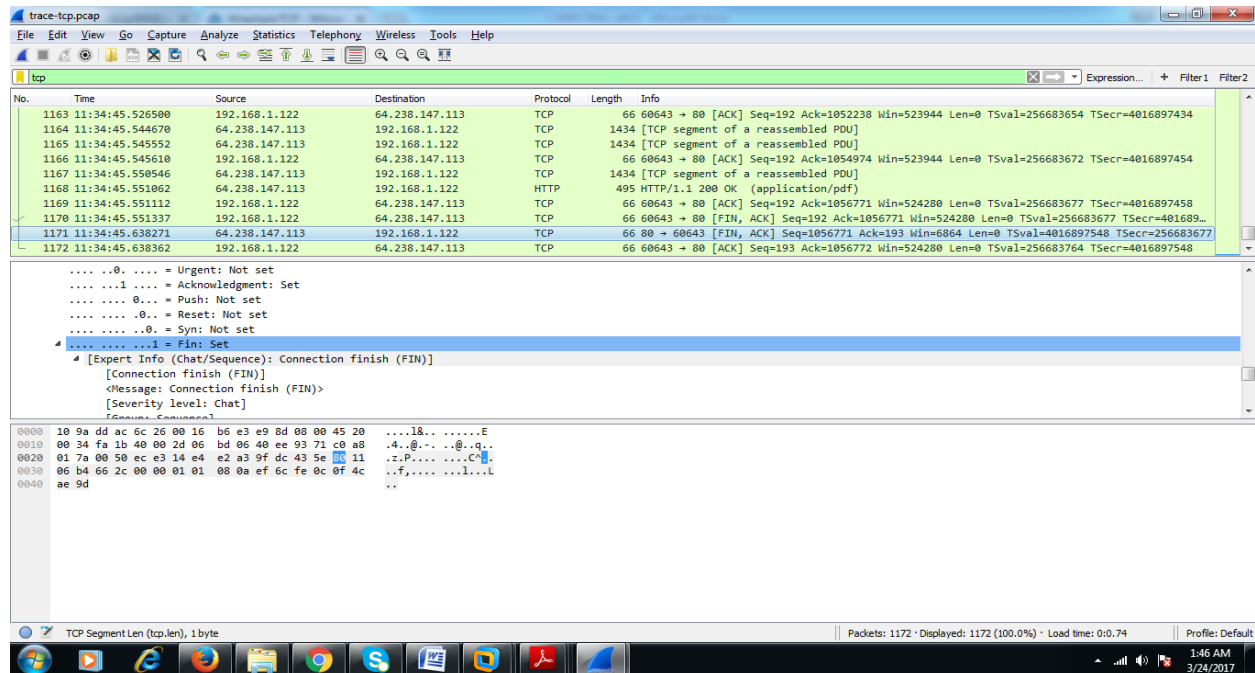
No.	Time	Source	Destination	Protocol	Length	Info
1	11:34:41.439558	192.168.1.122	64.238.147.113	TCP	78	60643 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=8 TSval=256679793 TSecr=0 SACK_PERM=1
2	11:34:41.527568	64.238.147.113	192.168.1.122	TCP	74	80 → 60643 [SYN, ACK] Seq=0 Ack=1 Win=5792 Len=0 MSS=1380 SACK_PERM=1 TSval=4016893437
3	11:34:41.527638	192.168.1.122	64.238.147.113	TCP	66	60643 → 80 [ACK] Seq=1 Ack=1 Win=524280 Len=0 TSval=256679881 TSecr=4016893437
4	11:34:41.528137	192.168.1.122	64.238.147.113	HTTP	257	GET /sigcomm/2011/papers/sigcomm/p2.pdf HTTP/1.1
5	11:34:41.617377	64.238.147.113	192.168.1.122	TCP	66	80 → 60643 [ACK] Seq=1 Ack=192 Win=6864 Len=0 TSval=4016893527 TSecr=256679881
6	11:34:41.617879	64.238.147.113	192.168.1.122	TCP	311	[TCP segment of a reassembled PDU]
7	11:34:41.617946	192.168.1.122	64.238.147.113	TCP	66	60643 → 80 [ACK] Seq=192 Ack=246 Win=524280 Len=0 TSval=256679970 TSecr=4016893528
8	11:34:41.628672	64.238.147.113	192.168.1.122	TCP	1434	[TCP segment of a reassembled PDU]
9	11:34:41.706263	64.238.147.113	192.168.1.122	TCP	1434	[TCP segment of a reassembled PDU]
10	11:34:41.706345	192.168.1.122	64.238.147.113	TCP	66	60643 → 80 [ACK] Seq=192 Ack=2982 Win=523944 Len=0 TSval=256680057 TSecr=4016893538

Flags: 0x002 (SYN)

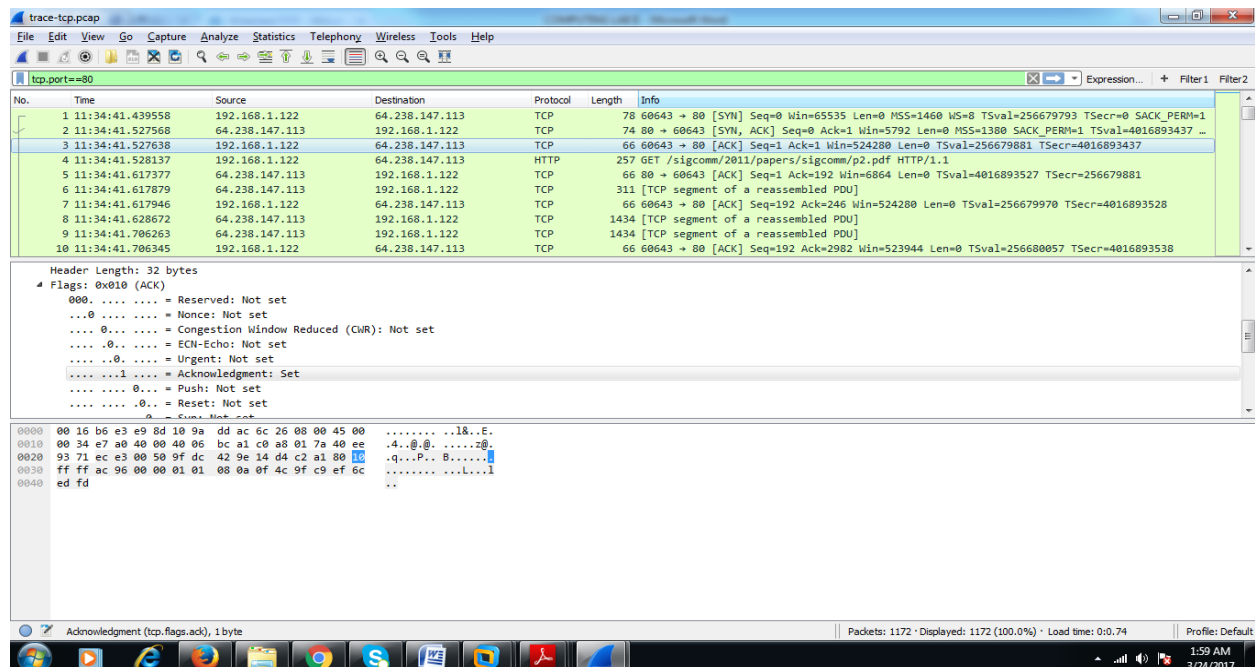
- 000. = Reserved: Not set
- ...0 = Nonce: Not set
- ...0 = Congestion Window Reduced (CWR): Not set
- ...0 = ECN-Echo: Not set
- ...0 = Urgent: Not set
- ...0 = Acknowledgment: Not set
- ...0 = Push: Not set
- ...0 = Reset: Not set
- ...1 = Syn: Set

0000 00 16 b6 e3 e9 8d 10 9a dd ac 6c 26 00 00 45 00l&..E.
0010 00 40 9f 0a 40 00 40 06 04 ac c0 a8 01 7a 40 ee@..@.....s@.
0020 93 71 ec e3 00 50 9f dc 42 9d 00 00 00 00 00 00q...P..B....
0030 ff ff 22 12 00 00 02 04 05 b4 01 03 03 01 01 01L.q.....
0040 00 0a 0f 4c 9f 71 00 00 00 00 04 02 00 00

2. Acknowledgement flag is set to 1 and SYN flag is set to 1



3. 2. Acknowledgement flag is set to 1



Connection Teardown:

Acknowledgment flag is set to 1 and Fin flag is set to 1

The screenshot shows the Wireshark interface with a packet capture of a TCP connection. The packet list on the left shows several packets, with packet 1171 selected. The packet details pane on the right shows the TCP segment for packet 1171, which is an acknowledgment (ACK) with the acknowledgment number 193 and the FIN flag set. The packet bytes pane at the bottom shows the raw data of the packet.

No.	Time	Source	Destination	Protocol	Length	Info
1163	11:34:45.526500	192.168.1.122	64.238.147.113	TCP	66	60643 → 80 [ACK] Seq=192 Ack=1052230 Win=523944 Len=0 TSval=256683654 TSecr=4016897434
1164	11:34:45.544670	64.238.147.113	192.168.1.122	TCP	1434	[TCP segment of a reassembled PDU]
1165	11:34:45.545552	64.238.147.113	192.168.1.122	TCP	1434	[TCP segment of a reassembled PDU]
1166	11:34:45.545610	192.168.1.122	64.238.147.113	TCP	66	60643 → 80 [ACK] Seq=192 Ack=1054974 Win=523944 Len=0 TSval=256683672 TSecr=4016897454
1167	11:34:45.550546	64.238.147.113	192.168.1.122	TCP	1434	[TCP segment of a reassembled PDU]
1168	11:34:45.551062	64.238.147.113	192.168.1.122	HTTP	495	HTTP/1.1 200 OK (application/pdf)
1169	11:34:45.551112	192.168.1.122	64.238.147.113	TCP	66	60643 → 80 [ACK] Seq=192 Ack=1056771 Win=524200 Len=0 TSval=256683677 TSecr=4016897458
1170	11:34:45.551337	192.168.1.122	64.238.147.113	TCP	66	60643 → 80 [FIN, ACK] Seq=192 Ack=1056771 Win=524200 Len=0 TSval=256683677 TSecr=4016897458
1171	11:34:45.638271	64.238.147.113	192.168.1.122	TCP	66	80 → 60643 [FIN, ACK] Seq=1056771 Ack=193 Win=6864 Len=0 TSval=4016897548 TSecr=256683677
1172	11:34:45.638362	192.168.1.122	64.238.147.113	TCP	66	60643 → 80 [ACK] Seq=193 Ack=1056772 Win=524200 Len=0 TSval=256683764 TSecr=4016897548

Admnowledgment (tcp.flags.ack), 1 byte

Packets: 1172 · Displayed: 1172 (100.0%) · Load time: 0:0.74 · Profile: Default

The screenshot shows the Wireshark interface with a packet capture of a TCP connection. The packet list on the left shows several packets, with packet 1171 selected. The packet details pane on the right shows the TCP segment for packet 1171, which is an acknowledgment (ACK) with the acknowledgment number 193 and the FIN flag set. The packet bytes pane at the bottom shows the raw data of the packet.

No.	Time	Source	Destination	Protocol	Length	Info
1163	11:34:45.526500	192.168.1.122	64.238.147.113	TCP	66	60643 → 80 [ACK] Seq=192 Ack=1052230 Win=523944 Len=0 TSval=256683654 TSecr=4016897434
1164	11:34:45.544670	64.238.147.113	192.168.1.122	TCP	1434	[TCP segment of a reassembled PDU]
1165	11:34:45.545552	64.238.147.113	192.168.1.122	TCP	1434	[TCP segment of a reassembled PDU]
1166	11:34:45.545610	192.168.1.122	64.238.147.113	TCP	66	60643 → 80 [ACK] Seq=192 Ack=1054974 Win=523944 Len=0 TSval=256683672 TSecr=4016897454
1167	11:34:45.550546	64.238.147.113	192.168.1.122	TCP	1434	[TCP segment of a reassembled PDU]
1168	11:34:45.551062	64.238.147.113	192.168.1.122	HTTP	495	HTTP/1.1 200 OK (application/pdf)
1169	11:34:45.551112	192.168.1.122	64.238.147.113	TCP	66	60643 → 80 [ACK] Seq=192 Ack=1056771 Win=524200 Len=0 TSval=256683677 TSecr=4016897458
1170	11:34:45.551337	192.168.1.122	64.238.147.113	TCP	66	60643 → 80 [FIN, ACK] Seq=192 Ack=1056771 Win=524200 Len=0 TSval=256683677 TSecr=4016897458
1171	11:34:45.638271	64.238.147.113	192.168.1.122	TCP	66	80 → 60643 [FIN, ACK] Seq=1056771 Ack=193 Win=6864 Len=0 TSval=4016897548 TSecr=256683677
1172	11:34:45.638362	192.168.1.122	64.238.147.113	TCP	66	60643 → 80 [ACK] Seq=193 Ack=1056772 Win=524200 Len=0 TSval=256683764 TSecr=4016897548

Admnowledgment (tcp.flags.ack), 1 byte

Packets: 1172 · Displayed: 1172 (100.0%) · Load time: 0:0.74 · Profile: Default

Week 12: Using RSA Algorithm Encrypt a Text data and Decrypt the same.

Program:

```
import java.io.DataInputStream;
import java.io.IOException;
import java.math.BigInteger;
import java.util.Random;
public class RSA
{
    private BigInteger p;
    private BigInteger q;
    private BigInteger N;
    private BigInteger phi;
    private BigInteger e;
    private BigInteger d;
    private int    bitlength = 1024;
    private Random  r;

    public RSA()
    {
        r = new Random();
        p = BigInteger.probablePrime(bitlength, r);
        q = BigInteger.probablePrime(bitlength, r);
        N = p.multiply(q);
        phi = p.subtract(BigInteger.ONE).multiply(q.subtract(BigInteger.ONE));
        e = BigInteger.probablePrime(bitlength / 2, r);

        while (phi.gcd(e).compareTo(BigInteger.ONE) > 0 && e.compareTo(phi) < 0)
        {
            e.add(BigInteger.ONE);
        }
        d = e.modInverse(phi);
    }

    public RSA(BigInteger e, BigInteger d, BigInteger N)
    {
        this.e = e;
        this.d = d;
        this.N = N;
    }

    @SuppressWarnings("deprecation")
    public static void main(String[] args) throws IOException
    {
        RSA rsa = new RSA();
        DataInputStream in = new DataInputStream(System.in);
        String teststring;
```

```

        System.out.println("Enter the plain text:");
        teststring = in.readLine();
        System.out.println("Encrypting String: " + teststring);
        System.out.println("String in Bytes: "
            + bytesToString(teststring.getBytes()));
// encrypt
    byte[] encrypted = rsa.encrypt(teststring.getBytes());
    // decrypt
    byte[] decrypted = rsa.decrypt(encrypted);
    System.out.println("Decrypting Bytes: " + bytesToString(decrypted));
    System.out.println("Decrypted String: " + new String(decrypted));
}

private static String bytesToString(byte[] encrypted)
{
    String test = "";
    for (byte b : encrypted)
    {
        test += Byte.toString(b);
    }
    return test;
}

// Encrypt message
public byte[] encrypt(byte[] message)
{
    return (new BigInteger(message)).modPow(e, N).toByteArray();
}
// Decrypt message
public byte[] decrypt(byte[] message)
{
    return (new BigInteger(message)).modPow(d, N).toByteArray();
}
}

```

Output:

```

$javac RSA.java
$ java RSA

```

```

Enter the plain text:
Sanfoundry
Encrypting String: Sanfoundry
String in Bytes: 8397110102111117110100114121
Decrypting Bytes: 8397110102111117110100114121
Decrypted String: Sanfoundry

```

